

Introduction to Business Analytics



Day 01

List of Contents

- ❑ What is Business Analytics?
- ❑ Types of Analytics
- ❑ Role of Data in Decision Making
- ❑ Data-driven vs Intuition-driven Business
- ❑ Industry Applications and Best Practices
- ❑ Lab / Demo: Sample Dataset

Learning Objectives

- ❑ Define Business Analytics and its scope
- ❑ Explain the four types of analytics
- ❑ Understand the role of data in business decisions
- ❑ Compare intuition vs data-driven approaches
- ❑ Explore industry applications
- ❑ Practice descriptive statistics on real data

Course Overview



Diploma in Data Science: Applied Focus



Mix of theory, labs, and projects



Industry-aligned curriculum



Capstone project at the end



Real-world datasets and case studies



Support for both technical & non-technical students

Why Study Business Analytics?

-  Data is the new oil – raw, but needs refining to create value
-  Competitive advantage – companies win by turning insights into strategy
-  Better decisions – improves accuracy and reduces guesswork
-  Business value & efficiency – optimizes costs, revenues, and resources
-  Career opportunities – rising global demand for analytics professionals
-  Foundation for advanced fields – builds pathway to Machine Learning & AI
-  Bridges business & technology – equips both technical & non-technical professionals

Skills You Will Gain



Data cleaning & preparation – organizing raw data for analysis



Using tools – Excel, Python, Power BI for business problems



Business metrics & KPIs – revenues, costs, profits, MAUs, growth rates



Basic statistics & analytics – histograms, segmentation, user economics



Building predictive models – e.g., churn analysis, supervised segmentation



Communicating insights – formal reports, memos, presentations to business users



Applied projects – real-world case studies & capstone work for portfolio

Expectations for Students





What is Business Analytics?

Business Analytics is the practice of using data, statistical methods, and analytical models to explore past business performance, gain insights, and support decision-making for future strategies.

Key Points:

Uses data to inform decisions

Combines statistics, technology, and business knowledge

Turns raw data into actionable insights

Guides efficiency, growth, and competitive advantage

Supports problem-solving, forecasting, and strategy

Applied across industries (retail, finance, healthcare, telecom)

Analytics vs Reporting vs BI

Reporting	Business Intelligence (BI)	Analytics
What happened? Static reports (tables, summaries) Ex: Monthly sales report	What is happening? Dashboards, KPIs, trends Ex: Real-time sales dashboard	Why + What's next? Predictive & prescriptive insights Ex: Sales drop → cause analysis
Focus: Past	Focus: Present	Focus: Future
Descriptive view	Monitoring & visualization	Decision-making & forecasting

Real-World Applications of Business Analytics

Amazon	Netflix	Banking
Product recommendations	Personalized recommendations	Fraud detection
'Customers also bought' feature	Predicts show success	Credit risk scoring
Uses predictive analytics	A/B testing for thumbnails	Customer segmentation
Boosts sales & loyalty	Improves engagement	Predicts loan defaults
Personalized shopping journey	Analytics drives content strategy	Improves trust & safety
Huge revenue impact	Data fuels creativity	Saves millions in losses

Data Cycle



Raw data collected



Insights extracted



Decisions made



Actions executed



Continuous improvement cycle

Data Analytics Types



Four Types of Analytics (Overview)

Descriptive → What happened?

(e.g., Last quarter's sales were \$2M)

Diagnostic → Why did it happen?

(e.g., Sales dropped due to customer churn)

Predictive → What will happen?

(e.g., Forecast shows 10% sales growth next quarter)

Prescriptive → What should we do?

(e.g., Launch retention campaign to reduce churn)

Descriptive Analytics



Focuses on past data and historical trends across business activities



Answers the question: “What happened?” clearly and factually



Uses tools like dashboards, reports, KPIs, and visualizations to summarize results



Example: Quarterly sales dropped by 10% compared to the previous quarter



Serves as the easiest entry point into analytics, requiring minimal complexity



Provides a clear baseline for comparison and performance measurement



Helps businesses track progress and spot patterns in key metrics



Builds the foundation for deeper analytics like diagnostic, predictive, and prescriptive

Diagnostic Analytics

Focus: causes, root factors, and explanations behind results

Answers: “Why did it happen?” by identifying drivers of change

Tools: drill-downs, correlations, statistical analysis, data mining

Example: Sales dropped because a competitor launched aggressive discounts

More complex than descriptive: requires deeper data exploration

Useful for finding patterns, relationships, and anomalies in data

Predictive Analytics

Focus: forecasting future outcomes and trends

Answers: “What is likely to happen?” based on data patterns

Tools: machine learning models, regression, classification, time-series forecasting

Example: Predicting customer churn or forecasting demand for next quarter

Relies on historical data and statistical correlations to make predictions

Produces probability-based outcomes (not certainty, but likelihoods)

Helps organizations anticipate risks and opportunities in advance

Prescriptive Analytics

- ❑ Focus: recommended actions and best possible strategies
- ❑ Answers: “What should we do?” in order to achieve desired outcomes
- ❑ Tools: optimization models, simulations, decision trees, scenario analysis
- ❑ Example: Suggesting a discount strategy or loyalty program to retain high-value customers
- ❑ Guides decision-making by evaluating multiple possible options
- ❑ Provides actionable recommendations rather than just predictions
- ❑ Considered the most advanced stage of analytics, combining descriptive, diagnostic, and predictive insights

Decision Making

What is Decision Making

The process of choosing between alternatives to achieve goals

In business: deciding on strategies, investments, operations, and customer actions

Core part of business analytics: analytics provides insights → decision-making applies them

Good decisions = better efficiency, competitiveness, and growth

Two main approaches:

- ❑ Traditional Decision-Making → based on intuition & experience
- ❑ Data-Driven Decision-Making → based on data, analytics, and evidence

Traditional Decision-Making

Relies heavily on intuition, gut feeling, and personal experience

High risk of personal bias and subjectivity

Often inconsistent and difficult to replicate

Works reasonably well in small-scale or less competitive businesses

Struggles in complex, fast-changing markets with many variables

Limited scalability and adaptability as business grows

Provides quick decisions, but not always the most accurate ones

What is Data Driven Decision Making



Uses facts, metrics, and data analysis instead of intuition



Answers business questions with evidence-based insights



Relies on tools: dashboards, BI, analytics, and machine learning



Ensures consistency and accuracy across decisions



Scales easily in large and complex organizations



Supports faster response to market changes



Drives innovation, efficiency, and competitive advantage



Examples: Netflix recommending shows, Amazon optimizing pricing, banks detecting fraud

Netflix Case Study (Steps)

Step 1: Collect Data

Netflix tracks viewing behavior, searches, ratings, and watch time.

Step 2: Identify Demand

Data showed high interest in political dramas.

Step 3: Use Data to Guide Content

Netflix created 'House of Cards' based on analytics.

Step 4: Personalize Experience

Recommendations based on each user's profile and history.

Step 5: Test and Optimize

Runs A/B testing on thumbnails and visuals to increase clicks.

Step 6: Drive Outcomes

Improved engagement, retention, and revenue — analytics fuels creativity.

Benefits of Data-Driven Approach

More accurate decisions → based on facts, not guesswork

Reduced risk of **errors** → minimizes human bias and subjectivity

Faster response to changes → real-time data enables agility in dynamic markets

Improves customer satisfaction → personalization and targeted strategies enhance user experience

Competitive advantage → companies using analytics outperform intuition-driven rivals

Encourages innovation → data reveals new opportunities and inspires fresh ideas

Supports scalability → decisions can adapt as business grows and markets expand

Builds accountability → decisions can be traced back to data, improving trust

Challenges of Data-Driven Approach

Data quality issues → incomplete, inaccurate, or outdated data can mislead decisions

High implementation costs → tools, infrastructure, and skilled staff can be expensive

Skill gaps → shortage of data-literate employees and analysts

Data silos → information spread across departments limits integration

Privacy & security concerns → handling sensitive data responsibly is critical

Change resistance → employees may prefer intuition over analytics-driven methods

Over-reliance on data → risk of ignoring context, creativity, or human judgment

Intuition-Based Decision-Making: Pros & Cons

Pros	Cons
Quick decisions in urgent or uncertain situations	High risk of bias and subjectivity
Leverages personal experience and knowledge	Inconsistent and hard to replicate
Useful when data is unavailable or incomplete	May ignore facts or evidence → poor outcomes
Can foster creativity and intuition	Struggles in complex, data-rich environments
Works well in small-scale or low-risk environments	Limited scalability as organizations grow

Case: Nokia vs Apple



Nokia relied on past success and intuition



Failed to adapt → missed the smartphone revolution



Apple studied consumer behavior & market data



Focused on design, usability, and customer needs



Decisions informed by analytics and user insights



Outcome: Apple dominated the smartphone market, while Nokia declined

Industry Applications of Business Analytics

Retail

- Demand forecasting
 - Inventory optimization
 - Personalized promotions
 - Pricing strategy
 - Customer segmentation
- Example: Walmart, Amazon

Healthcare

- Disease prediction
 - Patient monitoring (wearables)
 - Personalized treatment
 - Hospital resource optimization
 - Early detection saves lives
- Example: Diabetes risk

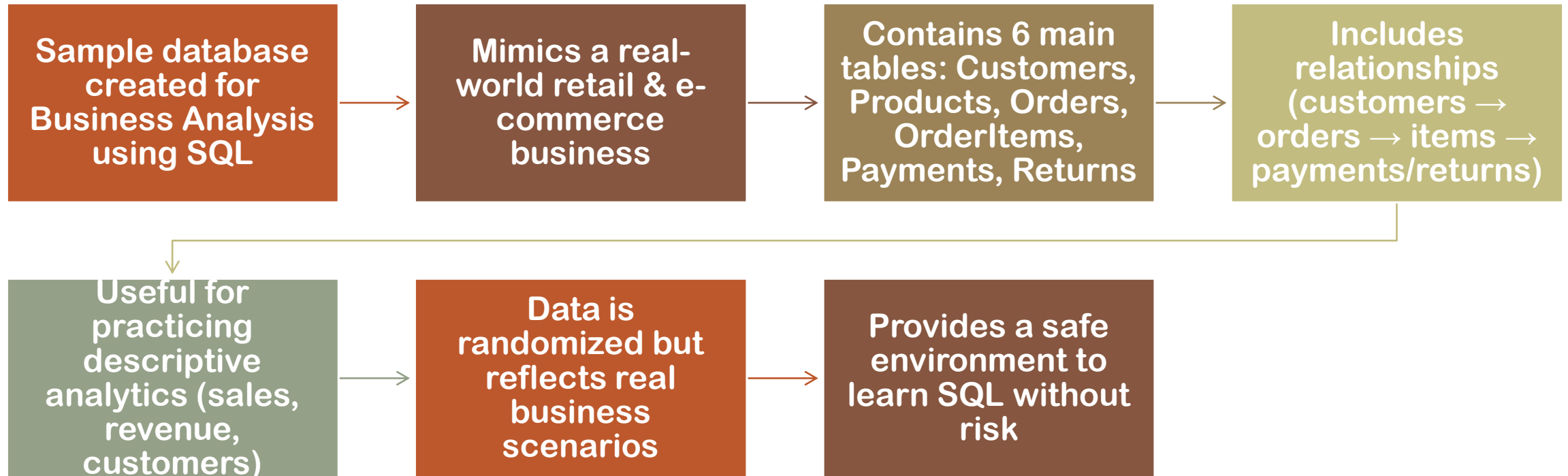
Finance

- Credit scoring
 - Fraud detection
 - Algorithmic trading
 - Customer segmentation
 - Risk management
- Example: Unusual card activity

Telecom

- Churn prediction
 - Network optimization
 - Customer segmentation
 - Targeted offers
 - Reduce customer loss
- Example: AT&T, Vodafone

Importing SQL



DataSet Overview

1. Open SQL Server Management Studio (SSMS) and connect to your server.
2. Go to File → Open → File.
3. Select the .sql file provided and click Open.
4. Ensure you are connected to the correct database.
5. Click Execute (or press F5) to run the script.
6. Check Messages tab for 'Query executed successfully'.
7. Refresh Object Explorer to confirm that tables and data are imported.

Steps to Import SQL File

Steps to Restore .bak File in SSMS

Open SQL Server Management Studio (SSMS) and connect to your server.

In Object Explorer, right-click Databases → Restore Database.

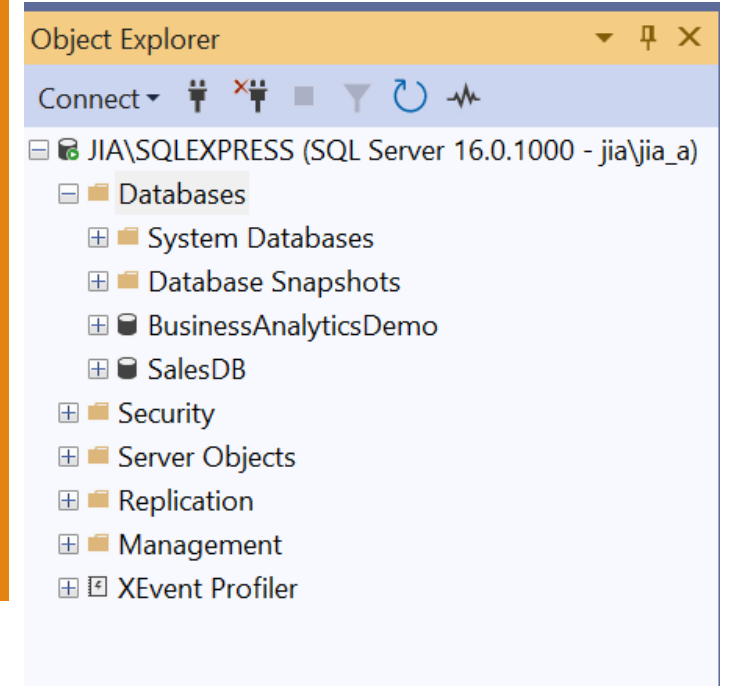
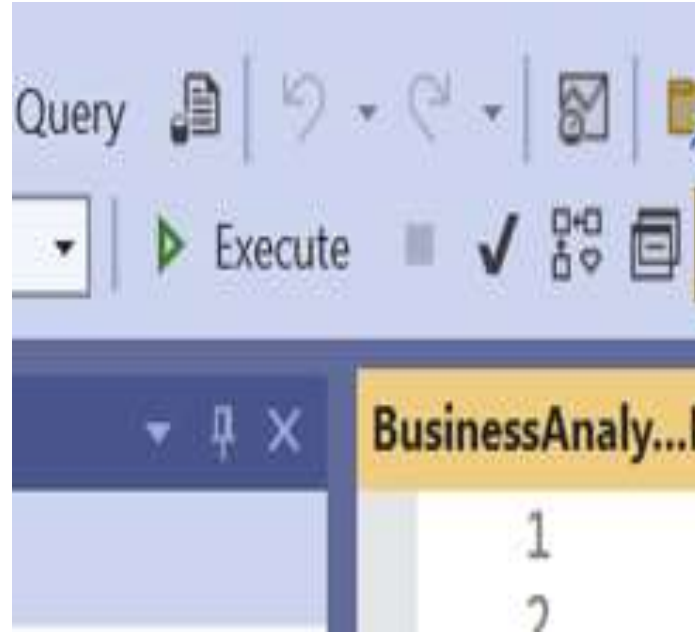
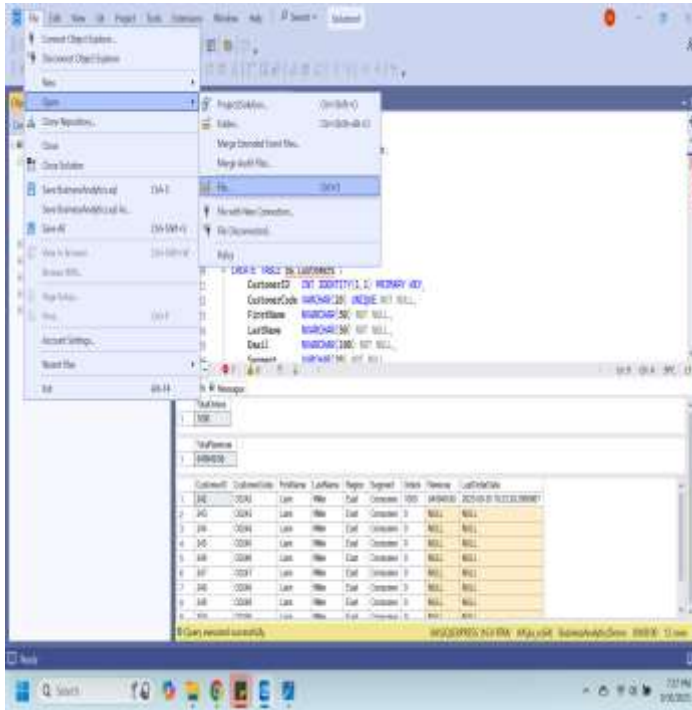
Select Device, then click ... and browse to your .bak file
(e.g., C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\Backup).

Check the box beside your backup file and click OK.

Choose a Database name (e.g., BusinessAnalyticsDemo) for restore.

Click OK to begin restoring — wait for the success message.

Refresh Object Explorer to confirm all tables and data are restored.



STEPS TO IMPORT SQL FILE

Why SQL for Business Analysis?

We can analyze data using Excel (formulas) and Python (code)

Both tools help us calculate averages, totals, and summaries

But real business data is often large and stored in databases

SQL is the industry standard for querying and analyzing such data

Same concepts (AVG, SUM, MAX, MIN) but scalable to millions of rows

In this course, we will focus on Business Analysis using SQL

Analysis Using SQL

AVG(LineTotal) → Finds the average sales amount (what a “typical” order looks like).

MAX(LineTotal) → Returns the highest sales value (the most expensive order).

MIN(LineTotal) → Returns the lowest sales value (the smallest order).

COUNT(*) → Counts the total number of orders in the database.

SUM(LineTotal) → Adds up all sales amounts to show the total revenue.

These functions give us descriptive statistics—quick business insights from raw data

Analysis Using SQL

```
-- Average, Max, Min revenue
SELECT
    AVG(LineTotal) AS Avg_Revenue,
    MAX(LineTotal) AS Max_Revenue,
    MIN(LineTotal) AS Min_Revenue
FROM ba.OrderItems;

-- Count total orders
SELECT COUNT(*) AS Total_Orders
FROM ba.Orders;

-- Sum total revenue
SELECT SUM(LineTotal) AS Total_Revenue
FROM ba.OrderItems;
```

100 % No issues found

Results Messages

	Avg_Revenue	Max_Revenue	Min_Revenue
1	840.949980	1417.65	283.53

	Total_Orders
1	1000

	Total_Revenue
1	840949.98

Q&A



Translating Business Needs into Analytics



Day 02

List of Contents

- ❑ What is problem framing?
- ❑ Importance of stakeholders in analytics
- ❑ From vague to structured problem statements
- ❑ Problem Metrics and Solution framework
- ❑ SQL hands-on demos

Learning Objectives

- ❑ Explain why framing problems is essential
- ❑ Collaborate effectively with stakeholders
- ❑ Convert vague business needs into structured statements
- ❑ Define metrics aligned with business goals
- ❑ Propose data-driven solutions using SQL insights
- ❑ Apply the framework to real-world cases

Why Problem Framing Matters



Prevents wasted effort → Without clear framing, analysts may spend weeks exploring irrelevant data, producing insights that don't solve the real issue.



Aligns with business strategy → A well-framed problem connects analytics work to company goals (e.g., increasing sales, reducing costs).



Focuses on root causes, not symptoms → Instead of just reacting to declining sales, framing helps uncover why sales dropped (e.g., poor customer retention).



Ensures measurable outcomes → Clear framing defines success metrics (KPIs) so we can measure progress and prove impact.



Builds trust with stakeholders → When stakeholders see that analysts understand the business context, collaboration becomes stronger.

Why Problem Framing Matters

Doctors never prescribe medicine without diagnosis
→ They first understand the patient's condition.

Business analysts must also diagnose the problem before proposing solutions → Jumping straight to solutions risks solving the wrong issue.

Problem framing = the diagnosis step → Identifying the real business challenge (e.g., “low customer retention” vs. “low sales”).

Metrics & SQL analysis = medical tests → Just like blood tests reveal hidden health issues, data analysis uncovers hidden business patterns.

Solutions = treatment plan
→ After diagnosis and tests, the analyst recommends the best course of action for the business.

Common Pitfalls Without Framing

Solving the wrong problem



Misaligned stakeholder expectations



Choosing metrics with no business meaning



Producing insights that no one uses



Team frustration & wasted resources

Importance of Stakeholders

Stakeholders in Analytics



Who are they? Executives, managers, data teams, IT staff, and sometimes even customers.



What they provide: Domain knowledge, real-world business context, and insights analysts can't get from data alone.



Their role in decision-making: They set priorities, budgets, and constraints that shape the analysis.



Why they matter: Without stakeholder involvement, analysis may solve the wrong problem or be ignored.



Partnership mindset: Analysts + stakeholders must work together for analysis to drive action.

Example of Stakeholder Input

Initial Problem: “Sales are low”

Clarifying Questions:

- Which product lines?
- Which regions?
- Which time period?

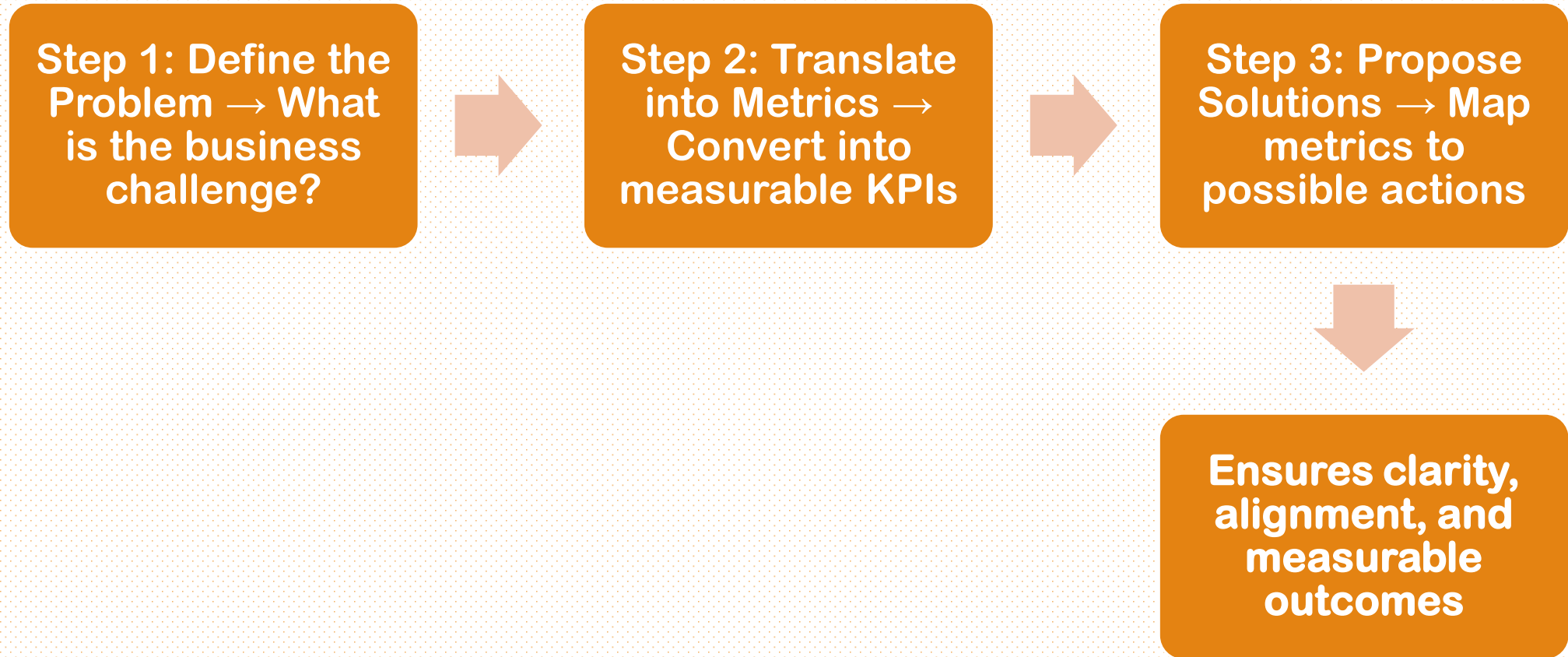
Refined Problem:

“Electronics sales in Region A dropped 20% in Q2”

Vague vs Structured Problem

Vague Problem	Structured Problem
“We want more customers”	“Increase online active users by 15% in Q2”
Unclear, not measurable	Specific, Measurable, Time-bound, Actionable

The Problem-Solving Framework



Case Study: Online Subscription Service

Problem:

High churn among premium customers

Metric:

Churn rate = % of customers cancelling within 3 months

Solutions:

- ☐ Improve onboarding experience
- ☐ Launch loyalty rewards program
- ☐ Enhance customer support

Analytics in Different Industries

Industry	Problem	Solution
Netflix 🎬	High churn (customers leaving)	Personalized recommendations
Amazon 🛒	Low sales in electronics	Cross-selling & promotions

Banking 🏦 – Problem: Loan defaults → Solution: Credit risk scoring

Healthcare 🏥 – Problem: Long patient wait times → Solution: Optimized scheduling

Hands On SQL

SQL Recap

Concept	Explanation
SQL (Structured Query Language)	Standard language to extract, summarize, and analyze data from relational databases
Why Analysts Use SQL	To answer questions like “What are total sales?” or “Which region performs best?”
SQL in Business Analytics	Used for cleaning, filtering, aggregating, and combining data for reports and dashboards
Data Model	Data is stored in tables (rows = records, columns = fields)
Focus of This Course	Learning to use SQL for analysis, not database administration

SQL Building Blocks

Command	Purpose / Description	Syntax Example
SELECT	Choose which columns to display in your results	SELECT column1, column2 FROM table_name;
FROM	Specify the table name	SELECT * FROM Customers;
WHERE	Filter data based on conditions	SELECT * FROM Orders WHERE City = 'Toronto';
GROUP BY	Summarize data by categories	SELECT City, SUM(Sales) FROM Orders GROUP BY City;
HAVING	Filter grouped data (used after GROUP BY)	SELECT City, SUM(Sales) FROM Orders GROUP BY City HAVING SUM(Sales) > 50000;
ORDER BY	Sort results ascending or descending	SELECT * FROM Products ORDER BY Price DESC;
TOP	Limit the number of returned rows	SELECT TOP 5 * FROM Customers;

Filtering Data

Command / Example	Explanation / Business Use	Syntax Example
WHERE City = 'Toronto'	Filters data for customers located in Toronto	SELECT * FROM Customers WHERE City = 'Toronto';
WHERE Amount > 1000	Shows only high-value orders	SELECT * FROM Orders WHERE Amount > 1000;
LIKE '%Phone%'	Finds any product name containing the word “Phone”	SELECT * FROM Products WHERE ProductName LIKE '%Phone%';
BETWEEN '2024-01-01' AND '2024-03-31'	Filters orders within Q1 2024	SELECT * FROM Orders WHERE OrderDate BETWEEN '2024-01-01' AND '2024-03-31';
Use Case	Targeting specific regions, time periods, or customer segments	

Sorting & Limiting Results

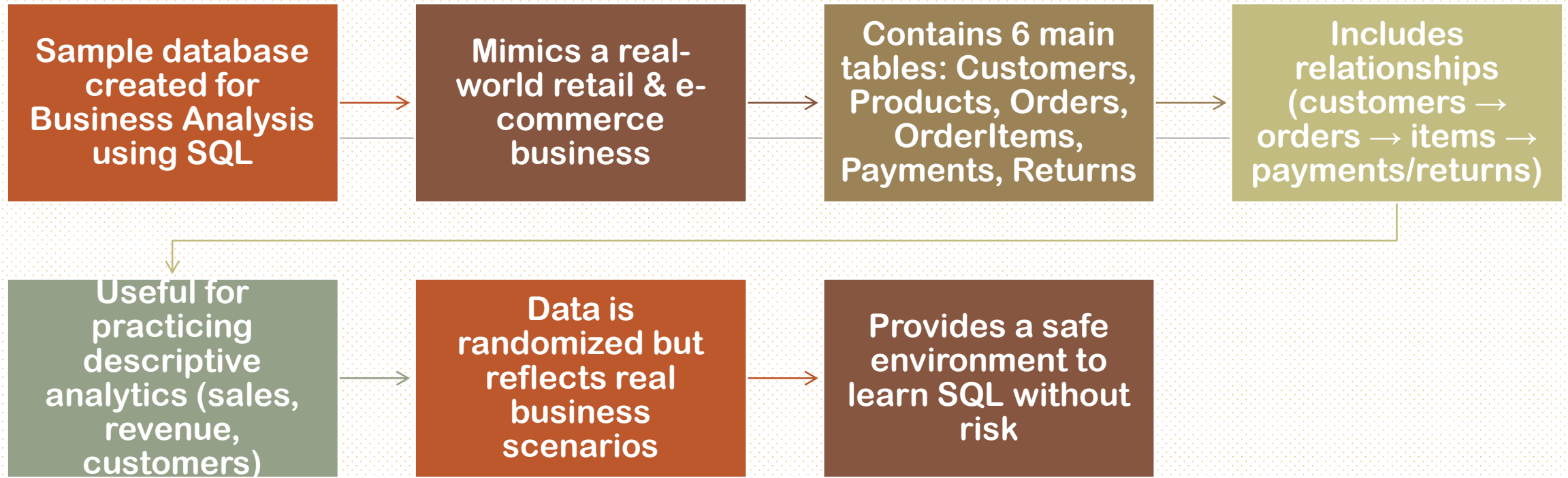
Command / Example	Explanation / Business Use	Syntax Example
ORDER BY Revenue DESC	Sorts results from highest to lowest revenue	SELECT * FROM Sales ORDER BY Revenue DESC;
ORDER BY OrderDate ASC	Lists orders chronologically	SELECT * FROM Orders ORDER BY OrderDate ASC;
SELECT TOP 10 * FROM Orders	Shows the first 10 records – quick data preview	SELECT TOP 10 * FROM Orders ORDER BY OrderDate DESC;
Use Case	Identify top-selling products or recent transactions	

Aggregate Functions

Function / Example	Purpose / Business Insight
COUNT(*)	Counts number of rows (e.g., total orders)
SUM(LineTotal)	Calculates total revenue or sales
AVG(LineTotal)	Finds average order value
MAX(LineTotal)	Highest order or maximum sale
MIN(LineTotal)	Smallest order value
Use Case	Used for KPIs – total revenue, average sale, churn rate, etc.

Joins Overview

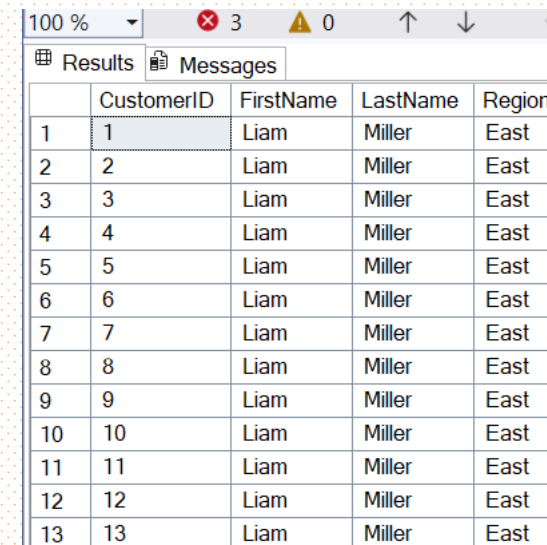
Join Type / Example	Purpose / Explanation
INNER JOIN	Combines rows from two tables where matches exist
LEFT JOIN	Keeps all records from the left table, even if no match
RIGHT JOIN	Keeps all records from the right table
FULL JOIN	Combines all records from both sides
Example	<code>SELECT c.CustomerName, o.OrderDate FROM Customers c JOIN Orders o ON c.CustomerID = o.CustomerID;</code>
Use Case	Connects customers, orders, and payments to create full business views



Database Overview

Filtering Row-Where

```
-- PURPOSE: Show all customers from the East region
SELECT CustomerID, FirstName, LastName, Region -- Pick useful customer columns
FROM ba.Customers -- From the Customers table
WHERE Region = 'East'; -- Only customers in East
```



The screenshot shows a SQL query results window with a toolbar at the top displaying '100 %', '3' errors, '0' warnings, and navigation arrows. Below the toolbar are tabs for 'Results' and 'Messages'. The 'Results' tab is active, showing a table with 5 columns: an index, CustomerID, FirstName, LastName, and Region. The table contains 13 rows of data, all with the region 'East'.

	CustomerID	FirstName	LastName	Region
1	1	Liam	Miller	East
2	2	Liam	Miller	East
3	3	Liam	Miller	East
4	4	Liam	Miller	East
5	5	Liam	Miller	East
6	6	Liam	Miller	East
7	7	Liam	Miller	East
8	8	Liam	Miller	East
9	9	Liam	Miller	East
10	10	Liam	Miller	East
11	11	Liam	Miller	East
12	12	Liam	Miller	East
13	13	Liam	Miller	East

GROUP BY – Grouping data

```
-- PURPOSE: Count customers by region
SELECT Region, COUNT(*) AS NumCustomers -- Count customers in each region
FROM ba.Customers                      -- From the Customers table
GROUP BY Region;                       -- Group rows by Region
```

Results		Messages
	Region	NumCustomers
1	East	300

HAVING – Filtering groups

```
--*****--  
-- PURPOSE: Show only regions with more than 50 customers  
✓ SELECT Region, COUNT(*) AS NumCustomers -- Count customers per region  
FROM ba.Customers -- From the Customers table  
GROUP BY Region -- Group rows by Region  
HAVING COUNT(*) > 50; -- Only keep groups with > 50 customers
```

Results Messages		
	Region	NumCustomers
1	East	300

Simple JOIN

```
-- PURPOSE: List orders with customer names
SELECT o.OrderID, o.OrderDate,      -- Order details
       c.FirstName, c.LastName, c.Region -- Customer details
FROM ba.Orders o                  -- Orders table
JOIN ba.Customers c              -- Join with Customers table
  ON o.CustomerID = c.CustomerID; -- Match customers to their orders
```

	OrderID	OrderDate	FirstName	LastName	Region
1	1	2025-07-16 19:23:29.2066667	Liam	Miller	East
2	2	2025-05-05 19:23:29.2066667	Liam	Miller	East
3	3	2024-10-20 19:23:29.2066667	Liam	Miller	East
4	4	2025-02-08 19:23:29.2066667	Liam	Miller	East
5	5	2025-08-15 19:23:29.2066667	Liam	Miller	East
6	6	2025-01-27 19:23:29.2066667	Liam	Miller	East
7	7	2025-03-05 19:23:29.2066667	Liam	Miller	East
8	8	2024-11-27 19:23:29.2066667	Liam	Miller	East
9	9	2025-09-18 19:23:29.2066667	Liam	Miller	East
10	10	2025-02-05 19:23:29.2066667	Liam	Miller	East

Join + Aggregation

```
-- PURPOSE: Show total sales by Region

SELECT
    c.Region,
    SUM(oi.LineTotal) AS TotalSales
FROM ba.Orders o
JOIN ba.Customers c
    ON c.CustomerID = o.CustomerID
JOIN ba.OrderItems oi
    ON oi.OrderID = o.OrderID
-- WHERE o.Status <> 'Cancelled'
GROUP BY c.Region
ORDER BY TotalSales DESC;
```

-- Take the region information from the Customers table
-- Add up all sales amounts (quantity × price)
-- Connect orders to customers (to know region)
-- Connect orders to their items (for sales amount)
-- Exclude cancelled orders (removed for now to avoid empty result)
-- Group results so each region has one row
-- Sort so the region with highest sales comes first

Results		Messages
	Region	TotalSales
1	East	840949.98

Best Practices

Best Practices for Framing Problems



Involve stakeholders early



Ask clarifying questions



Write structured problem statements



Tie problems to measurable metrics



Validate metrics before solutions

Checklist for Analysts

Clarity of the Problem

Is the business challenge clearly defined and specific?

Business Relevance

Do we understand the business impact and why it matters?

Measurable Success

Which KPIs/metrics define success? Are they measurable?

Data Availability

Do we have the right SQL queries, tables, and data sources to analyze it?

Solution Feasibility

Are the proposed solutions realistic, actionable, and aligned with constraints?

Stakeholder Alignment

Have stakeholders agreed on the problem, metrics, and success criteria?

Role of Communication in Analytics

- ✦ Translate insights into business language – move from technical output → actionable story
- ⊖ Avoid jargon-heavy explanations – keep it simple, clear, and relatable
- 💡 Use examples and analogies – e.g., instead of 'Churn=22%', say '1 in 5 customers leaves in 3 months'
- 🎯 Tailor message to the audience – executives want strategy, managers want operations, IT wants data details
- ☐ Outcome: Builds trust and drives action – clear communication ensures insights are implemented



Common Mistakes to Avoid

- ☐ Jumping to solutions without metrics
- ☐ Using irrelevant data
- ☐ Ignoring stakeholder context
- ☐ Overloading with technical detail
- ☐ Not validating results

What is CRISP-DM?

CRISP-DM = Cross-Industry Standard Process for Data Mining

Widely used framework for data science and analytics projects

Consists of 6 phases (iterative, not linear):

- 1. Business Understanding**
- 2. Data Understanding**
- 3. Data Preparation**
- 4. Modeling**
- 5. Evaluation**
- 6. Deployment**



Problem Framing in CRISP-DM

Business Understanding = Step 1 of CRISP-DM

Define objectives, constraints, and success criteria

Analytics must align with framed business problem

Without framing, later steps risk misalignment

Example: Goal = Reduce churn → Metric = Monthly churn rate → Solution = Improve onboarding

Summary

Problem framing = foundation of analytics

Stakeholders = source of knowledge

Metrics convert problems into measurable form

SQL provides insights, not answers

Solutions require collaboration

Q&A



Data Quality Cleaning and Preparation



Day 03

List of Contents

- ❑ Types of Business Data
- ❑ Data Sources in SQL Context
- ❑ Data Quality Dimensions
- ❑ Common Issues in SQL Data
- ❑ Handling Missing Data
- ❑ Duplicates, Outliers, Transformations

Learning Objectives

- ❑ Understand role of Data Understanding & Preparation
- ❑ Recognize common data quality issues in SQL databases
- ❑ Apply SQL to detect and fix missing values
- ❑ Use SQL to handle duplicates and outliers
- ❑ Clean and transform business data using SQL queries
- ❑ Create new features with SQL for analysis
- ❑ Split data into training/testing using SQL

CRISP-DM-RECAP



Six iterative phases from Business Understanding → Deployment



Data Understanding = exploring, detecting problems, summarizing data



Data Preparation = cleaning, transforming, restructuring for analysis



70–80% of project time is often spent here



Poor preparation leads to unreliable insights



Key principle: Garbage In → Garbage Out

Why Data Understanding is Critical

Ensures analysts know what each field means

Detects anomalies, missing values, duplicates early

Identifies relationships across tables (joins, keys)

Helps choose correct SQL operations for analysis

Prevents blind analysis without business context

Foundation for reliable insights and decisions

Why Data Cleaning is Important

Raw business data is messy, inconsistent, incomplete

Dirty data leads to misleading reports and KPIs

Cleaning ensures accuracy in business metrics

Enables fair comparisons with standardized values

Essential before SQL joins and aggregations

No cleaning = no trust in analysis results

Impact of Ignoring Data Cleaning



Wrong strategies from misleading insights



Over/under-estimated sales due to duplicates



Missed fraud detection from ignored anomalies



Poor model accuracy due to unhandled NULLs



Loss of stakeholder trust in analytics team



Costly business errors and financial losses

Importance for Data Scientists



70–80% of data science work is data preparation



SQL is critical for enterprise data cleaning tasks



Well-prepared data leads to faster, better modeling



Builds trust with stakeholders and executives



Separates strong data scientists from average ones



Quote: 'Data wrangling > Modeling in real life'



Real-World Data Quality Failures

Target Canada: Inconsistent product data → stockouts → \$2B loss

NHS UK: Duplicate IDs merged wrong patient records

Banking: Fraud missed due to unflagged anomalies

Poor data quality directly caused business failures

Lesson: Reliable data quality = survival

Why Data Preparation Matters in SQL

- ❑ SQL is backbone of enterprise analytics systems
- ❑ Clean data enables accurate joins and aggregations
- ❑ Prepped datasets drive better dashboards and KPIs
- ❑ Reduces time wasted in repeated fixes
- ❑ Improves collaboration between IT and business
- ❑ Bridges problem framing and reliable insights

Detecting Missing Data

Use SQL WHERE to find NULLs in important fields

Example: Missing Age or Region in Customers

Biases analysis if ignored

Must be corrected before reporting

Types of Data in Business

Type of Data	SQL Data Type	Examples in Business
Structured	Tables (rows/columns)	Customer records, Orders
Categorical	VARCHAR	Country, Gender
Numerical	INT, DECIMAL	Sales, Quantity
Date/Time	DATE, DATETIME	Order dates, Signup dates
Semi-structured	JSON, XML	Nested product attributes, Metadata
SQL Strength	Structured data	Strongest use case for SQL

Business Data Sources

Source	Example	Business Use
CRM Systems	Salesforce, HubSpot	Manage leads, customers
ERP Systems	SAP, Oracle	Orders, inventory, suppliers
E-commerce	Shopify, Amazon	Transactions, payments
Financial Systems	QuickBooks, SAP Finance	Accounts, invoices
Ops Logs	FedEx, DHL	Shipments, tracking
SQL Role	Schemas	Integrates all sources for analysis

Data Quality Dimensions



Accuracy – Are stored values correct?



Completeness – Are fields missing? (NULLs)



Consistency – Are labels standardized?



Timeliness – Is data recent and relevant?



Validity – Does it respect business rules? (Age > 0)



Uniqueness – Are IDs duplicated?

Data Quality Issues in SQL

Common SQL Data Issues

Missing values (NULLs) in critical fields

Duplicates from multiple entries

Inconsistent formats (USA vs United States)

Outliers like negative prices or extreme quantities

Invalid dates (string vs DATE format)

Imbalanced categories (e.g., churn data)

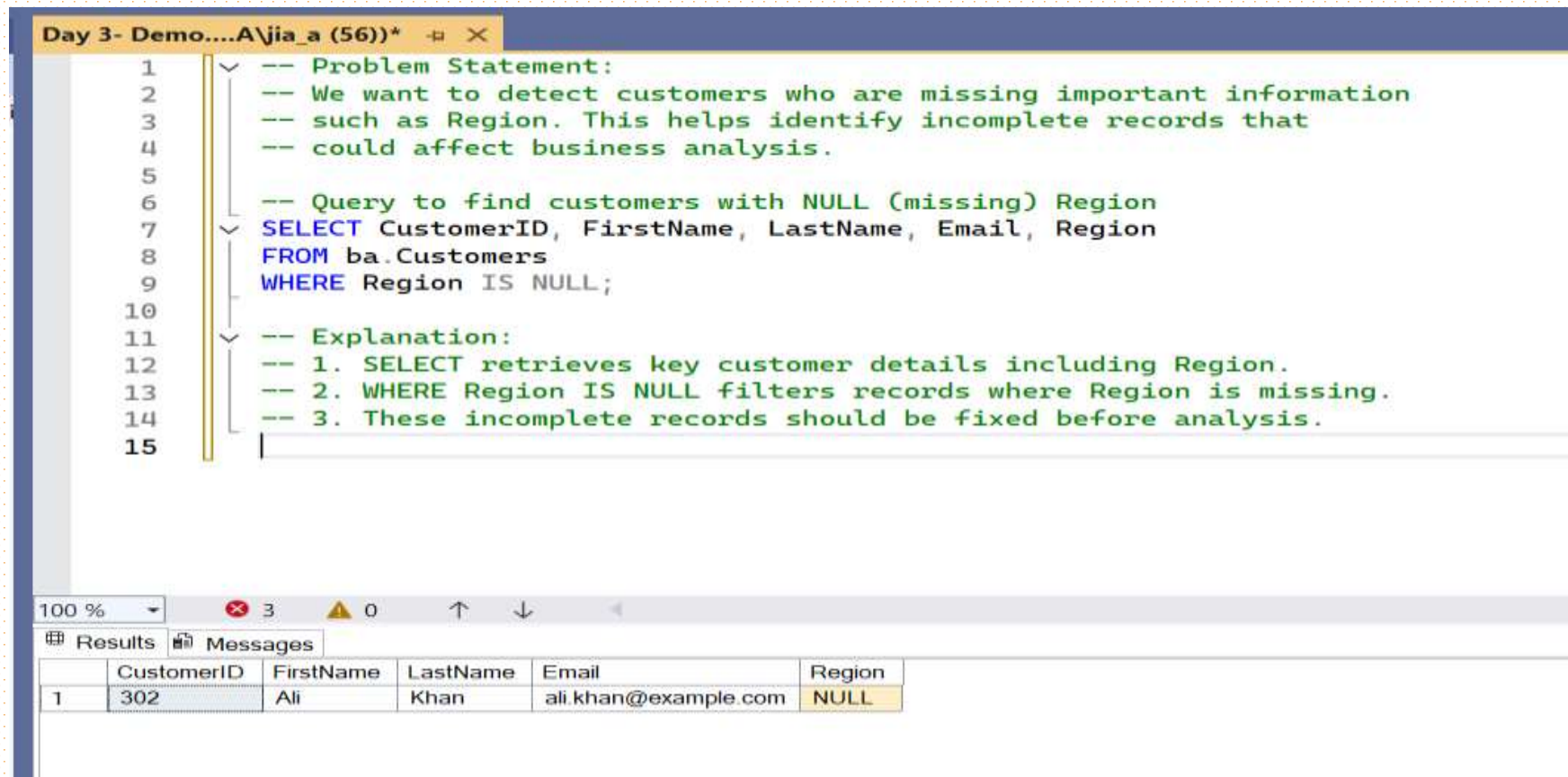
Detecting Missing Data (Concept)

- ❑ SQL stores missing values as NULL (not zero or empty)
- ❑ Ignoring them can distort totals, averages, and reports
- ❑ Detect using IS NULL, COUNT(*), and pattern checks

Handling options:

- ❑ Drop rows/columns with many NULLs
- ❑ Replace with defaults ('Unknown', 0)
- ❑ Impute with averages, medians, or rules
- ❑ Example: Missing country in customer data → weak segmentation
- ❑ Ensures reliable, unbiased business analysis

SQL Example: Detect Missing Data



The screenshot shows a SQL IDE window titled "Day 3- Demo....A\jia_a (56))*". The main editor contains the following SQL query and comments:

```
1  -- Problem Statement:
2  -- We want to detect customers who are missing important information
3  -- such as Region. This helps identify incomplete records that
4  -- could affect business analysis.
5
6  -- Query to find customers with NULL (missing) Region
7  SELECT CustomerID, FirstName, LastName, Email, Region
8  FROM ba.Customers
9  WHERE Region IS NULL;
10
11 -- Explanation:
12 -- 1. SELECT retrieves key customer details including Region.
13 -- 2. WHERE Region IS NULL filters records where Region is missing.
14 -- 3. These incomplete records should be fixed before analysis.
15
```

Below the editor, the "Results" tab is active, displaying a table with the following data:

	CustomerID	FirstName	LastName	Email	Region
1	302	Ali	Khan	ali.khan@example.com	NULL

Handling Missing Data (Concept)

- ❑ Deletion: Drop rows/columns with many NULLs (risk: data loss)
- ❑ Default Replacement: Use 'Unknown' or 0 (fast, but may bias)
- ❑ Imputation: Replace with Mean, Median, or Mode (balances accuracy & simplicity)
- ❑ Business Logic: Fill using related fields or domain rules
- ❑ SQL Tip: Use COALESCE() or CASE WHEN for replacements
- ❑ Why Important: Ensures reliable results before aggregations & joins

SQL Example: Replacing Missing Values

```
20
21  -- Problem Statement:
22  -- We want to fix missing values in the Customers table.
23  -- Some customers may not have their Region recorded.
24  -- Instead of leaving them NULL, we replace with 'Unknown'.
25
26  -- Query: Replace NULL Regions with 'Unknown'
27  UPDATE ba.Customers
28  SET Region = 'Unknown'
29  WHERE Region IS NULL;
30
31  -- Explanation:
32  -- 1. UPDATE modifies the table data.
33  -- 2. SET Region = 'Unknown' replaces the NULL with a default value.
34  -- 3. WHERE Region IS NULL ensures we only update missing entries.
35
```

100 % 3 0

Messages

(1 row affected)

Completion time: 2025-10-01T20:36:32.1567309-04:00

Duplicates in SQL (Concepts)

Occur when the same row repeats (e.g., same email or ID)

Causes: manual entry, system imports, missing constraints

Problems: inflated counts, wrong KPIs, wasted costs

Detection: GROUP BY, DISTINCT, ROW_NUMBER()

Handling: keep latest (MAX), delete extras, enforce UNIQUE

Preventing duplicates is key for accurate reporting

SQL Example: Detect & Remove Duplicates

```
-- Problem Statement:  
-- Some customers may appear more than once in the Customers table  
-- because of duplicate Email IDs.  
-- Duplicates cause inflated counts and wrong metrics, so we must detect  
-- and remove them.
```

```
-- Step 1: Detect duplicates based on Email
```

```
SELECT Email, COUNT(*) AS CountDup  
FROM ba.Customers  
GROUP BY Email  
HAVING COUNT(*) > 1;
```

```
-- Explanation:
```

```
-- GROUP BY groups all rows by Email.  
-- COUNT(*) counts how many times each email appears.  
-- HAVING COUNT(*) > 1 shows only those with duplicates.
```

Results Messages		
	Email	CountDup
1	duplicate@example.com	3

```

-----
-- Step 2: Remove duplicates, keeping only the first CustomerID
DELETE FROM ba.Customers
WHERE CustomerID NOT IN (
    SELECT MIN(CustomerID)
    FROM ba.Customers
    GROUP BY Email
);
-- Explanation:
-- MIN(CustomerID) picks the earliest record for each Email.
-- NOT IN ensures we delete all other duplicates.
-- This way, one clean record per Email remains.

```

```

36
37 -- Step 1: Detect duplicates based on Email
38 SELECT Email, COUNT(*) AS CountDup
39 FROM ba.Customers
40 GROUP BY Email
41 HAVING COUNT(*) > 1;
42
43 -- Explanation:
44 -- GROUP BY groups all rows by Email.
45 -- COUNT(*) counts how many times each email appears.
46 -- HAVING COUNT(*) > 1 shows only those with duplicates.
47
48 -- Step 2: Remove duplicates, keeping only the first CustomerID
49 DELETE FROM ba.Customers
50 WHERE CustomerID NOT IN (
51     SELECT MIN(CustomerID)
52     FROM ba.Customers
53     GROUP BY Email
54 );
55 -- Explanation:
56 -- MIN(CustomerID) picks the earliest record for each Email.

```

0 % No issues found

Results Messages

Email	CountDup
AFTER REMOVING	

SQL Example: Detect & Remove Duplicates

Outliers in SQL (Concepts)

- ❑ Outliers = values far outside expected range (e.g., \$1M sale, age 250)
- ❑ Causes: fraud, data entry mistakes, rare events
- ❑ Impact: distorts averages, misleads KPIs and trends

Detection Methods:

- ❑ SQL thresholds (value $> 3 \times \text{avg}$)
- ❑ Statistical tests (Z-score, IQR)
- ❑ Domain rules (e.g., age must be < 120)

Handling:

Investigate, validate, or flag

Remove or adjust only if error is confirmed

Remember: Outliers may be errors or hidden insights

Outliers in SQL (Concepts)

Aspect	Details
Definition	Values far outside normal range (e.g., \$1M sale, age 250)
Causes	Fraud, data entry mistakes, rare events
Impact	Distorts averages, misleads KPIs and trends
Detection Methods	SQL thresholds ($> 3 \times \text{avg}$), Z-score, IQR, domain rules
Handling	Investigate, validate, flag; remove/adjust if confirmed error
Key Insight	Outliers may be errors OR valuable insights

SQL Example: Outlier Detection

```
61
62 -- Problem Statement:
63 -- In business transactions, sometimes we find extreme values
64 -- (outliers) in order amounts. These may indicate fraud, data entry errors,
65 -- or rare purchases. We want to detect orders where the total
66 -- purchase amount is unusually high.
67
68 -- Step 1: Calculate PurchaseAmount for each order item
69 -- Step 2: Compare it to the mean + 3*standard deviation rule
70 -- (a common statistical method for outlier detection)
71
72 SELECT oi.OrderID, o.CustomerID,
73        (oi.Quantity * oi.UnitPrice) AS PurchaseAmount
74 FROM ba.OrderItems oi
75 JOIN ba.Orders o ON oi.OrderID = o.OrderID
76 WHERE (oi.Quantity * oi.UnitPrice) > (
77     SELECT AVG(Quantity * UnitPrice) + 3*STDEV(Quantity * UnitPrice)
78     FROM ba.OrderItems
79 );
80
81 -- Explanation:
82 -- 1. We calculate PurchaseAmount = Quantity * UnitPrice.
83 -- 2. AVG + 3*STDEV defines a threshold (upper limit).
84 -- 3. Any order above this threshold is flagged as an outlier.
85
```

100 % No issues found

Results Messages

OrderID	CustomerID	PurchaseAmount
307	242	999000.00

Cleaning Inconsistent Data (Concepts)

- ❑ Inconsistent values: same meaning, different formats
- ❑ Example: 'CA', 'CND', 'Canada'
- ❑ Impact: incorrect aggregations and misleading reports

Fixing Methods:

- ❑ Standardize using SQL functions (TRIM(), UPPER(), REPLACE())
- ❑ Map variations to one canonical value via lookup tables
- ❑ Apply business rules for consistent categories
- ❑ Why Important: ensures accurate KPIs in regional & categorical analysis

SQL Example: Standardization

```
86
87  -- Problem Statement:
88  -- Customers may have inconsistent values for Region.
89  -- Example: 'CA', 'CND', 'Can.', 'CANADA' should all be standardized
90  -- to 'Canada' for consistent reporting.
91
92  -- Query: Standardize inconsistent Region values
93  UPDATE ba.Customers
94  SET Region = 'Canada'
95  WHERE Region IN ('CA', 'CND', 'Can.', 'CANADA');
96
97  -- Explanation:
98  -- 1. UPDATE modifies the Region values.
99  -- 2. SET assigns the clean standardized value ('Canada').
100 -- 3. WHERE ensures only inconsistent variations are updated.
101
```

100 % ✓ No issues found

Messages

(1 row affected)

Completion time: 2025-10-01T21:06:46.0478338-04:00



Data Transformation (Concepts)

- ❑ **Scaling:** normalize values for comparison (e.g., sales vs. age).
- ❑ **Binning:** convert continuous to categories (Age groups).
- ❑ **Encoding:** convert text categories to codes (Gender → 0/1).
- ❑ **Date Functions:** extract YEAR, MONTH, DAY from dates.
- ❑ **String Functions:** clean/standardize text (UPPER(), LOWER(), TRIM()).
- ❑ **Why Important:** Prepares raw data for accurate analysis & reporting.

SQL Example: Age Binning

```
102
103 -- Problem Statement:
104 -- We want to group customers into Age ranges for segmentation.
105
106 SELECT CustomerID, Age,
107        CASE
108            WHEN Age BETWEEN 18 AND 25 THEN '18-25'
109            WHEN Age BETWEEN 26 AND 35 THEN '26-35'
110            WHEN Age BETWEEN 36 AND 50 THEN '36-50'
111            ELSE '51+'
112        END AS AgeGroup
113 FROM ba.Customers;
114
115 -- Explanation:
116 -- 1. CASE assigns a category label based on Age.
117 -- 2. Customers are grouped into bins (18-25, 26-35, 36-50, 51+).
118 -- 3. Useful for demographic segmentation in analytics.
119
```

100 % 4 0

Results Messages

	CustomerID	Age	AgeGroup
1	1	30	26-35
2	2	45	36-50
3	3	60	51+
4	4	22	18-25
5	5	30	26-35
6	6	45	36-50
7	7	60	51+
8	8	22	18-25
9	9	30	26-35
10	10	45	36-50

Feature Engineering (Concepts)

Process: Create new fields to add business meaning

Examples:

TotalSpend → SUM of purchases per customer

CustomerAge → difference between birthdate & today

Recency → days since last purchase

- ❑ Business Use: segmentation, churn analysis, recommendations
- ❑ SQL Implementation: aggregations, joins, date functions
- ❑ Impact: transforms raw data into actionable insights

SQL Example: Feature Engineering

```
121 -- Problem Statement:
122 -- We want to calculate the TotalSpend for each customer
123 -- as a new feature for analysis.
124 -- TotalSpend = SUM(Quantity * UnitPrice) across all their orders.
125
126 SELECT c.CustomerID,
127        c.FirstName + ' ' + c.LastName AS FullName,
128        SUM(oi.Quantity * oi.UnitPrice) AS TotalSpend
129 FROM ba.Customers c
130 JOIN ba.Orders o
131     ON c.CustomerID = o.CustomerID
132 JOIN ba.OrderItems oi
133     ON o.OrderID = oi.OrderID
134 GROUP BY c.CustomerID, c.FirstName, c.LastName;
135
136 -- Explanation:
137 -- 1. We join Customers → Orders → OrderItems.
138 -- 2. For each customer, we calculate SUM(Quantity * UnitPrice).
139 -- 3. GROUP BY ensures one row per customer.
140 -- 4. TotalSpend is a derived (engineered) feature useful in churn, segmentation, etc.
141
```

100 % 4 0

Results Messages

	CustomerID	FullName	TotalSpend
1	242	Liam Miller	1839949.98

Data Splitting (Concepts)

- ❑ Purpose: ensures fair evaluation & prevents overfitting
- ❑ SQL Limitation: no built-in ML split, but can simulate

Methods:

- ❑ Time-based split → before vs. after date (e.g., sales forecasting)
- ❑ Random split → use NEWID() or RAND() for sampling
- ❑ Result: separate train/test datasets for reliable analysis

Best Practices for SQL Data Prep

Best Practice	Why It Matters
Explore with SELECT before UPDATE/DELETE	Prevents accidental data loss
Use staging tables before overwriting data	Keeps original data safe while testing changes
Document every cleaning step in SQL comments	Ensures reproducibility and clarity
Validate row counts after transformations	Confirms no records were lost or duplicated
Automate recurring cleaning steps with stored procedures	Saves time and reduces human error
Always backup raw data	Allows recovery if cleaning goes wrong
Use transactions for risky operations	Rollback if something fails unexpectedly
Set constraints & keys	Maintains data integrity and prevents duplicates
Check for data types & ranges	Catches invalid dates, amounts, or IDs
Log changes in audit tables	Provides traceability for compliance

Summary

- ❑ Data understanding/prep = foundation of analytics
- ❑ SQL helps detect missing values, duplicates, outliers
- ❑ Cleaning ensures accuracy
- ❑ Transformations & features add value
- ❑ Splitting prepares data for fair evaluation
- ❑ Reliable prep = reliable business decisions

Q&A



Modeling & Evaluation Frameworks



Day 04

Content List

- ❑ CRISP-DM Recap
- ❑ Modeling – Concepts & SQL Examples
- ❑ Evaluation – Concepts & SQL Examples
- ❑ Case Walkthrough: Segmentation & Churn
- ❑ SQL Hands-On Lab
- ❑ Best Practices & Pitfalls

Learning Objectives

- ❑ Understand Modeling & Evaluation in CRISP-DM
- ❑ Build SQL-based business models (segmentation, ranking, churn)
- ❑ Apply evaluation metrics using SQL queries
- ❑ Interpret results and validate with KPIs
- ❑ Prepare insights for deployment in reports/dashboards

CRISP-DM Recap



Business Understanding →
define goals



Data Understanding →
explore raw data



Data Preparation →
clean & structure



Modeling → build
models to answer
business questions



Evaluation →
validate results
against KPIs



Deployment →
share results with
stakeholders

What is Modeling?

Definition: Modeling means applying business logic (often using SQL) to structure and analyze raw data.

Purpose: Transform vague business questions into measurable and testable insights.

Examples of Modeling Tasks:

Customer segmentation (grouping by behavior, region, or spend)

Ranking (e.g., top customers, top products)

Churn detection (identifying customers likely to leave)

Trend analysis (sales growth, seasonal changes)

Key Role: Acts as the bridge between raw data and business KPIs (Key Performance Indicators).

Outcome: Enables organizations to make data-driven decisions rather than relying on intuition.

Why Modeling is Important



Structures analysis into business-relevant outputs



Identifies top drivers of performance



Builds foundation for predictions



Ensures decisions are based on measurable data



Without modeling → insights remain scattered

Types of SQL Models

-  Segmentation: Divide customers into groups (e.g., region, behavior)
- ☆ Scoring: Assign values like risk or loyalty scores
-  Aggregation: Roll-up metrics (e.g., sales by region, avg order value)
-  Trend Analysis: Track changes over time (e.g., monthly revenue growth)
-  Churn Analysis: Identify inactive or lost customers

Dataset Used

Customers: CustomerID, Name,
Region, SignupDate

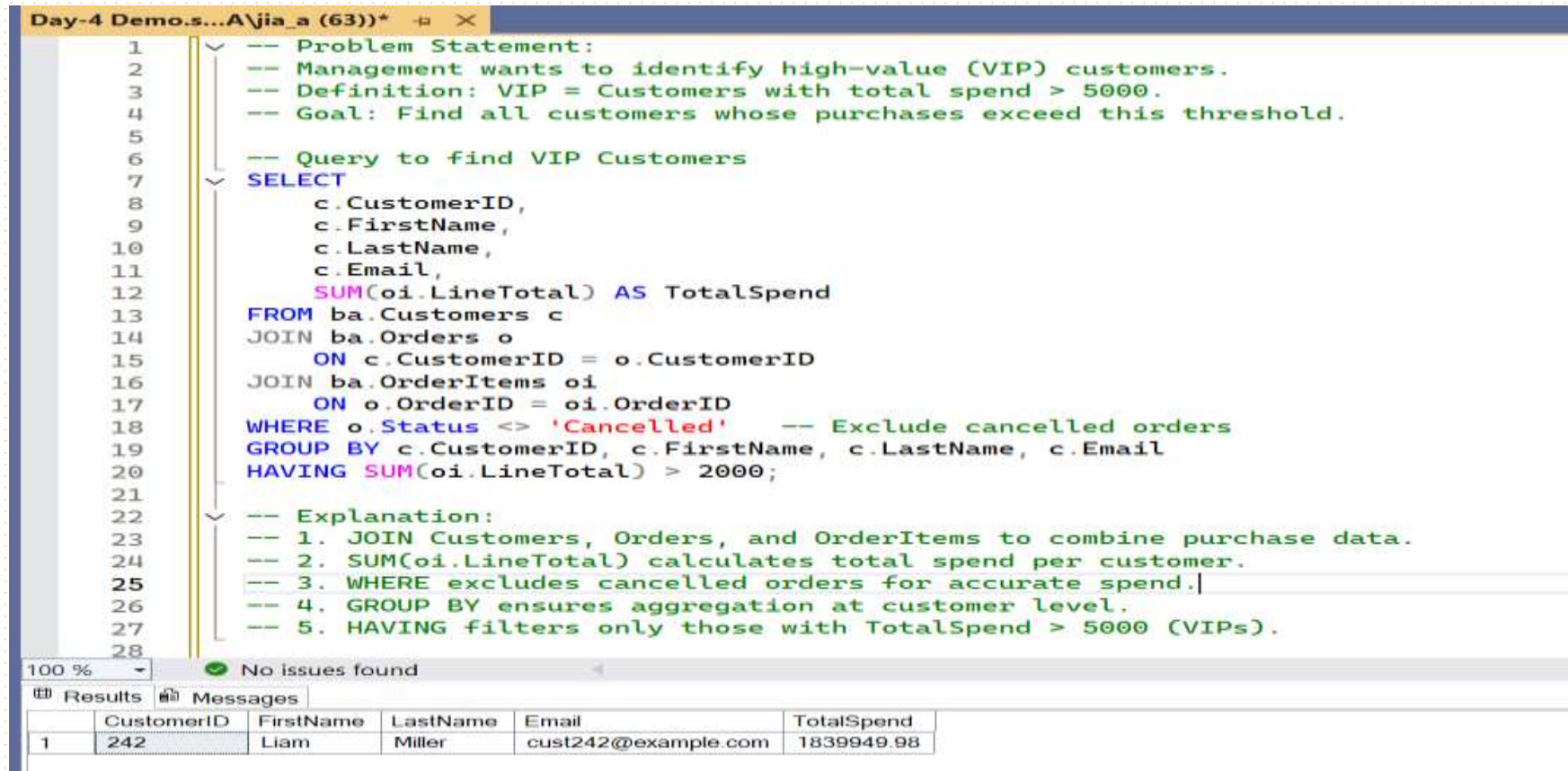


Sales: OrderID, CustomerID,
PurchaseDate, PurchaseAmount



We'll use these to create models
and evaluate them

Problem Statement 1 – VIP Customers



The screenshot shows a SQL IDE window titled "Day-4 Demo.s...A\jia_a (63))*". The editor contains a SQL query with comments explaining the problem statement and the query logic. The query finds VIP customers by joining the Customers, Orders, and OrderItems tables, calculating the total spend per customer, and filtering for those with a total spend greater than 2000. The results pane at the bottom shows a single result for CustomerID 242, Liam Miller, with a total spend of 1839949.98.

```
1 -- Problem Statement:
2 -- Management wants to identify high-value (VIP) customers.
3 -- Definition: VIP = Customers with total spend > 5000.
4 -- Goal: Find all customers whose purchases exceed this threshold.
5
6 -- Query to find VIP Customers
7 SELECT
8     c.CustomerID,
9     c.FirstName,
10    c.LastName,
11    c.Email,
12    SUM(oi.LineTotal) AS TotalSpend
13 FROM ba.Customers c
14 JOIN ba.Orders o
15     ON c.CustomerID = o.CustomerID
16 JOIN ba.OrderItems oi
17     ON o.OrderID = oi.OrderID
18 WHERE o.Status <> 'Cancelled' -- Exclude cancelled orders
19 GROUP BY c.CustomerID, c.FirstName, c.LastName, c.Email
20 HAVING SUM(oi.LineTotal) > 2000;
21
22 -- Explanation:
23 -- 1. JOIN Customers, Orders, and OrderItems to combine purchase data.
24 -- 2. SUM(oi.LineTotal) calculates total spend per customer.
25 -- 3. WHERE excludes cancelled orders for accurate spend.
26 -- 4. GROUP BY ensures aggregation at customer level.
27 -- 5. HAVING filters only those with TotalSpend > 5000 (VIPs).
```

100 % No issues found

Results Messages

	CustomerID	FirstName	LastName	Email	TotalSpend
1	242	Liam	Miller	cust242@example.com	1839949.98

```
-- Problem Statement:
-- Evaluate what % of total revenue comes from VIP customers.
-- KPI: Contribution Ratio = (Revenue from VIPs) / (Total Revenue).
```

```
-- Step 1: Compute spend per customer
```

```
;WITH Spend AS (
    SELECT
        c.CustomerID,
        SUM(oi.LineTotal) AS TotalSpend
    FROM ba.Customers c
    JOIN ba.Orders o ON c.CustomerID = o.CustomerID
    JOIN ba.OrderItems oi ON o.OrderID = oi.OrderID
    WHERE o.Status <> 'Cancelled'
    GROUP BY c.CustomerID
),
```

```
-- Step 2: Mark VIPs (threshold = 5000)
```

```
VIPs AS (
    SELECT CustomerID, TotalSpend
    FROM Spend
    WHERE TotalSpend > 5000
),
```

```
-- Step 3: Compute totals
Totals AS (
    SELECT
        (SELECT SUM(TotalSpend) FROM VIPs) AS VIP_Revenue,
        (SELECT SUM(TotalSpend) FROM Spend) AS Total_Revenue
)
-- Final Step: Contribution Ratio
SELECT
    VIP_Revenue,
    Total_Revenue,
    CAST(VIP_Revenue * 100.0 / NULLIF(Total_Revenue,0) AS DECIMAL(5,2))
    AS ContributionPct
FROM Totals;

-- Explanation:
-- 1. Spend CTE calculates total spend per customer.
-- 2. VIPs CTE filters customers above the 5000 threshold.
-- 3. Totals CTE computes overall and VIP-only revenue.
-- 4. Final SELECT shows % of revenue from VIPs.
```

Results			
	VIP_Revenue	Total_Revenue	ContributionPct
1	1839949.98	1839949.98	100.00

Evaluation – Contribution of VIPs

SQL Evaluation – Contribution Ratio

```
72 -----Contribution Ratio -----
73 -- Problem Statement:
74 -- Find what % of total revenue comes from VIPs (spend > 5000).
75 -- KPI: Contribution Ratio = VIP Revenue / Total Revenue.
76
77 SELECT
78     SUM(CASE WHEN TotalSpend > 5000 THEN TotalSpend ELSE 0 END) * 1.0
79     / SUM(TotalSpend) AS ContributionRatio
80 FROM (
81     SELECT
82         c.CustomerID,
83         SUM(oi.LineTotal) AS TotalSpend
84     FROM ba.Customers c
85     JOIN ba.Orders o ON c.CustomerID = o.CustomerID
86     JOIN ba.OrderItems oi ON o.OrderID = oi.OrderID
87     WHERE o.Status <> 'Cancelled'
88     GROUP BY c.CustomerID
89 ) t;
90
91 -- Explanation:
92 -- 1. Inner query: calculate TotalSpend per customer.
93 -- 2. CASE WHEN: only include spend > 5000 for VIP revenue.
94 -- 3. SUM(TotalSpend): denominator = all customer spend.
95 -- 4. Outer query divides VIP revenue by total revenue = Contribution Ratio.
96
```

5 % 5 0

Results Messages

	ContributionRatio
1	1.000000


```

-----CHURN CUSTOMERS-----

-- Problem Statement:
-- Detect customers who have not purchased in the last 6 months (churn risk).

-- Step 1: Define cutoff date = today - 6 months
DECLARE @Cutoff DATE = DATEADD(MONTH, -6, CAST(GETDATE() AS DATE));

-- Step 2: Find last purchase date per customer and flag churners
SELECT
    c.CustomerID,          -- unique customer ID
    c.FirstName,           -- customer first name
    c.LastName,            -- customer last name
    c.Email,               -- email for identification
    MAX(p.PaymentDate) AS LastPurchaseDate, -- most recent payment date
    CASE
        WHEN MAX(p.PaymentDate) IS NULL -- never purchased
        OR MAX(p.PaymentDate) < @Cutoff -- last purchase older than cutoff
        THEN 1 -- churn risk
        ELSE 0 -- still active
    END AS IsChurnRisk
FROM ba.Customers c
LEFT JOIN ba.Orders o ON o.CustomerID = c.CustomerID -- link to orders
LEFT JOIN ba.Payments p ON p.OrderID = o.OrderID -- link to payments (completed orders)
GROUP BY c.CustomerID, c.FirstName, c.LastName, c.Email
ORDER BY IsChurnRisk DESC, LastPurchaseDate;

-- Explanation:
-- 1. @Cutoff defines 6-month churn window.
-- 2. MAX(PaymentDate) = last valid purchase date.
-- 3. CASE WHEN: flag churners if last purchase is NULL or before cutoff.
-- 4. Output shows all customers with churn risk highlighted.
-- 5. ORDER BY shows churn-risk customers at the top.

```

85 %

5

0

↑

↓

⏪

Results

Messages

	CustomerID	FirstName	LastName	Email	LastPurchaseDate	IsChurnRisk
7	7	Liam	Miller	cust7@example.com	NULL	1
8	8	Liam	Miller	cust8@example.com	NULL	1
9	9	Liam	Miller	cust9@example.com	NULL	1
10	10	Liam	Miller	cust10@example.c...	NULL	1
11	11	Liam	Miller	cust11@example.c...	NULL	1
12	12	Liam	Miller	cust12@example.c...	NULL	1
13	13	Liam	Miller	cust13@example.c...	NULL	1
14	14	Liam	Miller	cust14@example.c...	NULL	1
15	15	Liam	Miller	cust15@example.c...	NULL	1

Problem Statement 2 – Churn Customers

Evaluation – Active vs Churn

```
130 -----ACTIVE VS CHURN-----
131
132 -- Problem: Show how many customers are Active vs Churn
133 -- Definition: Churn = no order in the last 6 months (demo-safe: uses Orders)
134
135 -- 1) Set the cutoff date (today - 6 months)
136 DECLARE @Cutoff DATE = DATEADD(MONTH, -6, CAST(GETDATE() AS DATE));
137
138 -- 2) Get each customer's last order date (any status -> always shows something in class)
139 ;WITH LastOrder AS (
140     SELECT
141         c.CustomerID,
142         MAX(o.OrderDate) AS LastOrderDate
143     FROM ba.Customers c
144     LEFT JOIN ba.Orders o ON o.CustomerID = c.CustomerID
145     GROUP BY c.CustomerID
146 )
147
148 -- 3) Output two rows: Active and Churn (very readable)
149 SELECT 'Active' AS CustomerStatus, COUNT(*) AS CustomerCount
150 FROM LastOrder
151 WHERE LastOrderDate >= @Cutoff
152 UNION ALL
153 SELECT 'Churn' AS CustomerStatus, COUNT(*)
154 FROM LastOrder
155 WHERE LastOrderDate < @Cutoff OR LastOrderDate IS NULL;
156
157 -- Explanation:
158 -- - LastOrderDate >= cutoff -> Active
159 -- - LastOrderDate < cutoff or NULL -> Churn (never ordered)
```

85 % 1 0

Results Messages

	CustomerStatus	CustomerCount
1	Active	1
2	Churn	306

SQL Evaluation – Churn Distribution

Churn distribution is the **split of customers** into two groups:

Active → customers who purchased in the last 6 months

Churn Risk → customers with no purchases in the last 6 months

Why it's Important:

Helps businesses see **how many customers are becoming inactive**

Allows tracking of **customer retention vs loss over time**

Provides a **KPI (churn rate)** that management can use for decision making

Guides actions like **loyalty programs, re-engagement campaigns, or discounts**

Problem Statement 3 – Monthly Trends

```
-----MONTHLY TRENDS-----  
  
-- Problem Statement: Monthly Trends  
-- Goal: Track monthly sales and calculate growth rate  
  
-- Step 1: Build monthly totals  
WITH MonthlySales AS (  
    SELECT  
        YEAR(o.OrderDate) AS Yr,           -- extract year from order date  
        MONTH(o.OrderDate) AS Mn,         -- extract month from order date  
        SUM(oi.LineTotal) AS TotalSales    -- total sales for that month  
    FROM ba.Orders o  
    JOIN ba.OrderItems oi  
        ON o.OrderID = oi.OrderID  
    WHERE o.Status <> 'Cancelled'          -- exclude cancelled orders  
    GROUP BY YEAR(o.OrderDate), MONTH(o.OrderDate) -- group data by month  
)
```

```
176  
177  
178 -- Step 2: Add previous month sales and calculate growth rate  
179 SELECT  
180     Yr, -- year  
181     Mn, -- month  
182     TotalSales, -- sales of that month  
183     LAG(TotalSales) OVER (ORDER BY Yr, Mn) AS PrevMonthSales,  
184     -- LAG(): brings previous month's sales value  
185     CASE  
186         WHEN LAG(TotalSales) OVER (ORDER BY Yr, Mn) IS NULL THEN NULL  
187         -- if there is no previous month (e.g. first month), leave NULL  
188         ELSE (TotalSales - LAG(TotalSales) OVER (ORDER BY Yr, Mn)) = 1.0 /  
189             LAG(TotalSales) OVER (ORDER BY Yr, Mn)  
190         -- formula: (Current - Previous) / Previous = Growth Rate  
191     END AS GrowthRate  
192 FROM MonthlySales  
193 ORDER BY Yr, Mn; -- display results in time order
```

85 % 1 0

	Yr	Mn	TotalSales	PrevMonthSales	GrowthRate
5	2025	1	63227.19	67480.14	-0.063025
6	2025	2	72016.62	63227.19	0.139013
7	2025	3	70315.44	72016.62	-0.023622
8	2025	4	63510.72	70315.44	-0.096774
9	2025	5	106761.4	63510.72	15.809985
10	2025	6	56706.00	106761.26	-0.946885
11	2025	7	72583.68	56706.00	0.280000
12	2025	8	84208.41	72583.68	0.160156
13	2025	9	73717.80	84208.41	-0.124579

SQL Evaluation – Growth Rate

```
196 -----GROWTH RATE-----
197
198 -- Problem Statement: Track monthly sales and growth
199 -- Table: Sales.Orders (using OrderDate and Sales columns)
200
201 -- Step 1: Calculate monthly totals
202 WITH Monthly AS (
203     SELECT
204         YEAR(OrderDate) AS Yr,           -- extract year
205         MONTH(OrderDate) AS Mn,         -- extract month
206         SUM(Sales) AS MonthlySales     -- total sales per month
207     FROM Sales.Orders
208     WHERE OrderStatus <> 'Cancelled'    -- optional filter to exclude cancelled
209     GROUP BY YEAR(OrderDate), MONTH(OrderDate)
210 )
211
212 -- Step 2: Add previous month and growth %
213 SELECT
214     Yr,
215     Mn,
216     MonthlySales,
217     LAG(MonthlySales) OVER (ORDER BY Yr, Mn) AS PrevMonthSales, -- previous month sales
218     CASE
219         WHEN LAG(MonthlySales) OVER (ORDER BY Yr, Mn) IS NULL THEN NULL
220         ELSE (MonthlySales - LAG(MonthlySales) OVER (ORDER BY Yr, Mn)) * 100.0
221             / NULLIF(LAG(MonthlySales) OVER (ORDER BY Yr, Mn), 0)
222     END AS GrowthRatePct -- % growth vs last month
223 FROM Monthly
224 ORDER BY Yr, Mn;
225
```

5 % 77 0

Results Messages

	Yr	Mn	MonthlySales	PrevMonthSales	GrowthRatePct
1	2025	1	85	NULL	NULL
2	2025	2	195	85	129.411764705882
3	2025	3	80	195	-58.974358974358

Best Practices – Modeling

Keep models simple & explainable

Use consistent thresholds & definitions

→ Example: VIP customer = total spend > 5000.

Document assumptions clearly

→ Always note filters, joins, time windows (e.g., “last 12 months”).

Validate results regularly

→ Compare against raw data or business reports to catch errors early.

Align models with business goals

→ Every model should answer a specific KPI question, not just produce data.

Ensure reproducibility

→ Save queries or views, don't rely on ad-hoc SQL runs.



Best Practices – Evaluation

Validate	Always validate with KPIs
Compare	Compare results with past benchmarks
Discuss	Discuss insights with stakeholders
Avoid	Avoid one-off metrics that can mislead

Common Pitfalls

Arbitrary thresholds with no justification

Ignoring NULL or missing values

Over-interpreting temporary trends

Confusing correlation with causation

Overcomplicating models

Summary

SQL as a modeling tool

→ Flexible, powerful, and explainable for business analysis.

Models we practiced

→ Customer segmentation, scoring/VIP analysis, churn detection, trend analysis, aggregations.

Evaluation is critical

→ Validates accuracy and ensures insights are trustworthy.

From insights to action

→ Models must connect to business KPIs and decision-making.

Deployment-ready results

→ SQL outputs can feed into dashboards, BI tools, and reporting systems

Q&A



Deployment Strategies



Day 05

Content List

- ❑ CRISP-DM Recap
- ❑ Step 6: Deployment – Concept & Importance
- ❑ Deployment in SQL Context
- ❑ Deployment Methods
- ❑ Case Walkthroughs (Top Customers, Churn)
- ❑ SQL Hands-On Labs
- ❑ Best Practices & Pitfalls

Learning Objectives

- ❑ Define Deployment in CRISP-DM lifecycle
- ❑ Explain why deployment is critical in business analytics
- ❑ Show how SQL results can be packaged for deployment
- ❑ Learn multiple deployment methods (reports, dashboards, automation)
- ❑ Understand how deployment ensures real business value
- ❑ Explore challenges and success factors in deployment
- ❑ Prepare for advanced modules in analytics

CRISP-DM Recap

Step 1: Business Understanding – define goals

Step 2: Data Understanding – explore raw data

Step 3: Data Preparation – clean and transform

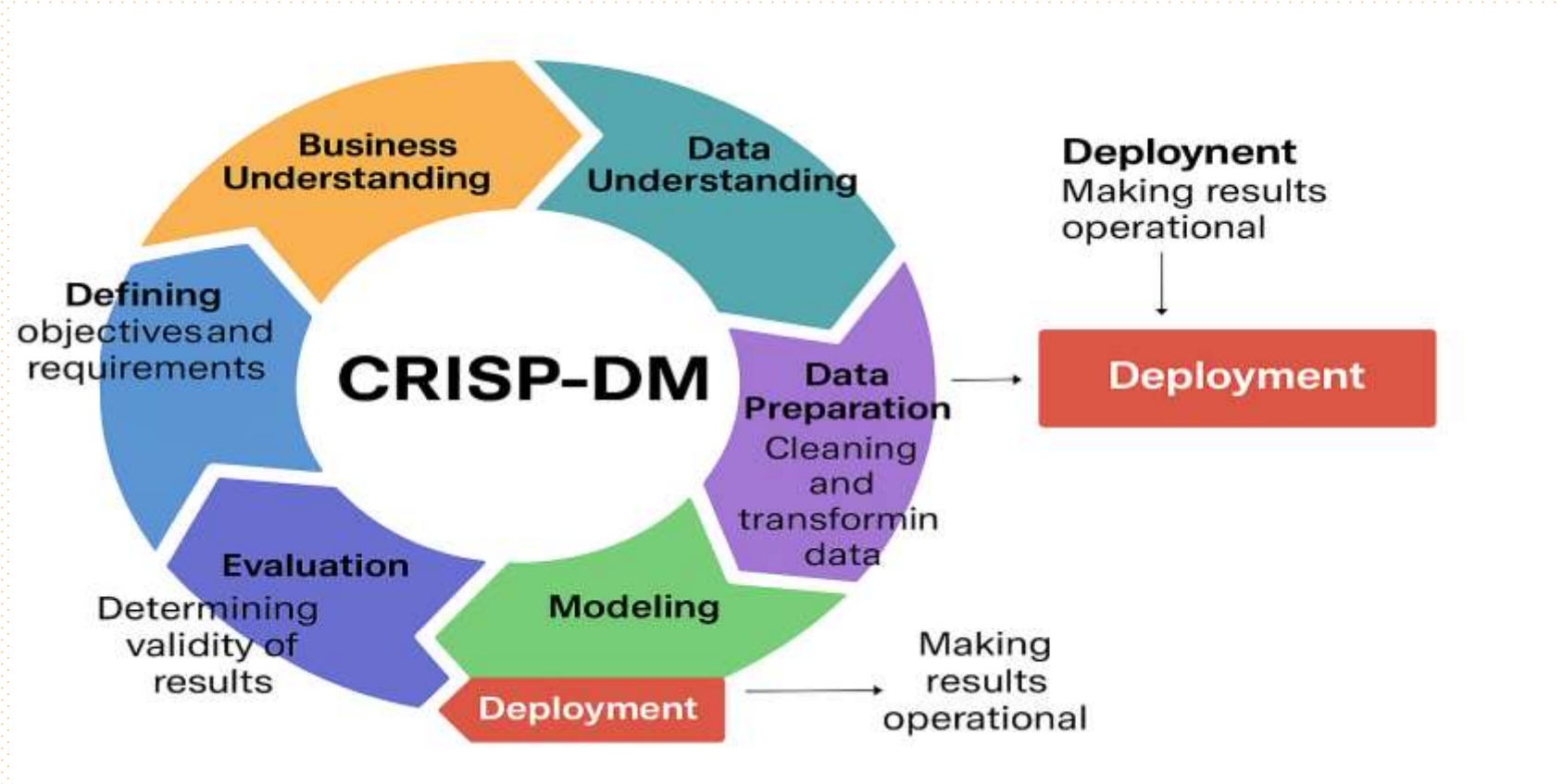
Step 4: Modeling – build SQL-based models

Step 5: Evaluation – validate results

Step 6: Deployment – Today's focus

Deployment ensures business impact

What Is Deployment



What is Deployment?

Final step of CRISP-DM cycle

Making analysis results accessible to stakeholders

Ensures insights are actionable and drive decisions

Converts SQL outputs into business-friendly formats

Bridges analytics team and business decision-makers

Without deployment, analysis often remains unused

Critical for turning data into measurable value

Why Deployment Matters

Ensures SQL analysis is applied in practice

Turns technical results into business insights

Drives adoption of data-driven culture

Provides measurable ROI from analytics projects

Helps prioritize future initiatives

Reduces wasted effort on unused analysis

Builds trust in analytics outputs

Deployment in SQL Context



SQL generates tables, KPIs, aggregations



Deployment delivers these outputs to stakeholders



Common forms: reports, dashboards, alerts



SQL outputs can be exported to Excel/CSV



Results can be integrated with BI dashboards



Automated SQL jobs ensure timely refresh



SQL → Deployment = action pipeline

Methods of Deployment

Method	Description
Reports	Scheduled exports (PDF, Excel, CSV)
Dashboards	BI tools (Power BI, Tableau)
Automation	SQL scripts and jobs
Integration	APIs connecting SQL results
Alerts	Notifications for thresholds
Embedded Analytics	SQL insights into apps
Note	Each method suits different use cases

Deployment Option – Reports



Most common deployment form



SQL queries generate static outputs



Shared via Excel, CSV, or PDF



Scheduled on daily/weekly basis



Ensures consistent insights delivery



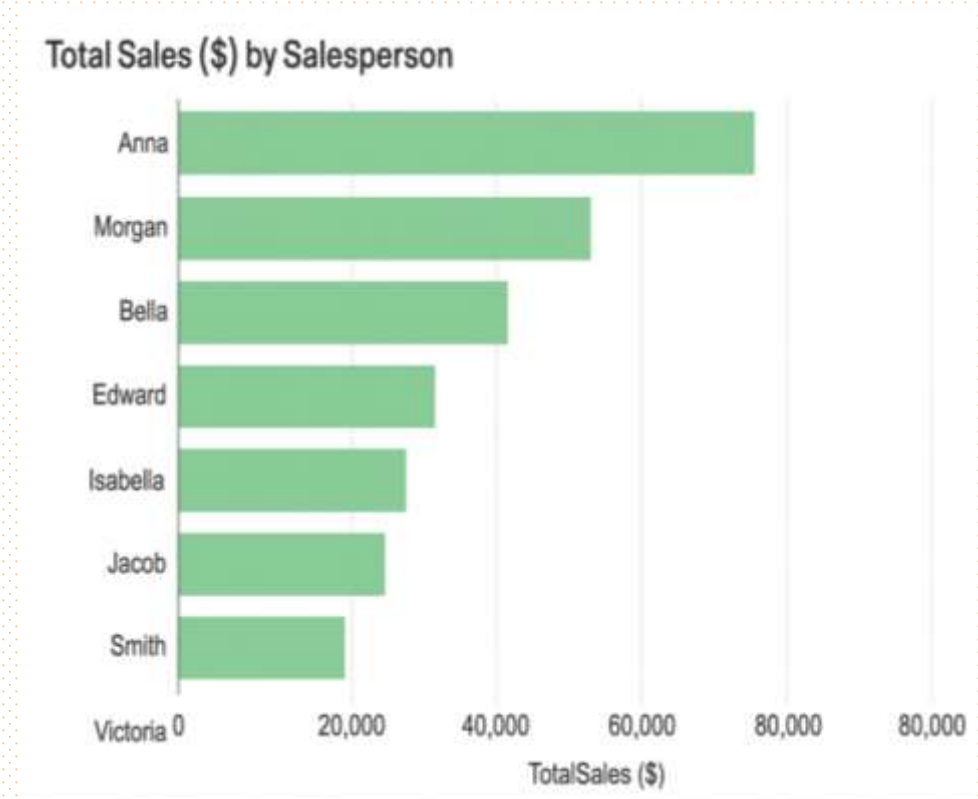
Useful for managers tracking KPIs



Limitation: Static, not interactive

Report

- ❑ The chart shows total sales (\$) achieved by each salesperson.
- ❑ Anna leads with the highest sales, followed by Morgan and Bella.
- ❑ Sales gradually decline across Edward, Isabella, Jacob, and Smith.
- ❑ Victoria has zero sales, indicating no contribution in the period.
- ❑ This helps identify top performers and those needing support or training.



Deployment Option – Dashboards



Interactive visualization platforms



SQL queries feed BI tools



Users can filter, drill, slice data



KPIs easier to interpret visually



Useful for executives & decision-makers



Dashboards update dynamically



Supports wider adoption of analytics

Dashboards

- ❑ KPI cards: Revenue, Orders, Profit with growth rates
- ❑ Sales by Region: US map showing geographic distribution
- ❑ Revenue & Profit: Monthly trend line and bar chart
- ❑ Category & State: Sales by product category + profit margin by state



Deployment Option – Automation



Automated SQL jobs ensure consistency



Alerts trigger when thresholds reached



Useful for inventory alerts/fraud detection



Scheduled tasks refresh tables nightly



Removes manual intervention



Increases trust in timely data



Links analytics with operations

Deployment Option – Integration

SQL outputs connected via APIs

Results embedded into CRM/ERP

Supports real-time processes

Example: recommendations in e-commerce

Provides seamless user experience

Used in enterprise-grade solutions

Direct impact on operations

Challenges in Deployment



Ensuring security and access controls



Performance issues in large SQL queries



Keeping data fresh: batch vs real-time



Aligning outputs with business needs



Maintaining system compatibility



Overcoming communication gaps



Balancing simplicity with detail

Success Factors

Clear and consistent KPIs

Validation of SQL outputs

Choosing right method for audience

Ensuring adoption through training

Monitoring and feedback loops

Automating repetitive reports

Iterative improvement post-deployment

Mini Case – Top 10 Products (Problem)

- Business Goal: Identify best-selling products
- Use SQL to calculate total revenue
- Join OrderItems with Products to get names
- Rank products by TotalRevenue to find leaders
- Insight: A few products drive majority of revenue
- Deployment: Reports, Dashboards, Automation

SQL Examples

SQL Query – Top 10 Customers

```
1  /* =====
2  1) Top 10 Products by Revenue
3  Goal: Identify best-selling products for reporting/deployment.
4  Works with: ba.OrderItems, ba.Products
5  ===== */
6  SELECT TOP 10
7      p.ProductID,
8      p.ProductName,
9      SUM(oi.Quantity * oi.UnitPrice) AS TotalRevenue
10 FROM ba.OrderItems oi
11 JOIN ba.Products p ON p.ProductID = oi.ProductID
12 GROUP BY p.ProductID, p.ProductName
13 ORDER BY TotalRevenue DESC;
14 /* Explanation:
15 - Join items to products to name each SKU.
16 - Revenue = Quantity x UnitPrice aggregated per product.
17 - ORDER BY DESC to rank best sellers.
18 */
19
```

85 % 1 0

Results Messages

	ProductID	ProductName	TotalRevenue
1	10	Product-10	999000.00
2	119	Product-119	840949.98

Mini Case – Churn Customers (Problem)

- ❑ Business Goal: Identify churn risks
- ❑ Rule: no purchase in last 6 months
- ❑ Use MAX(PurchaseDate) in SQL
- ❑ Output helps reactivation campaigns
- ❑ Supports retention strategy
- ❑ Stakeholders: Customer support
- ❑ Deployment: Reports, Alerts, Dashboards

SQL Query – Churn Customers

```
25
26  /* Problem: Find customers who haven't purchased in the last 6 months.
27   Goal: Flag churn-risk customers for reactivation campaigns. */
28
29  SELECT
30      c.CustomerID,
31      c.FirstName,
32      c.LastName,
33      MAX(o.OrderDate) AS LastPurchaseDate
34  FROM ba.Customers c
35  LEFT JOIN ba.Orders o
36      ON c.CustomerID = o.CustomerID
37  GROUP BY c.CustomerID, c.FirstName, c.LastName
38  HAVING MAX(o.OrderDate) IS NULL -- never purchased
39      OR MAX(o.OrderDate) < DATEADD(MONTH, -6, GETDATE()) -- last purchase > 6 months ago
40  ORDER BY LastPurchaseDate;
41
42  /*
43  Explanation:
44  1. MAX(OrderDate) → finds the most recent purchase per customer.
45  2. LEFT JOIN ensures all customers are included, even with no orders.
46  3. HAVING filters customers whose last purchase was more than 6 months ago.
47  4. DATEADD(MONTH, -6, GETDATE()) → defines the churn threshold.
48  5. Output: List of customers at risk of churn.
49  */
```

85 % 1 0

Results Messages

	CustomerID	FirstName	LastName	LastPurchaseDate
1	1	Liam	Miller	NULL

Regional Sales

```
50
51 -----REGIONAL SALES-----
52
53 /*
54 Problem: Find total sales by region.
55 Goal: Help management compare performance across regions.
56 Works with: ba.Orders, ba.OrderItems, ba.Customers
57 */
58
59 -- Step 1: Select region from Customers and calculate sales
60 SELECT
61     c.Region,                                -- region of customer
62     SUM(oi.Quantity * oi.UnitPrice) AS TotalSales -- revenue per region
63 FROM ba.Orders o
64
65 -- Step 2: Join Orders with Customers to get region info
66 JOIN ba.Customers c
67     ON c.CustomerID = o.CustomerID
68
69 -- Step 3: Join OrderItems to calculate sales per order
70 JOIN ba.OrderItems oi
71     ON o.OrderID = oi.OrderID
72
73 -- Step 4: Group by region so totals are per region
74 GROUP BY c.Region
75
76 -- Step 5: Sort results from highest to lowest
77 ORDER BY TotalSales DESC
```

85 % 1 0

Region	TotalSales
1 East	1839949.98

Lab Exercise – Top 10 Customers

```
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100  
101  
102  
103  
104  
105  
106  
107  
108
```

```
-----TOP CUSTOMER-----  
  
/*  
Problem: Find the Top 10 customers by spending.  
Goal: Identify most valuable customers for reporting/loyalty programs.  
Works with: ba.Orders, ba.OrderItems, ba.Customers  
*/  
  
-- Step 1: Calculate spend per customer  
SELECT TOP 10  
    c.CustomerID,  
    c.FirstName,  
    c.LastName,  
    SUM(oi.Quantity * oi.UnitPrice) AS TotalSpent  
FROM ba.Orders o  
  
-- Step 2: Join Orders with Customers for names  
JOIN ba.Customers c  
    ON o.CustomerID = c.CustomerID  
  
-- Step 3: Join OrderItems to compute purchase totals  
JOIN ba.OrderItems oi  
    ON o.OrderID = oi.OrderID  
  
-- Step 4: Group so totals are per customer  
GROUP BY c.CustomerID, c.FirstName, c.LastName  
  
-- Step 5: Sort so highest spenders come first  
ORDER BY TotalSpent DESC;
```

5 % 1 0

Results Messages

	CustomerID	FirstName	LastName	TotalSpent
1	242	Liam	Miller	1839949.98


```

111  /*
112  Problem: Analyze monthly sales trends.
113  Goal: Track growth rates and seasonality for business insights.
114  Works with: ba.Orders, ba.OrderItems
115  */
116
117  -- Step 1: Group sales by Year, Month
118  SELECT
119      YEAR(o.OrderDate) AS Yr,          -- extract year
120      MONTH(o.OrderDate) AS Mn,        -- extract month
121
122  -- Step 2: Calculate total sales per month
123      SUM(oi.Quantity * oi.UnitPrice) AS MonthlySales,
124
125  -- Step 3: Use LAG() to get previous month's sales
126      LAG(SUM(oi.Quantity * oi.UnitPrice))
127      OVER (ORDER BY YEAR(o.OrderDate), MONTH(o.OrderDate)) AS PrevMonthSales,
128  
```

```

129  -- Step 4: Compute growth rate %
130  (
131      (SUM(oi.Quantity * oi.UnitPrice) -
132       LAG(SUM(oi.Quantity * oi.UnitPrice))
133        OVER (ORDER BY YEAR(o.OrderDate), MONTH(o.OrderDate))
134       ) * 100.0
135      / NULLIF(LAG(SUM(oi.Quantity * oi.UnitPrice))
136              OVER (ORDER BY YEAR(o.OrderDate), MONTH(o.OrderDate)), 0)
137  ) AS GrowthRate
138  FROM ba.Orders o
139  JOIN ba.OrderItems oi
140      ON o.OrderID = oi.OrderID
141
142  GROUP BY YEAR(o.OrderDate), MONTH(o.OrderDate)
143  ORDER BY Yr, Mn;
144
145  
```

	Yr	Mn	MonthlySales	PrevMonthSales	GrowthRate
1	2024	9	4536.48	NULL	NULL
2	2024	10	78821.34	4536.48	1637.500000
3	2024	11	65211.90	78821.34	-17.266187
4	2024	12	67480.14	65211.90	3.478260
5	2025	1	63227.19	67480.14	-6.302521
6	2025	2	72016.62	63227.19	13.901345
7	2025	3	70315.44	72016.62	-2.362204
8	2025	4	63510.72	70315.44	-9.677419

Lab Exercise – Monthly Sales Trends

Best Practices

Automate	Automate recurring reports
Document	Document SQL logic
Validate	Validate before deployment
Tailor	Tailor outputs to audience
Use	Use visuals for executives
Iterate	Iterate based on feedback
Keep	Keep KPIs consistent

Common Pitfalls

Unvalidated
SQL outputs

Too many
overlapping
reports

Not aligning
with business
strategy

Ignoring
feedback post-
rollout

Over-focus on
visuals vs
accuracy

Weak data
security in
reports

Not monitoring
deployed
outputs

Deployment Lifecycle



Summary

- ❑ Deployment = essential CRISP-DM step
- ❑ SQL results deployed via multiple methods
- ❑ Mini cases: Top 10, Churn risk
- ❑ Hands-on labs for deployment tasks
- ❑ Best practices: validate, automate
- ❑ Pitfalls: avoid over-reporting
- ❑ Deployment turns analysis into action

Q&A



Financial Metrics - Revenue, Cost & Profit



Day 06

Content List

- ❑ What is Revenue and why it matters in business analysis
- ❑ Understanding Costs: fixed and variable explained clearly
- ❑ Profit as the key financial metric that drives businesses
- ❑ Monthly revenue, cost, and profit analysis in SQL queries
- ❑ Regional profitability analysis using SQL joins and grouping
- ❑ Business applications of these metrics in real decision making
- ❑ Hands-on Lab: building dashboards and financial reports

Learning Objectives

- ❑ Define clearly the terms Revenue, Costs, and Profit in business
- ❑ Learn SQL queries to calculate each metric from sales data
- ❑ Differentiate between fixed costs and variable costs in analysis
- ❑ Apply SQL GROUP BY functions to analyze monthly financial data
- ❑ Prepare data for visualization tools like Excel and Power BI
- ❑ Understand profit margins as percentages for better insight
- ❑ Develop regional profitability reports for business scenarios

Revenue, Cost and Profit



What is Revenue?

Revenue is the total money earned by a business from sales.

It is also called the 'top line' in financial statements.

Formula: $\text{Revenue} = \text{Price} \times \text{Quantity of goods sold}$.

Example: Selling 100 pens at \$2 each = \$200 revenue.

Revenue is measured daily, monthly, and yearly in business.

Higher revenue does not always mean higher profit margins.

Analysts compute revenue from order data in SQL databases.

Revenue analysis is the starting point for business reporting.

SQL Example – Revenue

To calculate revenue, multiply quantity sold by the unit price.

SQL query example:

```
SELECT SUM(oi.Quantity * oi.UnitPrice) AS TotalRevenue  
FROM ba.OrderItems oi;
```

SUM() is an aggregate function that adds values across rows.

This query produces one number: the company's total revenue.

Analysts can also GROUP BY month or region for breakdowns.

Revenue is a key metric for performance and growth reporting.

What are Costs?

Costs are expenses that a business must pay to operate.

Two main types: Fixed Costs (rent, salaries) and Variable Costs (materials).

Fixed costs stay the same regardless of sales levels.

Variable costs increase when more products are produced or sold.

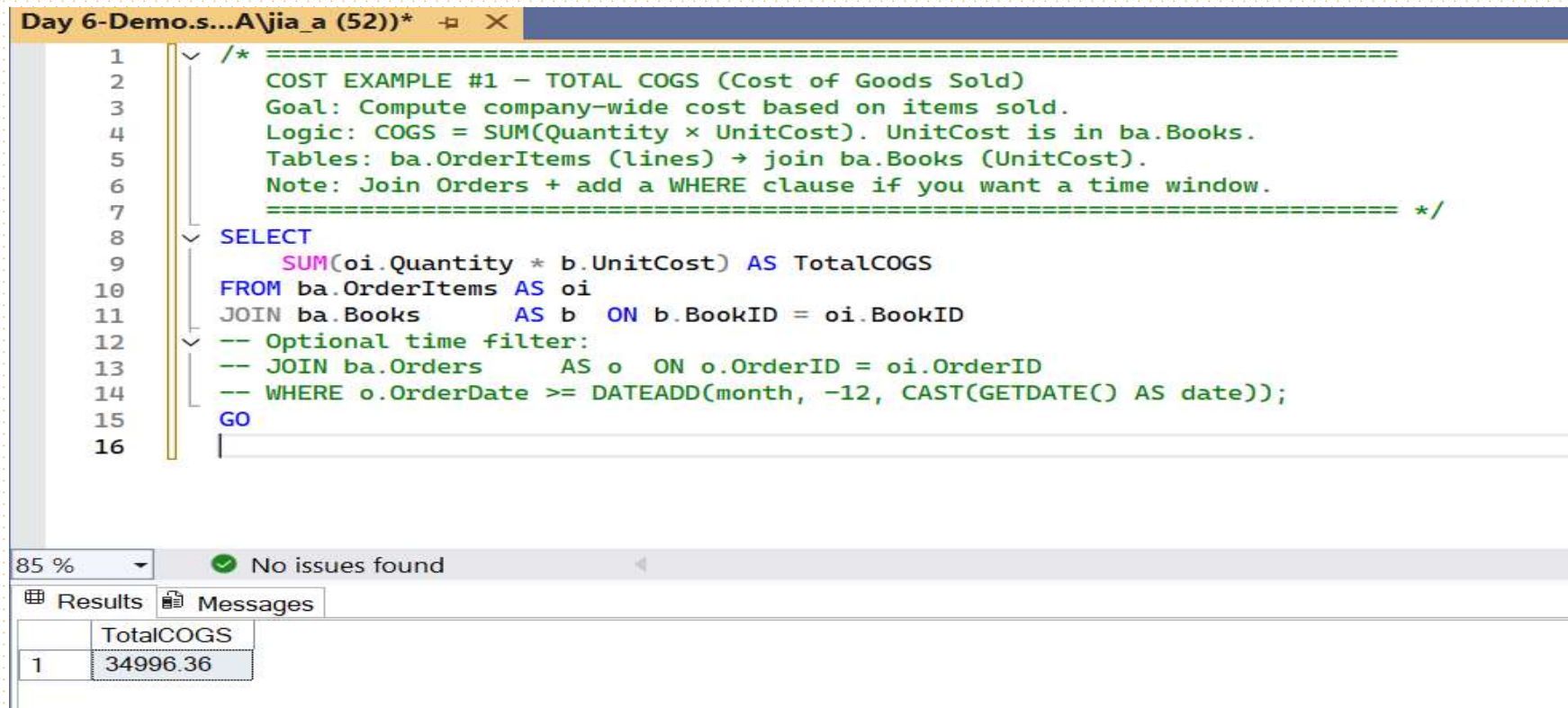
Costs are stored in databases as a unit cost per product.

Total Cost = Quantity × UnitCost calculated across sales.

Understanding costs helps identify efficiency opportunities.

Cost analysis is essential to control spending and improve profits.

SQL Example – Costs



The screenshot shows a SQL IDE window titled "Day 6-Demo.s...A\jia_a (52))*". The query is as follows:

```
1  /* =====
2  COST EXAMPLE #1 – TOTAL COGS (Cost of Goods Sold)
3  Goal: Compute company-wide cost based on items sold.
4  Logic: COGS = SUM(Quantity × UnitCost). UnitCost is in ba.Books.
5  Tables: ba.OrderItems (lines) → join ba.Books (UnitCost).
6  Note: Join Orders + add a WHERE clause if you want a time window.
7  ===== */
8  SELECT
9      SUM(oi.Quantity * b.UnitCost) AS TotalCOGS
10 FROM ba.OrderItems AS oi
11 JOIN ba.Books      AS b  ON b.BookID = oi.BookID
12 -- Optional time filter:
13 -- JOIN ba.Orders    AS o  ON o.OrderID = oi.OrderID
14 -- WHERE o.OrderDate >= DATEADD(month, -12, CAST(GETDATE() AS date));
15 GO
16 |
```

Below the query editor, the status bar shows "85 %", a green checkmark, and "No issues found". The "Results" tab is active, displaying a single row of data:

	TotalCOGS
1	34996.36

What is Profit?

Profit is the money left after subtracting total costs from revenue.

It is known as the 'bottom line' because it reflects net earnings.

Formula: Profit = Revenue – Cost, a basic financial measure.

Positive profit means the business is earning more than it spends.

Negative profit (loss) means costs are greater than revenue.

Profit shows sustainability and long-term business growth.

Managers and investors closely track profit margins regularly.

Profit is the most important measure of business performance.

SQL Example – Profit

```
17
18  /* =====
19  PROFIT EXAMPLE #1 – TOTAL REVENUE, COST, AND PROFIT (BooksDemo)
20  Goal: Compute company-wide profit from transaction lines.
21  Logic:
22  - Revenue = SUM(Quantity × UnitPrice) from ba.OrderItems
23  - Cost (COGS) = SUM(Quantity × UnitCost) via join to ba.Books
24  - Profit = Revenue - Cost (can compute directly per line)
25  Note: Join ba.Orders and add a WHERE clause if you need a date window.
26  ===== */
27  SELECT
28      SUM(oi.Quantity * oi.UnitPrice)          AS Revenue,
29      SUM(oi.Quantity * b.UnitCost)             AS Cost,
30      SUM(oi.Quantity * (oi.UnitPrice - b.UnitCost)) AS Profit
31  FROM ba.OrderItems AS oi
32  JOIN ba.Books      AS b  ON b.BookID = oi.BookID;
33  -- Optional time filter:
34  -- JOIN ba.Orders   AS o  ON o.OrderID = oi.OrderID
35  -- WHERE o.OrderDate >= DATEADD(month, -12, CAST(GETDATE() AS date));
36  GO
37
38
39
```

%

✓

No issues found

Results

Messages

Revenue	Cost	Profit
74169.49	34996.36	39173.13

Gross vs Net Profit



Gross Profit = Revenue – Direct Costs (like raw materials).



Net Profit = Gross Profit – All Expenses (including salaries, rent).



Gross profit shows profitability at the product or sales level.



Net profit shows the actual bottom line after all expenses.



Businesses analyze both to understand financial efficiency.



Gross profit is useful for product-level decisions.



Net profit is useful for investor reporting and sustainability.



SQL analysis can be adjusted to calculate both separately.

Profit Margins

Profit margin is the percentage of revenue kept as profit.

Formula: Profit Margin % = (Profit ÷ Revenue) × 100.

High margin indicates strong efficiency and cost control.

Low margin shows revenue is not converting well to profit.

Gross margin uses gross profit, net margin uses net profit.

Profit margins vary across industries and product categories.

Managers benchmark margins against competitors regularly.

SQL outputs can be used to compute margins in reports.

Monthly Profit Analysis

Businesses track profit monthly to see financial trends.

SQL GROUP BY YEAR() and MONTH() helps analyze this data.

Monthly revenue, cost, and profit tables highlight changes.

It shows if profits are improving or declining over time.

Seasonal businesses see peaks in certain months only.

Managers use monthly profit reports for budget planning.

Monthly profit trends can be visualized in dashboards.

It supports forecasting and long-term financial planning.

SQL Example – Monthly Profit

```
37
38
39  /* MONTHLY PROFIT (BooksDemo)
40  Returns one row per Year/Month with: Revenue, Cost (COGS), Profit.
41  Revenue = SUM(Quantity * UnitPrice) from ba.OrderItems
42  Cost    = SUM(Quantity * UnitCost) via join to ba.Books
43  Profit  = Revenue - Cost
44  */
45  SELECT
46      YEAR(o.OrderDate) AS Yr,                -- calendar year of the order
47      MONTH(o.OrderDate) AS Mn,              -- calendar month of the order
48      SUM(oi.Quantity * oi.UnitPrice)        AS Revenue, -- sales $
49      SUM(oi.Quantity * b.UnitCost)          AS Cost,    -- COGS $
50      SUM(oi.Quantity * (oi.UnitPrice - b.UnitCost)) AS Profit -- gross profit $
51  FROM ba.Orders AS o -- holds OrderDate
52  JOIN ba.OrderItems AS oi ON oi.OrderID = o.OrderID -- line quantities & prices
53  JOIN ba.Books AS b ON b.BookID = oi.BookID -- unit cost for each book
54  -- WHERE o.OrderDate >= '2024-01-01' -- (optional) time filter
55  GROUP BY YEAR(o.OrderDate), MONTH(o.OrderDate) -- aggregate by month
56  ORDER BY Yr, Mn; -- chronological output
57
58
59
60
```

5 % No issues found

Results Messages

	Yr	Mn	Revenue	Cost	Profit
1	2023	10	2678.11	1200.15	1477.96
2	2023	11	4305.21	2053.61	2251.60
3	2023	12	2643.14	1286.61	1356.53
4	2024	1	2504.52	1167.62	1336.90
5	2024	2	2368.51	1116.82	1251.69
6	2024	3	4487.22	2183.29	2303.93

Regional Profitability

Businesses want to know which regions generate the most profit.

SQL analysis can group revenue and profit by customer region.

This helps identify strong and weak markets for investment.

Regional profit analysis supports sales and marketing planning.

It also helps adjust logistics and supply chain decisions.

Profit by region can be compared with regional costs and growth.

Dashboards often show a map view of profitability by geography.

Regional insights drive targeted business strategies.

SQL Example – Profit by Region

```
58
59
60  /* PROFIT BY REGION (BooksDemo)
61     Returns one row per Region with: Revenue, Cost (COGS), Profit.
62     Joins: Customers -> Orders -> OrderItems -> Books (for UnitCost).
63     Revenue = SUM(Quantity * UnitPrice)
64     Cost    = SUM(Quantity * UnitCost)
65     Profit  = Revenue - Cost
66  */
67  SELECT
68      COALESCE(c.Region, 'Unknown') AS Region, -- region label
69      SUM(oi.Quantity * oi.UnitPrice) AS Revenue, -- sales $
70      SUM(oi.Quantity * b.UnitCost) AS Cost, -- COGS $
71      SUM(oi.Quantity * (oi.UnitPrice - b.UnitCost)) AS Profit -- gross profit $
72  FROM ba.Customers AS c
73  JOIN ba.Orders AS o ON o.CustomerID = c.CustomerID -- brings OrderDate, links buyer
74  JOIN ba.OrderItems AS oi ON oi.OrderID = o.OrderID -- line qty & sell price
75  JOIN ba.Books AS b ON b.BookID = oi.BookID -- unit cost per book
76  -- WHERE o.OrderDate >= '2024-01-01' -- optional time window
77  GROUP BY COALESCE(c.Region, 'Unknown')
78  ORDER BY Profit DESC; -- most profitable regions first
79
80
```

85 % No issues found

Results Messages

	Region	Revenue	Cost	Profit
1	South	20165.84	9557.80	10608.04
2	West	19526.20	9114.55	10411.65
3	East	18252.26	8577.85	9674.41
4	North	16225.19	7746.16	8479.03

Understanding Business Application CPA



Revenue analysis shows how much money the company earns.



Managers use revenue reports to track sales growth trends.



Revenue is often broken down by product, region, or time period.



High revenue products may receive more investment and promotion.



Low revenue products may be discontinued or redesigned.



Revenue is a common metric in marketing and sales dashboards.



It helps measure effectiveness of campaigns and strategies.



Revenue growth is often a key target for management teams.

Business Applications – Revenue

Business Applications – Costs

- ❑ Cost analysis helps managers control and reduce expenses.
- ❑ Fixed costs are managed by long-term contracts and efficiency.
- ❑ Variable costs are controlled through supplier management.
- ❑ Cost breakdowns show which areas consume the most resources.
- ❑ Reducing costs without lowering quality improves profit margins.
- ❑ Cost reports are often reviewed by operations and finance teams.
- ❑ Tracking costs over time highlights waste or inefficiencies.
- ❑ Cost analysis ensures business remains competitive and profitable.



Business Applications – Profit

Profit is the ultimate measure of business success.

Investors evaluate profit to assess company performance.

Managers use profit trends to make expansion decisions.

Profit can be analyzed per product, region, or business line.

Comparing profit margins across regions highlights efficiency.

Profit reports influence strategic choices in pricing and costs.

Sustained profit growth shows long-term business health.

Profit analysis supports forecasting and financial planning.

Dashboards for Revenue, Costs, and Profit

- ❑ Business dashboards visualize key financial metrics clearly.
- ❑ Revenue charts show sales growth and performance by segment.
- ❑ Cost charts highlight areas where expenses are rising or falling.
- ❑ Profit charts show whether the business is sustainable.
- ❑ Dashboards often compare revenue vs profit on a single chart.
- ❑ Geographic dashboards display profit by region on maps.
- ❑ Power BI and Excel are common tools for creating dashboards.
- ❑ Dashboards help executives make fast and informed decisions.

Lab Exercise – Revenue, Cost, Profit

Step 1: Connect to the sales database in SQL Server Management Studio.

Step 2: Write a SQL query to calculate revenue, cost, and profit.

Step 3: Group results by month using YEAR() and MONTH() functions.

Step 4: Verify results by checking totals with SUM().

Monthly Revenue, Cost, Profit

```

79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103

```

```

--Monthly Revenue, Cost, Profit--
/*
    Show revenue, cost, and profit for each month so managers can see trends.

    WHAT IT DOES:
    - Uses order date to group by Year and Month.
    - Revenue = SUM(Quantity * UnitPrice)
    - Cost     = SUM(Quantity * UnitCost)
    - Profit   = Revenue - Cost
    - Tables:  Orders (for date), OrderItems (qty/price), Books (cost)
*/
SELECT
    YEAR(o.OrderDate) AS Yr,           -- year of the order
    MONTH(o.OrderDate) AS Mn,         -- month of the order
    SUM(oi.Quantity * oi.UnitPrice) AS Revenue, -- sales $
    SUM(oi.Quantity * b.UnitCost) AS Cost,      -- COGS $
    SUM(oi.Quantity * (oi.UnitPrice - b.UnitCost)) AS Profit -- gross profit $
FROM ba.Orders AS o -- has OrderDate
JOIN ba.OrderItems AS oi ON oi.OrderID = o.OrderID -- line items
JOIN ba.Books AS b ON b.BookID = oi.BookID -- unit cost
-- WHERE o.OrderDate >= '2024-01-01' -- (optional) limit to a period
GROUP BY YEAR(o.OrderDate), MONTH(o.OrderDate) -- one row per month
ORDER BY Yr, Mn; -- oldest -> newest

```

5 % No issues found

Results		Messages			
	Yr	Mn	Revenue	Cost	Profit
1	2023	10	2678.11	1200.15	1477.96
2	2023	11	4305.21	2053.61	2251.60
3	2023	12	2643.14	1286.61	1356.53
4	2024	1	2504.52	1167.62	1336.90
5	2024	2	2368.51	1116.82	1251.69

One Total Line (Company-wide)

```
103 -----One Total Line (Company-wide)-----
104  /*
105   Give me the overall totals: revenue, cost, and profit for all time.
106
107   WHAT IT DOES:
108   - Adds up all lines in OrderItems.
109   - Pulls each book's UnitCost from Books.
110   - No GROUP BY → returns a single summary row.
111  */
112  SELECT
113      SUM(oi.Quantity * oi.UnitPrice) AS Revenue, -- total sales $
114      SUM(oi.Quantity * b.UnitCost) AS Cost, -- total COGS $
115      SUM(oi.Quantity * (oi.UnitPrice - b.UnitCost)) AS Profit -- total gross profit $
116  FROM ba.OrderItems AS oi
117  JOIN ba.Books AS b ON b.BookID = oi.BookID;
118  -- To limit by time, join Orders and add WHERE OrderDate >= '2024-01-01'
119
```

85 % No issues found

Results Messages

	Revenue	Cost	Profit
1	74169.49	34996.36	39173.13

Lab Exercise – Regional Profitability

Step 1: Join Customers, Orders, OrderItems, and Products tables.

Step 2: Calculate revenue, cost, and profit by region.

Step 3: Use GROUP BY Region to separate results geographically.

Step 4: Run the query and record the results per region.

Step 5: Discuss business strategies to improve weak regions.

Regional Profitability

```
103      -----REGIONAL PROFITABILITY-----
104      /* PROBLEM: Show revenue, cost, and profit by customer region (simple). */
105      SELECT
106          COALESCE(c.Region, 'Unknown')          AS Region,          -- geography
107          SUM(oi.Quantity * oi.UnitPrice)         AS Revenue,        -- sales $
108          SUM(oi.Quantity * b.UnitCost)           AS Cost,           -- COGS $
109          SUM(oi.Quantity * (oi.UnitPrice - b.UnitCost)) AS Profit    -- gross profit $
110      FROM ba.Customers AS c
111      JOIN ba.Orders    AS o  ON o.CustomerID = c.CustomerID
112      JOIN ba.OrderItems AS oi ON oi.OrderID   = o.OrderID
113      JOIN ba.Books     AS b  ON b.BookID     = oi.BookID
114      -- WHERE o.OrderDate >= '2024-01-01'      -- optional period
115      GROUP BY COALESCE(c.Region, 'Unknown')
116      ORDER BY Profit DESC;                    -- best → worst
117
```

85 % No issues found

Results Messages

	Region	Revenue	Cost	Profit
1	South	20165.84	9557.80	10608.04
2	West	19526.20	9114.55	10411.65
3	East	18252.26	8577.85	9674.41
4	North	16225.19	7746.16	8479.03

SQL Example – Cumulative Revenue

```
119  /* PROBLEM: Show monthly revenue and a running total (cumulative revenue). */
120
121  /* Step 1: Get revenue per calendar month */
122  WITH m AS (
123      SELECT
124          YEAR(o.OrderDate) AS Yr,
125          MONTH(o.OrderDate) AS Mn,
126          SUM(oi.Quantity * oi.UnitPrice) AS MonthlyRevenue -- revenue this month
127      FROM ba.Orders o
128      JOIN ba.OrderItems oi ON oi.OrderID = o.OrderID
129      -- WHERE o.OrderDate >= '2024-01-01' -- optional period filter
130      GROUP BY YEAR(o.OrderDate), MONTH(o.OrderDate)
131  )
132
133  /* Step 2: Add a running total using a window function */
134  SELECT
135      Yr,
136      Mn,
137      MonthlyRevenue,
138      SUM(MonthlyRevenue) OVER (ORDER BY Yr, Mn
139                               ROWS UNBOUNDED PRECEDING) AS CumulativeRevenue
140  FROM m
141  ORDER BY Yr, Mn; -- oldest -> newest
142
```

85 % No issues found

Results Messages

	Yr	Mn	MonthlyRevenue	CumulativeRevenue
1	2023	10	2678.11	2678.11
2	2023	11	4305.21	6983.32
3	2023	12	2643.14	9626.46
4	2024	1	2504.52	12130.98
5	2024	2	2368.51	14499.49
6	2024	3	4487.22	18986.71
7	2024	4	4085.79	23072.50

Contribution Margin

- ❑ Contribution margin measures how much revenue exceeds variable costs.
- ❑ **Formula: Contribution Margin = Revenue – Variable Costs.**
- ❑ It helps businesses understand profitability per unit sold.
- ❑ High contribution margin means strong control over variable costs.
- ❑ Low contribution margin signals issues with pricing or cost control.
- ❑ SQL can compute this by separating fixed and variable costs.
- ❑ Contribution margin supports break-even and pricing analysis.
- ❑ It is widely used in managerial accounting and business planning.



Break-even Analysis

- ❑ Break-even point is the sales level where profit becomes zero.
- ❑ **Formula: Break-even = Fixed Costs ÷ Contribution Margin per unit.**
- ❑ It tells businesses how many units must be sold to cover costs.
- ❑ SQL can assist by computing contribution margin values.
- ❑ Example: If fixed costs are \$10,000 and margin per unit \$50, break-even = 200 units.
- ❑ Break-even analysis guides managers in setting sales targets.
- ❑ It also informs decisions about pricing and discounts.
- ❑ Businesses use this to evaluate new product launches.

Break-even Analysis

```
143 -- Break-even Analysis Example
144 -- Assume Fixed Costs = 10000
145 -- Contribution Margin = (UnitPrice - UnitCost)
146
147 SELECT
148     b.Title,
149     b.UnitPrice,
150     b.UnitCost,
151     (b.UnitPrice - b.UnitCost) AS ContributionMarginPerUnit,
152     CAST(10000.0 / (b.UnitPrice - b.UnitCost) AS INT) AS BreakEvenUnits
153 FROM ba.Books b
154 WHERE b.Title = 'The Art of Finance for Kids'; -- Example book
155
```

85 %  No issues found

 Results  Messages

	Title	UnitPrice	UnitCost	ContributionMarginPerUnit	BreakEvenUnits
1	The Art of Finance for Kids	24.49	14.72	9.77	1023

Revenue Up, Profit Down

- ❑ Sometimes revenue grows while profit declines at the same time.
- ❑ SQL analysis helps identify why profit is falling despite sales growth.
- ❑ Possible causes: rising costs, heavy discounts, or low margins.
- ❑ Example: SQL shows revenue up 15% but costs up 25%, lowering profit.
- ❑ Managers must identify whether variable or fixed costs are driving losses.
- ❑ Business lesson: profit analysis is as important as revenue tracking.
- ❑ Analysts must check both revenue and cost queries together.
- ❑ SQL provides evidence to explain financial performance changes.



Visualizing Profitability in Dashboards

SQL results can be exported to visualization tools like Power BI or Excel.

Dashboards show trends in revenue, costs, and profits visually.

Common visuals: line chart for monthly profit, bar chart by region.

Pie charts often display cost breakdowns by category.

Dashboards allow managers to interact with filters and slicers.

Combining SQL data with visuals makes insights easier to consume.

Executives prefer dashboards to raw tables for decision making.

SQL powers the backend, dashboards deliver the business value.

Q&A



KPIs for Business Analysis



Day 07

List of Contents

- ❑ KPI basics – definition, SMART, leading vs lagging
- ❑ Marketing KPIs – conversion, new buyers, repeat rate
- ❑ Sales KPIs – revenue, AOV, basket size, growth %
- ❑ Ops KPIs – inventory turnover, days of cover, alerts
- ❑ Customer KPIs – active customers, ARPC, churn risk
- ❑ KPI dashboards – cards & monthly trends

Learning Objectives

- ❑ Define KPI and differentiate leading vs lagging indicators
- ❑ Design SMART KPIs that are specific, measurable, and actionable
- ❑ Explain marketing, sales, ops, and customer KPIs in business terms
- ❑ Build monthly KPI tables using GROUP BY and window functions
- ❑ Create alerts for operational risks from SQL outputs
- ❑ Interpret KPIs and connect them to business actions

What is a KPI?

KPI = Key Performance Indicator: a measurable signal of performance

KPIs track progress toward goals, not just raw numbers or activity

They are aligned to business outcomes (revenue, retention, profit)

Leading KPIs predict outcomes; lagging KPIs confirm outcomes

Examples: Conversion rate (leading), Profit margin (lagging)

KPIs must be defined unambiguously: unit, period, dimensions

Data source matters: here, all KPIs come from BooksDemo SQL

KPIs drive better decisions when trended and compared



SMART KPI Design

S = Specific (clear definition, owner, and scope)

M = Measurable (numeric value with transparent calculation)

A = Achievable (realistic targets given resources and constraints)

R = Relevant (aligned to business outcomes and priorities)

T = Time-bound (measured Daily/Weekly/Monthly/Quarterly)

Include segment breakdowns (region, category, customer segment)

Document formula, tables, and filters inside the slide deck

Use the same SQL query each month for consistent comparison

Map KPIs to BooksDemo Tables

- ❑ Customers: CustomerID, Region, SignupDate (marketing, customer KPIs)
- ❑ Orders: OrderID, CustomerID, OrderDate (time-based metrics)
- ❑ OrderItems: OrderID, BookID, Quantity, UnitPrice (revenue metrics)
- ❑ Books: BookID, UnitCost, UnitPrice, StockQty, Category (ops)
- ❑ Categories: CategoryName (category level breakdowns)
- ❑ Join keys: Orders↔OrderItems (OrderID), Orders↔Customers (CustomerID)
- ❑ Costs via Books.UnitCost; revenue via Quantity×UnitPrice
- ❑ All examples today use only these base tables

Marketing KPI

First Purchase Conversion (30 days)

```
1  /* PROBLEM:
2   What % of signups place their FIRST order within 30 days?
3   GOAL:
4   Calculate a single conversion rate for all signups (BooksDemo).
5   */
6
7  WITH first_order AS (
8    SELECT o.CustomerID, MIN(o.OrderDate) AS FirstOrderDate
9    FROM ba.Orders o
10   GROUP BY o.CustomerID
11  )
12  SELECT
13    CAST(
14      100.0 * SUM(
15        CASE
16          WHEN fo.FirstOrderDate IS NOT NULL
17            AND DATEDIFF(day, c.SignupDate, fo.FirstOrderDate) BETWEEN 0 AND 30
18            THEN 1 ELSE 0
19          END
20      ) / COUNT(*) AS DECIMAL(10,2)
21    ) AS Conversion30DayPct
22  FROM ba.Customers c
23  LEFT JOIN first_order fo ON fo.CustomerID = c.CustomerID;
24
25  /* EXPLANATION (easy):
26  1) first_order finds each customer's earliest OrderDate.
27  2) LEFT JOIN keeps every signup, even if they never ordered.
28  3) Numerator: signups whose first order happened within 0-30 days.
29  4) Denominator: all signups returned by the query.
30  5) CAST gives a clean percentage (e.g., 23.45).|
31  */
32
```

85 % No issues found

Results Messages

	Conversion30DayPct
1	2.00

SQL – New Buyers per Month

```
33  /* PROBLEM:
34  How many **new buyers** (customers placing their FIRST order) do we get each month?
35  GOAL:
36  Count first-time purchasers by Year/Month using BooksDemo tables.
37  */
38
39  WITH first_order AS (
40  SELECT o.CustomerID, MIN(o.OrderDate) AS FirstOrderDate
41  FROM ba.Orders AS o
42  GROUP BY o.CustomerID
43  )
44  SELECT
45  YEAR(FirstOrderDate) AS Yr,
46  MONTH(FirstOrderDate) AS Mn,
47  COUNT(*) AS NewBuyers
48  FROM first_order
49  WHERE FirstOrderDate IS NOT NULL
50  GROUP BY YEAR(FirstOrderDate), MONTH(FirstOrderDate)
51  ORDER BY Yr, Mn;
52
53  /*
54  1) first_order finds each customer's earliest order date.
55  2) We count those first orders per Year/Month → "new buyers per month".
56  3) WHERE filters out customers with no orders (if any).
57  4) Use this to track acquisition momentum.
58  . */
59
```

85 % No issues found

Results Messages

	Yr	Mn	NewBuyers
1	2023	10	15
2	2023	11	18
3	2023	12	15
4	2024	1	10
5	2024	2	7

Average Order Value (AOV) & Basket Size

```
61  /* PROBLEM:
62  What is our Average Order Value (AOV) and average basket size (units per order)?
63  GOAL:
64  Compute two KPIs from transaction lines in BooksDemo.
65  */
66
67  WITH order_totals AS (
68  SELECT
69      oi.OrderID,
70      SUM(oi.Quantity * oi.UnitPrice) AS OrderValue, -- $ value of this order
71      SUM(oi.Quantity) AS Units -- total units in this order
72  FROM ba.OrderItems AS oi
73  -- Optional: JOIN ba.Orders o ON o.OrderID = oi.OrderID
74  -- WHERE o.OrderDate >= '2024-01-01' -- limit to a period
75  GROUP BY oi.OrderID
76  )
77  SELECT
78      CAST(AVG(OrderValue) AS DECIMAL(12,2)) AS AOV, -- avg $ per order
79      CAST(AVG(Units * 1.0) AS DECIMAL(12,2)) AS AvgUnitsPerOrder -- avg items per order
80  FROM order_totals;
81
82  /* EXPLANATION:
83  1) order_totals collapses each order to two numbers: OrderValue and Units.
84  2) Final SELECT averages across orders → AOV and average basket size.
85  3) Add the optional JOIN/WHERE if you want AOV for a specific time window.
86  Business use:
87  - AOV reflects pricing, cross-sell, and upsell effectiveness.
88  - Avg units/order shows basket building and promo strength. */
```

85 % No issues found

Results Messages

	AOV	AvgUnitsPerOrder
1	185.42	6.11

SQL – Monthly Revenue Growth %

```
90
91      /* PROBLEM: Show monthly revenue and % growth vs previous month (simple). */
92  WITH m AS (
93      SELECT
94          DATEFROMPARTS(YEAR(o.OrderDate), MONTH(o.OrderDate), 1) AS MonthStart,
95          SUM(oi.Quantity * oi.UnitPrice) AS Revenue
96      FROM ba.Orders o
97      JOIN ba.OrderItems oi ON oi.OrderID = o.OrderID
98      -- WHERE o.OrderDate >= '2024-01-01' -- optional filter
99      GROUP BY DATEFROMPARTS(YEAR(o.OrderDate), MONTH(o.OrderDate), 1)
100  )
101  SELECT
102      m.MonthStart,
103      m.Revenue,
104      /* Grab previous month's revenue with a tiny correlated subquery */
105      (SELECT m2.Revenue
106       FROM m AS m2
107       WHERE m2.MonthStart = DATEADD(month, -1, m.MonthStart)) AS PrevRevenue,
108      CAST(
109          (m.Revenue
110           - (SELECT m2.Revenue FROM m AS m2
111             WHERE m2.MonthStart = DATEADD(month, -1, m.MonthStart)))
112          * 100.0 /
113          NULLIF((SELECT m2.Revenue FROM m AS m2
114                 WHERE m2.MonthStart = DATEADD(month, -1, m.MonthStart)), 0)
115          AS DECIMAL(10,2)
116      ) AS GrowthPct
117  FROM m
118  ORDER BY m.MonthStart;
119  /* Explanation :
120  - m: one row per calendar month with total Revenue.
121  - The small subquery fetches the revenue for the prior month.
122  - GrowthPct = (Revenue - PrevRevenue) / PrevRevenue * 100.
123  - First month will show NULL for PrevRevenue (no prior month). */
```

76 % No issues found

Results Messages

	MonthStart	Revenue	PrevRevenue	GrowthPct
1	2023-10-01	2678.11	NULL	NULL
2	2023-11-01	4305.21	2678.11	60.76
3	2023-12-01	2643.14	4305.21	-38.61
4	2024-01-01	2504.52	2643.14	-5.24

Active Customers (Monthly Buyers)

```
127
128      /* PROBLEM:
129      How many UNIQUE customers placed at least one order in each month?
130      (Treat this as "Monthly Active Buyers" - MAU for purchasers.)
131      GOAL:
132      A simple monthly count of distinct buyers from BooksDemo.
133      */
134
135      SELECT
136      YEAR(o.OrderDate) AS Yr,
137      MONTH(o.OrderDate) AS Mn,
138      COUNT(DISTINCT o.CustomerID) AS ActiveCustomers
139      FROM ba.Orders AS o
140      -- WHERE o.OrderDate >= '2024-01-01' -- optional time window
141      GROUP BY YEAR(o.OrderDate), MONTH(o.OrderDate)
142      ORDER BY Yr, Mn;
143
144      /* EXPLANATION (easy):
145      - COUNT(DISTINCT o.CustomerID) = number of unique buyers per month.
146      - Buyers (people who ordered) ≠ signups; this is a monetization KPI.
147      - Add the WHERE to limit to recent months if needed. */
148
```

76 % No issues found

Results Messages

	Yr	Mn	ActiveCustomers
1	2023	10	15
2	2023	11	21
3	2023	12	18
4	2024	1	13
5	2024	2	14
6	2024	3	23
7	2024	4	24
8	2024	5	13

SQL – Churn Risk (No Purchase in 90 Days)

```
148      -----CHURN CUSTOMERS-----
149      /* PROBLEM: List customers who haven't purchased in the last 90 days (churn risk). */
150
151      SELECT
152          c.CustomerID,
153          c.FirstName,
154          c.LastName,
155          MAX(o.OrderDate) AS LastOrder,           -- last purchase date
156          DATEDIFF(day, MAX(o.OrderDate), CAST(GETDATE() AS date)) AS DaysSinceOrder
157      FROM ba.Customers c
158      LEFT JOIN ba.Orders o ON o.CustomerID = c.CustomerID      -- keep customers with no orders
159      GROUP BY c.CustomerID, c.FirstName, c.LastName
160      HAVING MAX(o.OrderDate) IS NULL                -- never purchased
161          OR MAX(o.OrderDate) < DATEADD(day, -90, CAST(GETDATE() AS date)) -- last purchase > 90 days ago
162      ORDER BY LastOrder;                             -- NULLs (never) appear first
163
164      /* Easy explanation:
165      - LEFT JOIN keeps everyone; orders may be NULL.
166      - MAX(OrderDate) = most recent purchase per customer.
167      - HAVING filters to "never ordered" OR "last order older than 90 days".
168      - Change 90 to 30/60/etc to adjust the window. */
169
```

76 % No issues found

Results Messages

	CustomerID	FirstName	LastName	LastOrder	DaysSinceOrder
1	2	Carol	Campbell	NULL	NULL
2	5	Dorothy	Miller	NULL	NULL
3	7	Margaret	Nguyen	NULL	NULL
4	13	Barbara	Thompson	NULL	NULL
5	15	Nancy	Perez	NULL	NULL
6	18	Richard	Roberts	NULL	NULL
7	33	Deborah	Williams	NULL	NULL
8	34	Michelle	Gonzalez	NULL	NULL
9	46	Mark	Gonzalez	NULL	NULL

KPI Dashboard – Cards & Trends

Element	Details
Cards (Set 1)	Revenue, Orders, AOV, Active Customers, Repeat Rate
Cards (Set 2)	ARPC, Inventory Turnover, Days of Cover, Alerts Count
Trends	Monthly Revenue & Growth %, Active Customers Trend
Breakdowns	Revenue by Category, Region, Top 10 Books
Filters	Month, Region, Category
Data Source	SQL views or daily exports
Audience	Leadership (cards) and Managers (trends)
Guideline	Prioritize clarity, consistency

Business Interpretation – From KPIs to Actions

If AOV is low: consider bundle pricing and cross-sell offers

If Repeat Rate drops: launch retention emails to past buyers

If Growth % slows: analyze category/region mix for issues

If ARPC declines: prices may be too low or discounts too high

If Days of Cover is low: expedite purchase orders for top books

If Alerts spike: rebalance inventory and update forecasting

If New Buyers stall: improve signup-to-purchase onboarding

Always assign owners, targets, and review cadence for KPIs

KPI Glossary – Clear Definitions

KPI: a key, decision-driving metric with an owner and target

Metric: any measured value; not all metrics are KPIs

Dimension: a way to slice metrics (Region, Category, Month)

Leading indicator: predicts outcomes (conversion rate)

Lagging indicator: confirms outcomes (profit margin)

North Star Metric: the single KPI that best reflects value

Operational metric: daily health checks (stock alerts)

Diagnostic metric: helps explain why a KPI moved

KPI vs Metric vs Dimension

Type	Example	Owner/Use
KPI	Monthly Revenue growth %	Sales lead
KPI	Repeat Purchase Rate	CRM lead
Metric	Basket size (units per order)	Context measure
Metric	Category revenue share %	Mix understanding
Dimension	Region (North/South/East/West)	Segmentation
Dimension	Category (Fiction/Tech/etc.)	Product view
Dimension	Customer tenure (new vs returning)	Customer analysis

KPI Tree – From North Star to Drivers



Targets & Thresholds – Make KPIs Actionable



Set annual target,
break into
quarterly/monthly
checkpoints



Define
green/amber/red
thresholds for each
KPI



Use absolute and
relative moves (e.g.,
-2pp vs -10%)



Tie alerts to owners
with time-bound
follow-ups



Document exceptions
and seasonality
adjustments



Avoid target gaming:
publish calculation
logic



Store definitions in a
shared KPI catalog



Review targets in
monthly business
reviews

Ownership & Cadence – Keep KPIs Alive



Each KPI has an OWNER and a backup; no orphan KPIs



Weekly health checks; monthly deep dives; quarterly reset



Decision forum: which meeting reviews which KPI



Version control on KPI definitions and SQL views



Change log: who changed logic/thresholds and why



Consistent runbook for data refresh and validations



Escalation path when KPI breaches thresholds



Celebrate improvements; run post-mortems for misses

Data Lineage

—

BooksDemo

→ KPIs

**Revenue KPIs: Orders + OrderItems
(Quantity×UnitPrice)**

Cost/COGS: OrderItems joined to Books.UnitCost

**Customer KPIs: Customers joined to Orders by
CustomerID**

Ops KPIs: Books.StockQty + recent sales windows

Dimensions: Categories (product), Region (customer)

Time: OrderDate is the canonical event date

**Document joins/filters in a lineage diagram
(slide/Confluence)**

One source of truth: use views for repeated KPI logic

Data Quality Checklist (Before Publishing KPIs)

Inspect	Nulls and outliers: inspect recent months for anomalies
Duplicate	Duplicate orders/items: enforce PKs and unique constraints
Time	Time gaps: ensure complete coverage in the period
Price	Price vs cost sanity: $\text{UnitPrice} \geq \text{UnitCost}$ generally
Join	Join keys coverage: unmatched IDs and referential integrity
Recompute on	Recompute on a sample by hand to spot logic errors
Reconcile	Reconcile totals with finance/ops when applicable
Add	Add automated validation rules to your pipeline

Dashboard Design – Best Practices

Start with 4–6 KPI cards (north star + drivers)

Trends on the left; breakdowns on the right

Use consistent time windows and color scales

Add filters for Month/Region/Category only

Limit tables to top/bottom items with sorting

Annotate spikes/dips with business events

Provide a one-slide KPI glossary in the deck

Test readability on projector and mobile

Books KPI Dashboard



Chart Choice Guide – Tell the Right Story

Chart Type	When to Use	Example KPI / Metric
Line Chart	Show trends over time	Revenue, Buyers, Growth %
Bar Chart	Compare categories or regions	Category/Region revenue mix
Stacked Bar	Show share of total	Category share %
Combo (Line+Bar)	Overlay trends with breakdowns	Revenue line with Growth % bars
Scatter Plot	Relationship between two variables	Price vs Volume, Margin vs Revenue
Table	Detailed ranking with highlights	Top 10 books with conditional colors
Map	Geographical distribution (optional)	Region performance



Scenario: Email promo for Technology books in April



Observed: Revenue +12%, AOV +8%, Repeat Rate +3pp



But: Profit margin -2pp due to higher discounts



Inventory: Days of cover fell to 9 on top sellers



Action: Reduce discount to 10%, expedite replenishment



Next step: A/B test subject lines to lift conversion



Lesson: Balance growth with profitability and stock



Document: include KPI snapshots before/after campaign

Case Study – Campaign Effect (Interpretation)

Case Study – Inventory Risk (Interpretation)

Scenario: Two fantasy titles drive 25% of monthly revenue

Signal: Low-stock + high-velocity alerts spiked this week

Risk: Stock-out could reduce AOV and repeat purchases

Action: Rush PO; prioritize inbound; cap deep discounts

Communication: sales/marketing pause heavy promos

Follow-up: increase safety stock for top-velocity SKUs

Add KPI: service level % or fill rate for priority items

Track: days of cover trend after replenishment arrives

Pitfalls & Anti-Patterns



Metric soup: too many metrics; few true KPIs



Shifting definitions: changing formulas without notice



Vanity metrics: page views without conversion context



Inconsistent time windows across visuals



Comparing apples to oranges (mixed filters/segments)



Target gaming: optimizing the number, not the outcome



SQL drift: multiple versions of the same calculation



No owner: KPIs without accountability fade away

Ethics & Fair Use of Metrics

Respect	Respect privacy: avoid exposing identifiable customer data
Use	Use aggregated views for dashboards when possible
Be	Be transparent about discounts and pricing impacts
Ensure	Ensure thresholds don't penalize healthy seasonality
Avoid	Avoid biased segments; validate fairness in actions
Document	Document assumptions behind KPIs and cohorts
Provide	Provide opt-out for experimental groups (A/B tests)
Review	Review ethics with leadership for sensitive KPIs

Summary

Today we defined KPIs and designed SMART indicators

We mapped KPIs to BooksDemo and wrote SQL for each KPI

Marketing: conversion, new buyers, repeat purchase rate

Sales: revenue, AOV, basket size, growth %, mix by category

Ops: turnover proxy, days of cover, low-stock alerts

Customer: active customers, ARPC, churn risk list

We planned card + trend dashboards and two hands-on labs

Q&A



MAU, DAU, and Cohort Retention Analysis



Day 08

List of Contents

- ❑ Understanding Daily and Monthly Active Users (DAU & MAU)
- ❑ Importance of engagement metrics for business success
- ❑ SQL queries to calculate DAU and MAU
- ❑ User growth trends and ratios
- ❑ Cohort Analysis and retention tracking
- ❑ Real-world examples and practical cases

Learning Objectives

- ❑ Define and differentiate DAU and MAU.
- ❑ Write simple SQL queries to calculate active users.
- ❑ Calculate and interpret user growth rates.
- ❑ Explain DAU/MAU ratio and what it means for engagement.
- ❑ Understand Cohort Analysis and retention trends.
- ❑ Link SQL results with business insights.
- ❑ Identify areas of improvement from user activity data.
- ❑ Prepare for next-level analysis and dashboards.

DAU and MAU

How Engagement Leads to Sustainable Growth



- 🔍 Every business begins by tracking user engagement —
- 👤 who's active daily or monthly (DAU & MAU).
- 📈 Sustained engagement builds retention and loyalty over time (cohorts & repeat users).
- 📊 High retention creates a foundation for consistent growth in users and revenue.
- 🔄 DAU → Retention → Growth forms a continuous feedback cycle.
- 📉 These metrics together show both short-term usage and long-term business health.
- 🔍 SQL helps measure each step, turning raw data into actionable insights.

What Are Active Users



Active users are people who interact with a business or app within a defined period.

Common timeframes are daily (DAU) and monthly (MAU).

DAU = unique users active per day.

MAU = unique users active per month.

Tracking them shows engagement and growth.

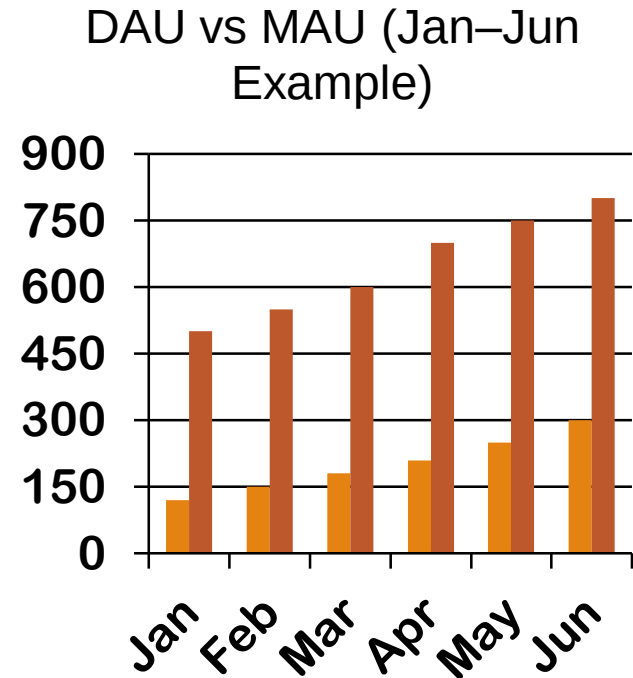
Higher numbers mean stronger interest and retention.

A drop signals a problem in engagement or usability.

SQL helps identify these users easily from transaction data.

Comparing (DAU) vs (MAU)

- ❑ DAU represents the number of unique users active each day.
- ❑ MAU tracks how many unique users were active at least once in the month.
- ❑ Higher DAU means stronger daily engagement and product stickiness.
- ❑ A rising MAU shows that new users are adopting the product.
- ❑ Comparing DAU and MAU helps identify both short-term and long-term user activity.



Importance of DAU and MAU

Businesses use DAU and MAU to monitor performance.

DAU measures everyday activity and short-term engagement.

MAU captures the overall size of the active user base.

Comparing both shows how often users return.

A steady increase shows healthy user growth.

A fall in DAU but stable MAU means less frequent use.

Ratio analysis (DAU/MAU) indicates loyalty.

These numbers help plan marketing and growth actions.

SQL Query: Daily Active Users (DAU)

Day-8 Demo.s...A\jia_a (52))*

```
1
2  /* =====
3     Problem: Find how many unique customers made a purchase each day
4     Goal: Compute Daily Active Users (DAU)
5     Table: ba.Orders
6     ===== */
7
8  SELECT
9     CAST(o.OrderDate AS DATE) AS OrderDate,           -- ensures we only use the date part
10    COUNT(DISTINCT o.CustomerID) AS DailyActiveUsers -- counts each customer once per day
11  FROM ba.Orders AS o
12  GROUP BY CAST(o.OrderDate AS DATE)                  -- one result per calendar day
13  ORDER BY OrderDate;                                -- sorted chronologically
14
15  /* =====
16     Explanation:
17     - CAST(OrderDate AS DATE): removes time, so all same-day orders group together.
18     - COUNT(DISTINCT CustomerID): counts unique customers per day.
19     - GROUP BY: aggregates results by each date.
20     - ORDER BY: displays daily activity trend over time.
21     ===== */
22
23
```

76 % 1 0

Results Messages

	OrderDate	DailyActiveUsers
1	2023-10-03	2
2	2023-10-04	1
3	2023-10-05	1
4	2023-10-12	1
5	2023-10-13	1

SQL Query: Monthly Active Users (MAU)

```
44
22
23 -----MONTHLY USERS-----
24
25 /* =====
26 Problem: Find how many unique customers were active each month
27 Goal: Compute Monthly Active Users (MAU)
28 Table: ba.Orders
29 ===== */
30
31 SELECT
32     YEAR(o.OrderDate) AS [Year],           -- extract year from order date
33     MONTH(o.OrderDate) AS [Month],         -- extract month number
34     COUNT(DISTINCT o.CustomerID) AS MonthlyActiveUsers -- count of unique customers per month
35 FROM ba.Orders AS o
36 GROUP BY YEAR(o.OrderDate), MONTH(o.OrderDate) -- group data by year and month
37 ORDER BY [Year], [Month];                 -- display months in order
38
39 /* =====
40 Explanation:
41 - YEAR() and MONTH() break OrderDate into calendar parts.
42 - COUNT(DISTINCT CustomerID): counts each customer only once per month.
43 - GROUP BY groups all orders for the same month together.
44 - ORDER BY shows how monthly actives change over time.
45 - Ideal for identifying user growth, seasonality, and retention patterns.
46 ===== */
47
```

76 %



1



0



Results



Messages

	Year	Month	MonthlyActiveUsers
1	2023	10	15
2	2023	11	21
3	2023	12	18
4	2024	1	13
5	2024	2	14



What Is DAU/MAU Ratio

Once we know unique users (DAU) and monthly actives (MAU),

we can measure how often users return each month.

DAU/MAU ratio links daily engagement to monthly reach.

Helps businesses see if users use the product regularly or rarely.

A higher ratio = users visit frequently → stronger retention.

A lower ratio = users log in rarely → weaker engagement.

This ratio bridges SQL metrics with business insights.

Ideal next step after calculating DAU and MAU in SQL.

```

48 -----RATIO-----
49
50 /* =====
51 Problem: Find how active monthly users are on a daily basis
52 Goal: Compute DAU/MAU Ratio (Engagement Level)
53 Table: ba.Orders
54 ===== */
55
56 -- Step 1: Count unique users active each day
57 SELECT
58     CAST(OrderDate AS DATE) AS OrderDate,
59     COUNT(DISTINCT CustomerID) AS DailyActiveUsers
60 INTO #DailyUsers
61 FROM ba.Orders
62 GROUP BY CAST(OrderDate AS DATE);
63
64 -- Step 2: Count unique users active each month
65 SELECT
66     YEAR(OrderDate) AS Year,
67     MONTH(OrderDate) AS Month,
68     COUNT(DISTINCT CustomerID) AS MonthlyActiveUsers
69 INTO #MonthlyUsers
70 FROM ba.Orders
71 GROUP BY YEAR(OrderDate), MONTH(OrderDate);
72

```

```

73 -- Step 3: Pick one month (example June 2024)
74 DECLARE @Year INT = 2024;
75 DECLARE @Month INT = 6;
76 -- Step 4: Find average daily users for that month
77 DECLARE @DAU DECIMAL(10,2);
78 DECLARE @MAU DECIMAL(10,2);
79
80 SELECT @DAU = AVG(DailyActiveUsers * 1.0)
81 FROM #DailyUsers
82 WHERE YEAR(OrderDate) = @Year AND MONTH(OrderDate) = @Month;
83
84 SELECT @MAU = MonthlyActiveUsers
85 FROM #MonthlyUsers
86 WHERE Year = @Year AND Month = @Month;
87 -- Step 5: Calculate ratio
88 SELECT
89     @DAU AS DAU,
90     @MAU AS MAU,
91     ROUND(@DAU / @MAU, 2) AS DAU_MAU_Ratio,
92     ROUND((@DAU / @MAU) * 100, 2) AS RatioPercent;
93 -- Step 6: Clean up
94 DROP TABLE #DailyUsers;
95 DROP TABLE #MonthlyUsers;
96
97 /* =====
98 Explanation:
99 - Step 1 & 2: Get daily and monthly unique customers.
100 - Step 3: Choose month/year.
101 - Step 4: Find average daily users in that month.
102 - Step 5: Ratio = DAU ÷ MAU × % of monthly users active daily.
103 - Step 6: Drop temp tables after use.
104 ===== */
105

```

75 % 10 0 ↑ ↓ ◀ ▶

Results Messages

	DAU	MAU	DAU_MAU_Ratio	RatioPercent
1	1.00	10.00	0.100000000000000	10.0000000000000

DAU/MAU Example



From Growth to Retention: Do Users Stay?

After measuring user growth, the next question is — are we keeping those users?

High growth means new users are joining, but retention shows if they continue using the product.

Retention analysis helps identify loyalty, churn, and product satisfaction. We start by finding how many new users joined each month, then track if they stayed active later.

This is where Cohort Analysis begins — grouping users by signup period to measure long-term engagement.

Importance of Growth Rate

- ❑ Growth rate measures the change in active users over time.
- ❑ **Formula: $(\text{Current} - \text{Previous}) \div \text{Previous} \times 100$.**
- ❑ Shows how fast the user base is expanding or shrinking.
- ❑ **Positive growth means the business is gaining traction.**
- ❑ **Negative growth warns of churn or market issues.**
- ❑ Used for both user and revenue tracking.
- ❑ Helps predict future trends.
- ❑ Easy to compute once MAU results are available.

```

105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136
/* ===== MAU GROWTH =====
Problem: Find month-over-month user growth (MAU Growth %)
Goal: See how monthly active users are increasing or slowing down
Table: ba.Orders
===== */

-- Step 1: Count unique active users per month
SELECT
    YEAR(OrderDate) AS Year,
    MONTH(OrderDate) AS Month,
    COUNT(DISTINCT CustomerID) AS MonthlyActiveUsers
INTO #MAU
FROM ba.Orders
GROUP BY YEAR(OrderDate), MONTH(OrderDate)
ORDER BY Year, Month;

-- Step 2: Calculate simple month-over-month growth
SELECT
    m1.Year,
    m1.Month,
    m1.MonthlyActiveUsers AS Current_MAU,
    m2.MonthlyActiveUsers AS Prev_Month_MAU,
    CASE
        WHEN m2.MonthlyActiveUsers IS NULL THEN NULL
        ELSE ROUND(((m1.MonthlyActiveUsers - m2.MonthlyActiveUsers) * 100.0 / m2.MonthlyActiveUsers), 2)
    END AS GrowthPercent
FROM #MAU m1
LEFT JOIN #MAU m2
    ON (m1.Year = m2.Year AND m1.Month = m2.Month + 1) -- joins current month to previous month
ORDER BY m1.Year, m1.Month;

```

```

136 137 138 139 140 141 142 143 144 145 146 147 148
-- Step 3: Clean up
DROP TABLE #MAU;

/* =====
Explanation:
- Step 1: Gets total unique active users per month.
- Step 2: Joins each month with its previous month.
- Growth % = (Current - Previous) / Previous * 100.
- Positive growth = user base expanding.
- Negative growth = requires business investigation.
===== */

```

75 % 10 0 ↑ ↓

Results Messages

	Year	Month	Current_MAU	Prev_Month_MAU	GrowthPercent
1	2023	10	15	NULL	NULL
2	2023	11	21	15	40.000000000000
3	2023	12	18	21	-14.290000000000
4	2024	1	13	NULL	NULL

Monthly Growth Rate Example

Retention and Cohort Analysis

Retention analysis



Retention analysis measures how many users stay active over time.



It shows how well a business keeps customers engaged after signup.



Focuses on users who return in later weeks or months.



Helps identify when customers stop using the product (churn).



Used to evaluate product quality, satisfaction, and loyalty.



Often visualized as a cohort table or retention curve.



High retention = strong engagement; low retention = weak loyalty.



Key metric for improving long-term business growth.

SQL for New Users Per Month

```
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
```

```
-----NEW CUSTOMERS-----
/* =====
Problem: Count how many new customers joined each month
Goal: Track new user sign-ups and growth over time
Table: ba.Customers
===== */

SELECT
    YEAR(SignupDate) AS Year,      -- extract the year from signup date
    MONTH(SignupDate) AS Month,    -- extract the month number
    COUNT(CustomerID) AS NewUsers  -- total new users who joined that month
FROM ba.Customers
GROUP BY
    YEAR(SignupDate),
    MONTH(SignupDate)
ORDER BY
    Year, Month;

/* =====
Explanation:
- YEAR() and MONTH() group users by their signup month.
- COUNT(CustomerID): counts how many joined in each period.
- GROUP BY: ensures one row per month.
- ORDER BY: shows results in time order.
- Helps identify growth, seasonal trends, or marketing impact.
===== */
```

75 % 10 0

Results Messages

	Year	Month	NewUsers
1	2020	4	1
2	2020	5	6
3	2020	6	2
4	2020	7	12

Introducing Cohort Analysis

Cohort = group of users who started at the same time.

Example: users who signed up in January form one cohort.

Used to study retention and engagement over time.

Focuses on behavior patterns, not overall totals.

Helps track loyalty and product value.

Common in SaaS, e-commerce, and mobile apps.

Gives long-term view of user satisfaction.

Easy to start with simple SQL joins.



Why Cohort Analysis Matters

Shows user retention patterns over time.

Identifies when and why users leave.

Measures impact of product or campaign changes.

Helps decide which users to re-engage.

Compares performance of signup groups over time.

Reveals product's long-term sustainability.

Used in dashboards to visualize user lifecycle.

Turns raw data into actionable retention insights.

Interpreting Engagement Trends

Rising DAU and MAU indicate expanding user base.

Rising DAU/MAU ratio signals improving loyalty.

Stable MAU but falling DAU means less frequent use.

High growth but low retention means weak engagement.

High retention but low growth shows loyal niche users.

Balanced growth and retention = healthy user ecosystem.

SQL metrics help balance acquisition and retention.

Businesses track both for sustainable growth.



Real-World Case Studies

Industry	Issue Found	Action Taken	Result
E-commerce	DAU dropping while MAU stable	Launched daily limited-time offer emails	DAU +18%, engagement ratio 0.09 → 0.12
Streaming Platform	DAU dropped sharply, MAU steady	Added new shows and releases	DAU +30%, ratio 0.25 → 0.41
SaaS Product	High churn after free trial	Onboarding & reminder emails	Retention 65% → 78%
Banking App	Low daily activity, MAU stable	Weekend promotional campaigns	MAU +8% QoQ, better weekend logins

Key Learnings Across Industries

- ❑ SQL-based DAU/MAU tracking detected engagement issues early.
- ❑ Data-driven actions boosted retention and user activity.
- ❑ Marketing and onboarding improved daily engagement.
- ❑ Consistent monitoring linked data to business performance.
- ❑ Cohort and churn metrics guided product strategies.
- ❑ Retention directly influenced long-term revenue growth.
- ❑ Demonstrates the power of SQL in business decision-making.



Common Mistakes in SQL Analysis

Forgetting DISTINCT leads to duplicate counts.

Confusing OrderDate with SignupDate causes errors.

Ignoring NULL values for inactive or missing users.

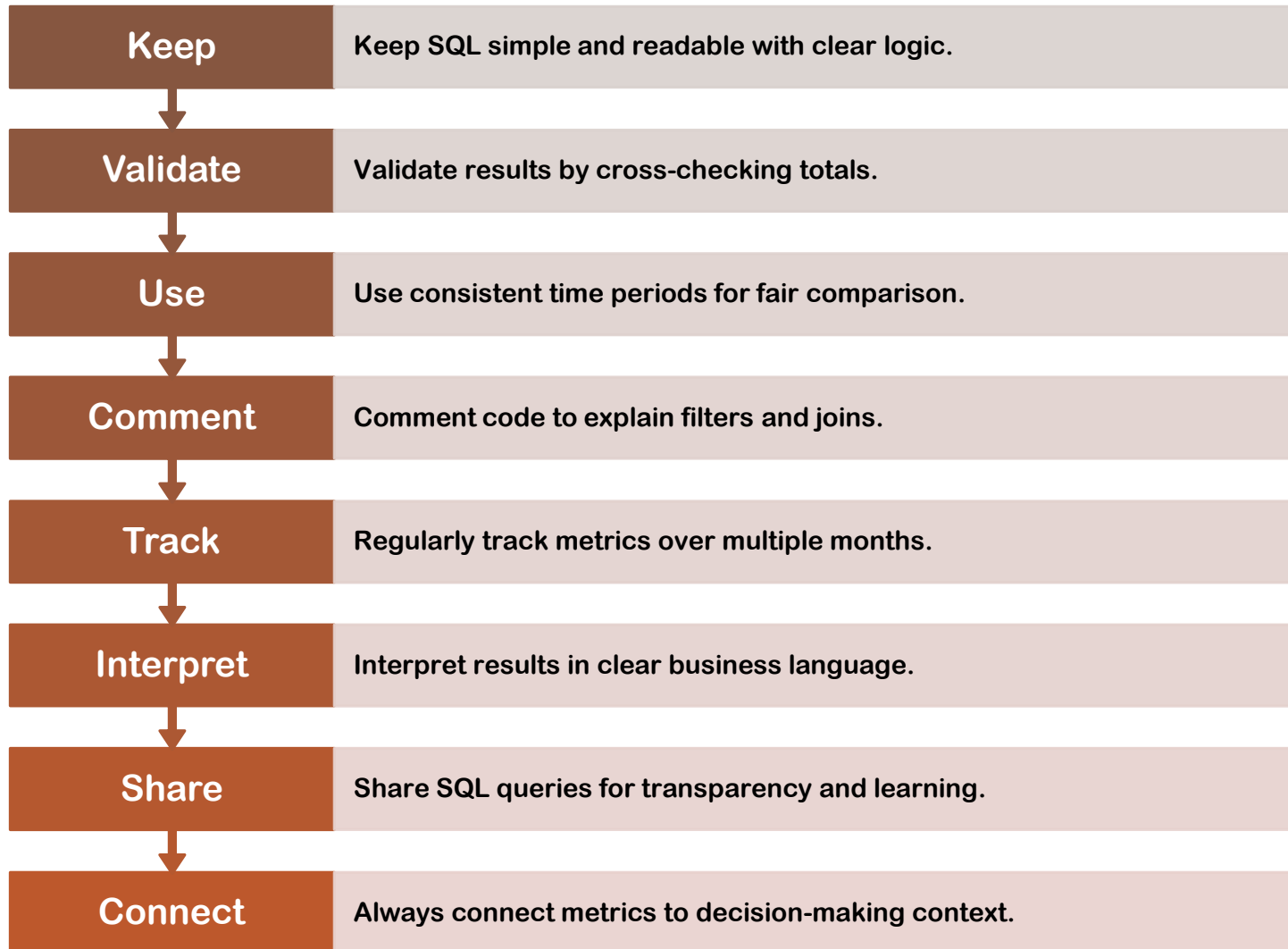
Grouping by wrong columns skews results.

Comparing non-consecutive months misleads trends.

Incorrect joins can inflate user counts.

Overinterpreting temporary spikes as trends.

Always validate queries before reporting results.



Best Practices for Analysis

Dashboard Metrics to Track

DAU and MAU
trend lines
showing
growth.

DAU/MAU ratio
highlighting
engagement.

Monthly growth
rate card
showing user
expansion.

Retention
percentage
table by cohort
month.

Comparison of
new vs
returning users.

Marketing
impact on DAU
growth
patterns.

Churn and
reactivation
insights.

All metrics
sourced
directly from
SQL results.



DAU and MAU metrics help evaluate campaign success.



Operations teams plan staff based on peak usage hours.



Finance uses active user data to forecast revenue.



Product managers track adoption after new releases.



Retention rates influence long-term product planning.



SQL insights inform strategy across departments.



Data-driven planning improves performance accuracy.



User metrics link directly to business outcomes.

Business Impact of Metrics

Real World Reflection- Question



What does engagement mean in your dataset?



Which metric shows performance most clearly?



How can retention improvements raise revenue?



Which SQL query was easiest to understand?



What new insight did you gain about user data?



How could these metrics apply to your projects?



Reflect on accuracy, clarity, and interpretation.



Discussion helps connect data to real-world cases.

Common Pitfalls in Analysis

Looking at metrics without time context leads to confusion.

Using too short or incomplete data periods causes bias.

Ignoring retention while focusing only on growth is misleading.

Failing to filter out inactive or test users inflates numbers.

Incorrect date joins cause errors in monthly reports.

Overinterpreting single spikes as long-term patterns.

Not validating SQL output with real business data.

Avoid these pitfalls to ensure reliable analysis results.

Ethical Use of Data



Always protect customer and user data privacy at all times.



Never expose personal or identifiable information in reports.



Aggregate and anonymize data before sharing externally.



Follow company and legal compliance guidelines strictly.



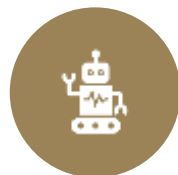
Use analytics to improve user experience, not exploit behavior.



Be transparent about how and why data is collected.



Maintain data security when storing or processing files.



Ethical handling of data builds long-term trust in analytics.

Final Reflection

-  Data becomes valuable when it guides meaningful action.
-  Engagement metrics translate raw data into business strategy.
-  Simple SQL queries can deliver professional-level insight.
-  Regular measurement ensures accuracy and progress tracking.
-  Every dataset tells a unique business story.
-  The analyst's job is to uncover and explain that story.
-  Consistency, clarity, and accuracy define great analysis.
-  Focus on insights and impact rather than just numbers.

SUMMARY

- ❑ DAU tracks short-term engagement, MAU tracks adoption.
- ❑ DAU/MAU ratio measures frequency of use.
- ❑ Growth rate shows speed of user expansion.
- ❑ Cohort analysis reveals long-term retention trends.
- ❑ SQL provides accurate and repeatable analysis.
- ❑ Metrics inform both strategy and execution.
- ❑ Continuous tracking ensures sustainable growth.
- ❑ Data interpretation is key to meaningful action.

Q&A



SQL for User Economics & Lifetime Value Insights



Day 09

Content List

- ❑ User Economics
- ❑ ARPU, churn, retention
- ❑ Data Models
- ❑ Simple SQL: revenue & ARPU :actives & churn
- ❑ Simple CLV from ARPU + churn
- ❑ Segments/cohorts & next steps

Learning Objectives

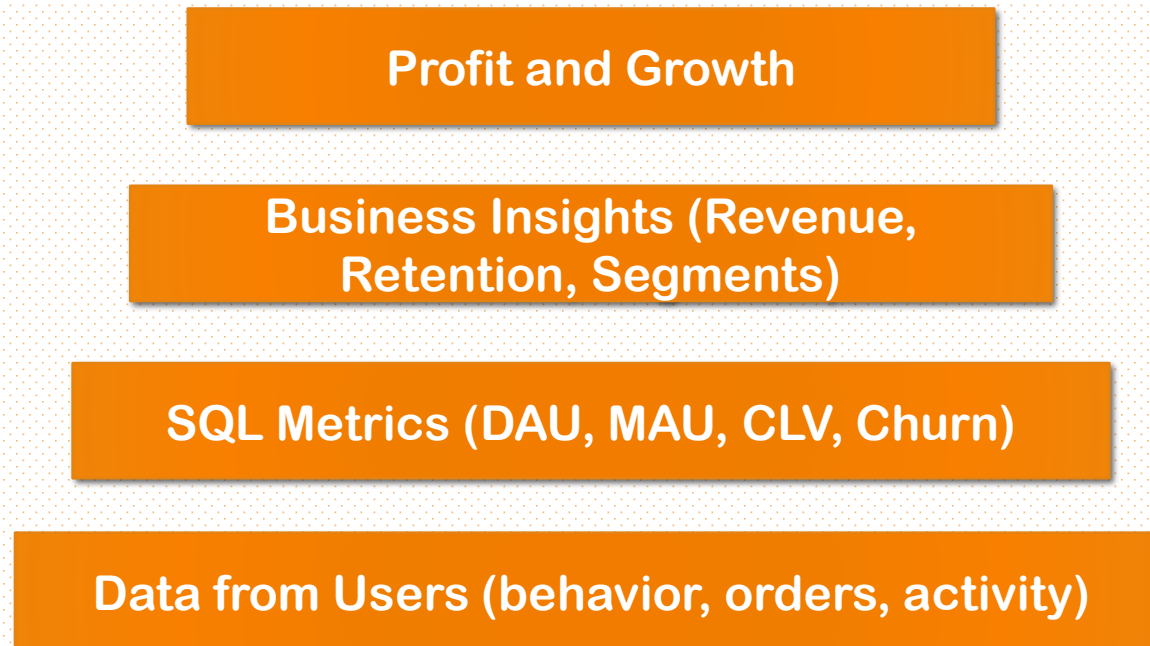
- ❑ Explain ARPU, retention, churn, CLV
- ❑ Map metrics to tables/columns
- ❑ Compute revenue per user
- ❑ Compute monthly ARPU
- ❑ Estimate churn for two months
- ❑ Turn ARPU + churn into CLV
- ❑ Interpret CLV for business insights

User Economics

Understanding User Economics

- ❑ Helps understand how user actions create business value.
- ❑ Shows where the company earns money from customers.
- ❑ Helps decide how much to spend to gain or keep users.
- ❑ Focuses on keeping existing users happy and active.
- ❑ Finds which user groups are most profitable.
- ❑ Makes it easier to track results in dashboards.
- ❑ All these insights can be measured with simple SQL queries.

User Economics: Pyramid Model



Core Terms

Term	Meaning
User	A unique customer in the database
Revenue	Total money earned from all orders
Active User	A customer who made a purchase in the selected period
ARPU	Average Revenue Per User = $\text{Total Revenue} \div \text{Active Users}$
Churn	Users who stopped being active compared to the last period
Retention	Users who stayed active compared to the last period
CLV (Customer Lifetime Value)	Total expected revenue or margin from a customer over time

Dataset BookDemo

ba.Customers(CustomerID, Segment, Region, SignupDate, IsActive)

ba.Orders(OrderID, CustomerID, OrderDate, Status)

ba.OrderItems(OrderID, ProductID, Qty, UnitPrice, LineTotal)

ba.Payments(PaymentID, OrderID, PaidAt, Amount)

We'll join Orders → OrderItems for analysis

Ignore cancelled orders

Group monthly using YEAR() and MONTH()

Practical Assumptions

Assumption	Explanation
Completed orders	Only count completed or paid orders as valid sales.
$\text{LineTotal} = \text{Qty} \times \text{UnitPrice}$	Each order line total is quantity multiplied by unit price.
$\text{Month} = \text{YEAR} + \text{MONTH}(\text{OrderDate})$	Used to group orders or users by month.
Active users	Users who placed at least one order during the month.
Churn window	Compare active users from the previous and current month.
$\text{Lifetime} \approx 1 / \text{monthly churn}$	Estimates how long a typical customer stays active.
Margin%	Optional metric to calculate net profit instead of total revenue.

Business Analysis Questions

1. How much do users spend total?
2. What is the ARPU trend monthly?
3. How many users are active per month?
4. What's the churn between two months?
5. How can we estimate CLV simply?
6. Which segments have higher CLV?
7. Which cohorts retain better over time?

Total Revenue

SQLQuery4.sql ...IA\jia_a (66))* Day-09 Demo....\jia_a (69))*

```
1  /* =====
2  PROBLEM STATEMENT:
3  Find the total company revenue for all completed orders.
4  ===== */
5
6  -- The goal is to calculate total revenue from all sales.
7  -- We use SUM(Quantity * UnitPrice) to calculate total revenue.
8
9  SELECT
10     SUM(oi.Quantity * oi.UnitPrice) AS TotalRevenue -- Total revenue for completed orders
11 FROM ba.Orders o
12 JOIN ba.OrderItems oi
13     ON oi.OrderID = o.OrderID -- Link each order with its items
14 WHERE o.OrderStatus = 'Completed'; -- Filter only completed orders
15
16 /* =====
17 EXPLANATION:
18 - SUM() multiplies Quantity and UnitPrice for each line item.
19 - JOIN connects Orders and OrderItems tables by OrderID.
20 - WHERE keeps only completed orders.
21 - The result shows total revenue for the company.
22 ===== */
23
24
```

75 % 1 0

Results Messages

	TotalRevenue
1	22697.41

Revenue per User (All Time)

```
23 -----REVENUE PER USER-----
24
25
26 /* =====
27 PROBLEM STATEMENT:
28 Show total spending (revenue) per customer across all time.
29 Sort from highest spender to lowest.
30 ===== */
31
32 SELECT
33     o.CustomerID, -- the user
34     SUM(oi.Quantity * oi.UnitPrice) AS RevenuePerUser -- total money spent by this user
35 FROM ba.Orders o
36 JOIN ba.OrderItems oi
37     ON oi.OrderID = o.OrderID -- link each order to its items
38 -- WHERE o.OrderStatus = 'Completed' -- OPTIONAL: uncomment if you added this column
39 GROUP BY
40     o.CustomerID -- one row per customer
41 ORDER BY
42     RevenuePerUser DESC; -- highest spenders first
43
44 /* =====
45 EXPLANATION:
46 - SUM(Quantity * UnitPrice) computes the value of each line item,
47   then adds them up per customer.
48 - JOIN connects Orders with OrderItems using OrderID.
49 - GROUP BY makes one row per CustomerID.
50 - ORDER BY sorts customers by their total spend (largest to smallest).
51 - If your Orders table has OrderStatus, uncomment the WHERE line to
52   include only 'Completed' orders.
53 ===== */
54
```

75 %

1

0

↑

↓

◀

Results Messages

	CustomerID	RevenuePerUser
1	144	1404.24
2	90	1400.58
3	147	1330.34
4	67	1330.27
5	84	1266.31

Monthly Revenue (Totals)

```
--
56 -----Monthly REVENUE-----
57
58 /* =====
59 PROBLEM STATEMENT:
60 Show month-by-month total company revenue.
61 Helps identify trends and seasonality in sales.
62 ===== */
63
64 SELECT
65     YEAR(o.OrderDate) AS Yr,           -- Extract year
66     MONTH(o.OrderDate) AS Mn,         -- Extract month
67     SUM(oi.Quantity * oi.UnitPrice) AS MonthlyRevenue -- Total revenue for that month
68 FROM ba.Orders o
69 JOIN ba.OrderItems oi
70     ON oi.OrderID = o.OrderID         -- Link each order with its items
71 -- WHERE o.OrderStatus = 'Completed' -- Optional: filter if you added this column
72 GROUP BY
73     YEAR(o.OrderDate), MONTH(o.OrderDate) -- Group by year and month
74 ORDER BY
75     Yr, Mn;                          -- Sort results by time
76
77 /* =====
78 EXPLANATION:
79 - YEAR() and MONTH() extract the date parts from OrderDate.
80 - SUM() adds up (Quantity * UnitPrice) for each month.
81 - GROUP BY combines all rows from the same month.
82 - ORDER BY ensures the results appear in chronological order.
83 ===== */
84
85
```

75 %

✖ 1

⚠ 0

↑

↓

◀

Results Messages

	Yr	Mn	MonthlyRevenue
1	2023	10	2678.11
2	2023	11	4305.21
3	2023	12	2643.14
4	2024	1	2504.52
5	2024	2	2368.51

Monthly Active Users (MAU)

```
84
85 -----ACTIVE USERS-----
86 /* =====
87 PROBLEM STATEMENT:
88 Find how many unique customers made purchases each month.
89 ===== */
90
91 SELECT
92     YEAR(o.OrderDate) AS Yr,           -- Extract year
93     MONTH(o.OrderDate) AS Mn,         -- Extract month
94     COUNT(DISTINCT o.CustomerID) AS ActiveUsers -- Unique customers who ordered that month
95 FROM ba.Orders o
96 -- WHERE o.OrderStatus = 'Completed' -- Optional: use if you added this column
97 GROUP BY
98     YEAR(o.OrderDate), MONTH(o.OrderDate) -- Group by year and month
99 ORDER BY
100     Yr, Mn;                          -- Show months in order
101
102 /* =====
103 EXPLANATION:
104 - COUNT(DISTINCT CustomerID) counts each customer only once per month.
105 - YEAR() and MONTH() split OrderDate into time components.
106 - GROUP BY groups orders by month for aggregation.
107 - ORDER BY arranges output chronologically.
108 ===== */
109
110
111
```

75 % 1 0

Results Messages

	Yr	Mn	ActiveUsers
1	2023	10	15
2	2023	11	21
3	2023	12	18
4	2024	1	13
5	2024	2	14

Query executed successfully. JIA\SQLEXPRESS (16)

APRU & CLV

From ARPU to Recency

So far, we've measured how much users spend using ARPU.

But business health also depends on how often users come back.

This is where Recency becomes important — it tracks when a user last made a purchase or engaged.

High recency (recent activity) means users are actively engaged.

Low recency (long gaps) signals potential churn or inactivity.

Combining ARPU + Recency helps us predict which users are still valuable.

It's the first step toward RFM (Recency, Frequency, Monetary) analysis used in marketing.

What Is ARPU?

ARPU means average money earned from each active user.

It helps measure how valuable each customer is.

Calculated as: $\text{Total Revenue} \div \text{Number of Active Users}$.

Example: If revenue = \$10,000 and active users = 500 →
ARPU = \$20.

Higher ARPU means customers are spending more.

Used to compare performance across months or user segments.

Common in apps, telecom, and online subscriptions.



Monthly ARPU

```

109 -----MONTHLY ARPU
110
111 /* =====
112 PROBLEM STATEMENT:
113 Compute Monthly ARPU (Average Revenue Per User).
114 ARPU per month = (SUM of revenue in that month) / (unique buyers in that month)
115 ===== */
116
117 SELECT
118     YEAR(o.OrderDate) AS Yr,                -- year
119     MONTH(o.OrderDate) AS Mn,              -- month
120     SUM(oi.Quantity * oi.UnitPrice) AS MonthlyRevenue, -- total revenue in month
121     COUNT(DISTINCT o.CustomerID) AS ActiveUsers,      -- unique buyers in month
122     SUM(oi.Quantity * oi.UnitPrice) * 1.0
123     / NULLIF(COUNT(DISTINCT o.CustomerID), 0) AS ARPU -- revenue ÷ users (avoid /0)
124 FROM ba.Orders o
125 JOIN ba.OrderItems oi
126     ON oi.OrderID = o.OrderID
127     -- WHERE o.OrderStatus = 'Completed' -- OPTIONAL if you added this column
128 GROUP BY YEAR(o.OrderDate), MONTH(o.OrderDate)
129 ORDER BY Yr, Mn;
130
131 /* =====
132 EXPLANATION:
133 - SUM(Quantity * UnitPrice) = revenue.
134 - COUNT(DISTINCT CustomerID) = monthly active users.
135 - NULLIF(...) prevents divide-by-zero.
136 - GROUP BY month to get one row per month.
137 ===== */
138
139

```

75 %

1

0

↑

↓

◀

Results

Messages

	Yr	Mn	MonthlyRevenue	ActiveUsers	ARPU
1	2023	10	2678.11	15	178.540666
2	2023	11	4305.21	21	205.010000
3	2023	12	2643.14	18	146.841111
4	2024	1	2504.52	13	192.655384

What Is Recency?

Recency means how recently a customer made a purchase or visited.

It tells how fresh the user's activity is.

Recent buyers are usually more interested in the product.

Older inactive users may need re-engagement.

Example: A user who bought a book 2 days ago = high recency.

A user who hasn't bought in 6 months = low recency.

Used by businesses to plan offers or reminders.

Why Measure Recency?



Recent activity predicts churn



Helps target re-engagement campaigns



Identifies loyal vs dormant customers



Simple SQL makes segmentation possible



Recency + frequency = retention metric



Improves marketing focus

What Is Lifetime Value (LTV or CLV)?



Lifetime Value means total revenue a business earns from one customer over time.



It shows how long and how much a customer contributes.



Simple formula:



$LTV = ARPU \times \text{Average Customer Lifetime (months)}$



Example: $ARPU = \$25/\text{month}$, $\text{Lifetime} = 12 \text{ months} \rightarrow LTV = \300 .



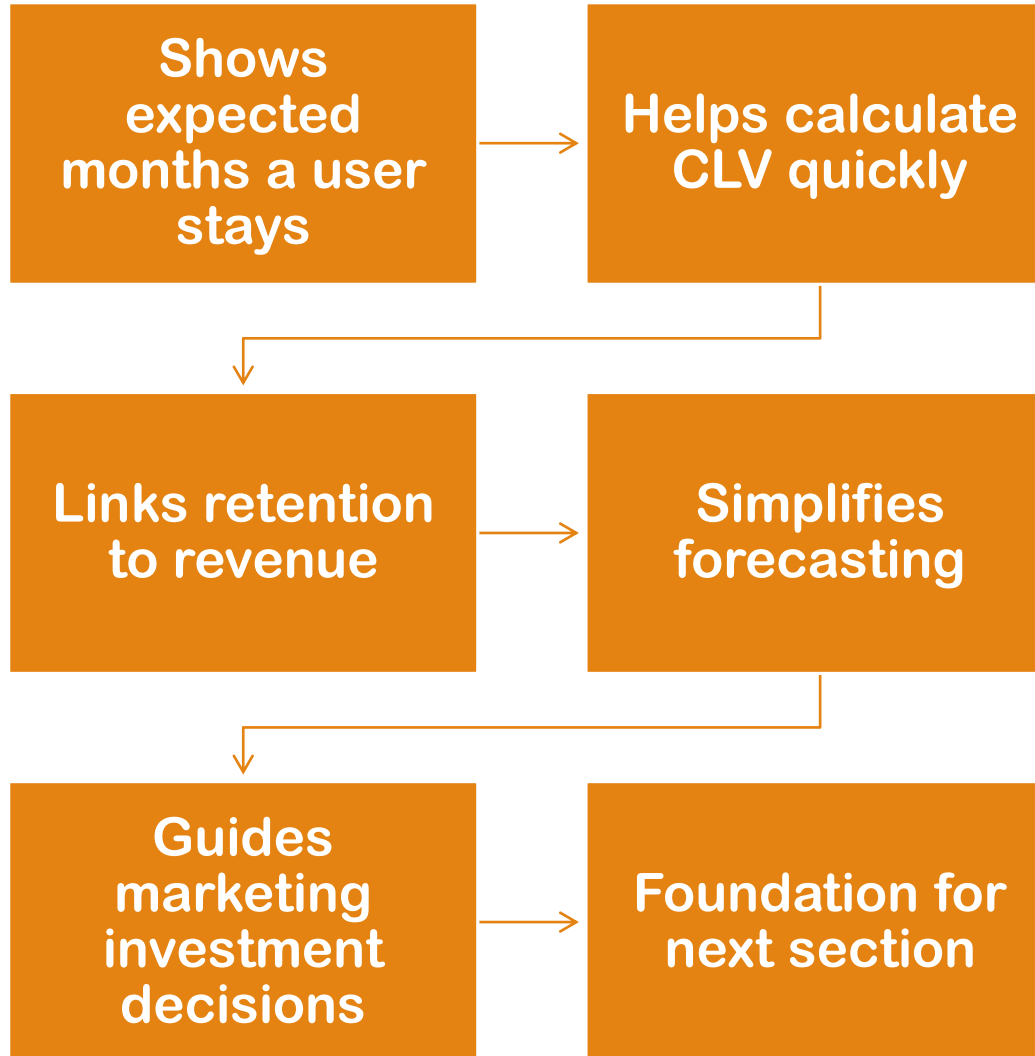
Higher LTV = loyal and profitable customers.



Helps decide how much to spend to acquire new users (CAC vs LTV).



Important for planning marketing budgets and retention.



Why Lifetime Matters

CLV Intuition

- ❑ If ARPU = \$40 per month and users stay for 10 months, CLV = \$400
- ❑ If churn improves, CLV rises directly
- ❑ If margin improves, CLV also rises
- ❑ Even small churn drops create huge CLV gains
- ❑ We can compute CLV using SQL data already built

CLV Calculation Logic

```
130
131  /* =====
132  EXPLANATION:
133  - SUM(Quantity * UnitPrice) = revenue.
134  - COUNT(DISTINCT CustomerID) = monthly active users.
135  - NULLIF(...) prevents divide-by-zero.
136  - GROUP BY month to get one row per month.
137  ===== */
138
139  /* =====
140  PROBLEM STATEMENT:
141  Calculate CLV (Customer Lifetime Value)
142  using ARPU * Lifetime * Margin%.
143  ===== */
144
145  -- Assume we already know:
146  -- ARPU = 120      -- from earlier ARPU calculation
147  -- Average Lifetime = 12 months
148  -- Profit Margin = 0.6 (60%)
149
150  SELECT
151  120 * 12 * 0.6 AS CLV;  -- = ARPU * Lifetime * Margin%
152
153  /* =====
154  EXPLANATION:
155  - ARPU = average revenue per user per month.
156  - Lifetime = average months a customer stays.
157  - Margin% = percentage of revenue that is profit.
158  - Multiply these three to estimate Customer Lifetime Value.
159  ===== */
160
```

75 % 1 0

Results Messages

Segment- Level Analysis

Different user groups have different ARPU

Churn and retention also vary by segment

Compute CLV per segment for deeper insights

Target higher CLV segments with more marketing

Focus retention programs on low-CLV groups

SQL helps reveal these patterns clearly

SQL: CLV by Segment

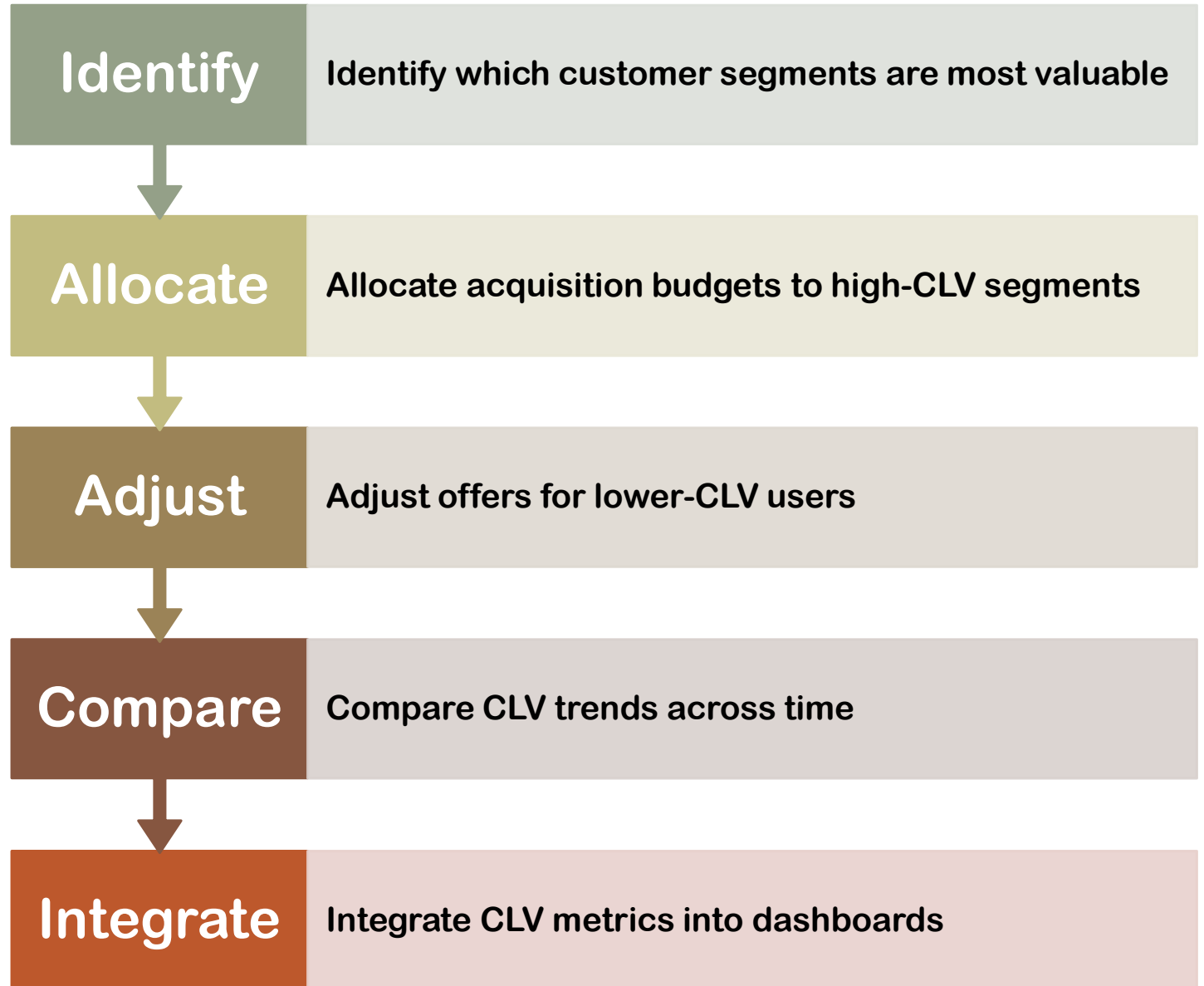
```
160
161
162 -----CLV BY SEGMENT-----
163 /*
164  PROBLEM STATEMENT:
165  Compare average revenue per user (CLV proxy) across REGIONS.
166  */
167
168 SELECT
169     c.Region,
170     AVG(x.RevenuePerUser) AS AvgRevenuePerUser
171 FROM (
172     SELECT
173         o.CustomerID,
174         SUM(oi.Quantity * oi.UnitPrice) AS RevenuePerUser
175     FROM ba.Orders o
176     JOIN ba.OrderItems oi
177     ON oi.OrderID = o.OrderID
178     GROUP BY o.CustomerID
179 ) AS x
180 JOIN ba.Customers c
181 ON c.CustomerID = x.CustomerID
182 GROUP BY c.Region
183 ORDER BY AvgRevenuePerUser DESC;
184
185 /*
186  EXPLANATION:
187  - Inner query (x): revenue per customer.
188  - Join to Customers to get Region.
189  - Group by Region to compare customer value across regions.
190  */
191
192
193
```

5 % 3 0

Results Messages

	Region	AvgRevenuePerUser
1	South	448.129777
2	North	438.518648
3	West	424.482608
4	East	357.887450

Insights from Segment CLV



Practical Interpretation of CLV



Higher CLV = higher long-term profitability



Low CLV = warning sign (poor retention or margin)



Use CLV to guide acquisition spending



CLV should exceed CAC (Customer Acquisition Cost)



Review monthly to ensure steady growth



Common Mistakes to Avoid

Using inconsistent time windows

Including cancelled or refunded orders

Not filtering fake or duplicate users

Forgetting to document margin% assumptions

Comparing CLV across different currencies

Overcomplicating models too early

Quick Wins Using CLV Insights



**FOCUS ON
SEGMENTS
WITH HIGHEST
CLV**



**REDUCE
CHURN
THROUGH
REACTIVATION
CAMPAIGNS**



**PERSONALIZE
OFFERS TO
HIGH-VALUE
USERS**



**USE CLV FOR
BUDGET
PLANNING
AND
FORECASTING**



**VISUALIZE CLV
TRENDS WITH
POWER BI
DASHBOARDS**

SUMMARY

- ❑ CLV links revenue, churn, and retention
- ❑ ARPU gives short-term value; CLV gives lifetime value
- ❑ SQL allows easy automation of these metrics
- ❑ You can track trends and test improvements
- ❑ CLV is the bridge between marketing and finance

Q&A



Revenue Segmentation & Business Insights



Day 10

List of Contents

- ❑ Why visualize data distribution
- ❑ Understanding histograms
- ❑ Creating revenue buckets
- ❑ Hands-on SQL histogram examples
- ❑ Analyzing user segments
- ❑ Business insights & next steps

Learning Objectives

- ❑ Understand purpose of histograms
- ❑ Learn grouping users by revenue
- ❑ Write SQL for distribution analysis
- ❑ Interpret results to identify high-value segments
- ❑ Apply insights to business decisions

Histogram and Bucketing

Understanding Customer Revenue



Imagine an online bookstore with thousands of customers.

Some buy occasionally; others purchase every month. The company wants to know who contributes most to total revenue.

Raw sales numbers alone don't reveal spending patterns.

A histogram helps visualize how customers fall into low, medium, or high spenders.

These insights guide discounts, loyalty offers, and marketing budgets.

Histogram vs Bar Chart

Histogram:

- ❑ Used for continuous numeric data (e.g., revenue, age).
- ❑ Bars touch each other to show continuity.
- ❑ X-axis = value ranges (bins).
- ❑ Shows data distribution (shape & skewness).

Bar Chart:

- ❑ Used for categorical data (e.g., product type, region).
- ❑ Bars separated; order not important.
- ❑ X-axis = distinct categories.
- ❑ Shows comparisons, not distribution.



Introduction to Data Distribution

Not all users spend equally.

Visualize revenue distribution to understand trends.

Histograms highlight concentration and skewness.

Example: Most users spend between \$20–\$50.

Reveals insights about loyalty and pricing.

What Is a Histogram?

Shows frequency of data values in defined ranges (bins).

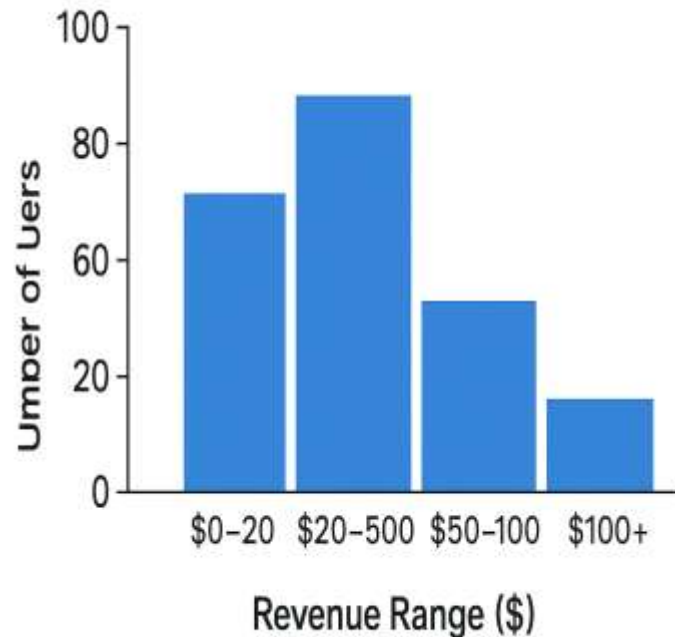
Each bar = users within revenue range.

X-axis: revenue range, Y-axis: count of users.

Highlights skewness or concentration in data.

SQL can simulate histograms using CASE + GROUP BY.

Histogram



Most users spend between \$20–\$50 (highest bar).

Fewer users spend above \$100 — shows a right-skewed pattern.

Indicates many low/moderate spenders, few high spenders.

Useful for targeting marketing or pricing strategies.

Reveals user behavior patterns and revenue concentration.

What is Bucketing?

- ❑ Bucketing means grouping continuous values into fixed ranges.
- ❑ Converts raw numbers into categories (e.g., Low, Medium, High).
- ❑ Example: Revenue
 - Low: < \$50
 - Medium: \$50–\$150
 - High: > \$150
- ❑ Makes it easier to analyze spending or activity levels.
- ❑ Commonly used in business dashboards and SQL CASE statements.
- ❑ Helps turn complex data into actionable insights.

SQL

Concept: CASE for Bucketing

```
SQLQuery1.sql... \jia_a (71))*
1  /* =====
2  PROBLEM STATEMENT
3  =====
4  Bucket customers in BooksDemo by total spending (revenue)
5  to label them as Low (<$50), Medium ($50-$150), or High (>$150)
6  ===== */
7
8  -- ☒ Bucket customers by total revenue (BooksDemo.ba)
9  SELECT
10     c.CustomerID,
11     SUM(oi.Quantity * oi.UnitPrice) AS TotalRevenue, -- calculate revenue per customer
12     CASE
13         WHEN SUM(oi.Quantity * oi.UnitPrice) < 50 THEN 'Low'
14         WHEN SUM(oi.Quantity * oi.UnitPrice) BETWEEN 50 AND 150 THEN 'Medium'
15         ELSE 'High'
16     END AS RevenueBucket
17 FROM BooksDemo.ba.Customers AS c
18 JOIN BooksDemo.ba.Orders AS o
19     ON o.CustomerID = c.CustomerID
20 JOIN BooksDemo.ba.OrderItems AS oi
21     ON oi.OrderID = o.OrderID
22 GROUP BY c.CustomerID
23 ORDER BY TotalRevenue DESC;
24
25  /* =====
26  EXPLANATION
27  =====
28  1) SUM(oi.Quantity * oi.UnitPrice): computes each customer's total spend.
29  2) CASE: assigns Low/Medium/High buckets based on revenue.
30  3) JOIN: connects Customers → Orders → OrderItems.
31  4) GROUP BY: one row per customer.
32  5) ORDER BY: shows biggest spenders first.
```

74 % No issues found

Results Messages

	CustomerID	TotalRevenue	RevenueBucket
1	144	1404.24	High
2	90	1400.58	High
3	147	1330.34	High
4	67	1330.27	High
5	84	1266.31	High

Hands-On Goal

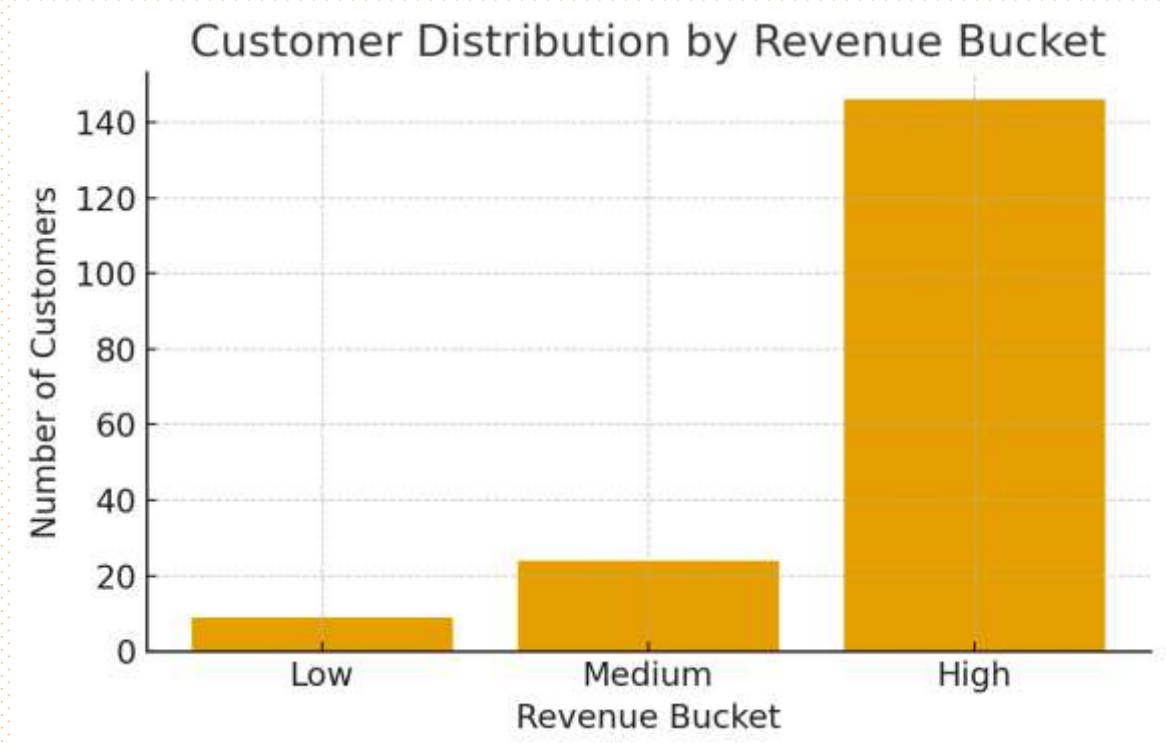
Objective: Build histogram of user revenues in SQL.

Tasks:

1. Aggregate revenue per user.
2. Create buckets with CASE.
3. Count users per bucket.
4. Visualize results in Power BI or Excel.

Sample-Visualization Histogram

- ❑ Majority of customers fall into the High revenue group (146 customers).
- ❑ Medium spenders (24) form a smaller mid-tier segment.
- ❑ Very few Low spenders (9) contribute little to total revenue.
- ❑ Indicates a skewed distribution — a few customers drive most of the business.
- ❑ Useful for targeted marketing and customer segmentation strategies.



```

84  ----- */
85
86  -----DYNAMIC BUCKET QUERY -----
87  /* -----
88  PROBLEM STATEMENT:
89  Create equal-size revenue buckets and count customers
90  in each bucket for quick analysis.
91  ----- */
92
93  -- Step 1: Calculate total revenue per customer
94  WITH CustomerTotals AS (
95      SELECT
96          c.CustomerID,
97          SUM(oi.Quantity * oi.UnitPrice) AS TotalRevenue
98      FROM ba.Customers AS c
99      JOIN ba.Orders AS o ON o.CustomerID = c.CustomerID
100     JOIN ba.OrderItems AS oi ON oi.OrderID = o.OrderID
101     GROUP BY c.CustomerID
102 )

```

```

103
104  -- Step 2: Bucket customers dynamically into 5 equal ranges
105  SELECT
106      CASE
107          WHEN TotalRevenue < 100 THEN '0 - 99'
108          WHEN TotalRevenue < 200 THEN '100 - 199'
109          WHEN TotalRevenue < 300 THEN '200 - 299'
110          WHEN TotalRevenue < 400 THEN '300 - 399'
111          ELSE '400+'
112      END AS RevenueBucket,
113      COUNT(*) AS Customers
114  FROM CustomerTotals
115  GROUP BY
116      CASE
117          WHEN TotalRevenue < 100 THEN '0 - 99'
118          WHEN TotalRevenue < 200 THEN '100 - 199'
119          WHEN TotalRevenue < 300 THEN '200 - 299'
120          WHEN TotalRevenue < 400 THEN '300 - 399'
121          ELSE '400+'
122      END
123  ORDER BY RevenueBucket;

```

```

125  /* -----
126  EXPLANATION:
127  10 SUM(Quantity * UnitPrice) + total revenue per customer.
128  20 CASE creates 5 simple revenue ranges (equal-width bins).
129  30 COUNT(*) + number of customers per range.
130  40 Easy to visualize in Excel or Power BI as histogram.
131  ----- */
132
133
134

```

74 % ✓ No issues found

Results Messages

	RevenueBucket	Customers
1	0 - 99	22
2	100 - 199	27
3	200 - 299	31
4	300 - 399	26

Dynamic Bucketing


```

133 -----QUARTILE BUCKETING-----
134 /*
135 PROBLEM STATEMENT
136
137 In BooksDemo, split customers into 4 quartiles (25% groups)
138 by TotalRevenue, then count customers per quartile (histogram-ready).
139 -----*/
140 WITH CustomerTotals AS (           -- total spend per customer
141     SELECT
142         c.CustomerID,
143         SUM(oi.Quantity * oi.UnitPrice) AS TotalRevenue
144     FROM ba.Customers AS c
145     JOIN ba.Orders AS o ON o.CustomerID = c.CustomerID
146     JOIN ba.OrderItems AS oi ON oi.OrderID = o.OrderID
147     GROUP BY c.CustomerID
148 ),
149 Quartiled AS (                     -- assign quartile; tie-break with CustomerID
150     SELECT
151         CustomerID,
152         TotalRevenue,
153         NTILE(4) OVER (ORDER BY TotalRevenue, CustomerID) AS Quartile
154     FROM CustomerTotals
155 )
156 SELECT
157     Quartile,
158     CASE Quartile
159         WHEN 1 THEN 'Q1 (lowest 25%)'
160         WHEN 2 THEN 'Q2'
161         WHEN 3 THEN 'Q3'
162         WHEN 4 THEN 'Q4 (top 25%)'
163     END AS QuartileLabel,
164     COUNT(*) AS Customers
165 FROM Quartiled
166 GROUP BY Quartile
167 ORDER BY Quartile;
168 /*
169 EXPLANATION
170
171 1) CustomerTotals: SUM(Quantity*UnitPrice) + total revenue per customer.
172 2) NTILE(4) OVER (ORDER BY TotalRevenue, CustomerID) splits customers
173    into 4 equal-sized groups; CustomerID breaks ties.
174 3) Final SELECT counts customers in each quartile for a clean screenshot.
175 -----*/

```

66 % 1 0 ↑ ↓

Results Messages

	Quartile	QuartileLabel	Customers
1	1	Q1 (lowest 25%)	45
2	2	Q2	45
3	3	Q3	45
4	4	Q4 (top 25%)	44

Quartile Bucketing

Skewness and Business Cases

What is Skewness?

Skewness measures how data values are distributed around the mean.

A symmetric (normal) distribution has equal spread on both sides.

A right-skewed (positive skew) curve has a long tail to the right — few very high values.

A left-skewed (negative skew) curve has a long tail to the left — few very low values.

Skewness helps reveal spending imbalance or outlier behavior.

Useful in business to detect dominant spenders or low-activity users.

Identifying Skewness

Right skew = few high spenders dominate.



Left skew = majority spend high.



Balanced = normal distribution.



Use skewness to target loyalty campaigns.

Business Use Cases



Identify premium, mid-range, and budget user segments.



Adjust pricing tiers based on spending behavior.



Spot loyalty program candidates among top spenders.



Detect at-risk users with declining purchase patterns.



Plan tiered marketing campaigns for each segment.



Use insights to improve revenue forecasting and customer retention.

Business Use Cases of Revenue Insights

Understand	Understand customer value tiers — premium, mid, and budget users.
Refine	Refine pricing strategy by aligning plans with user spending power.
Identify	Identify loyalty program opportunities to retain top spenders.
Detect	Detect inactive or at-risk users and plan re-engagement offers.
Support	Support marketing segmentation for personalized campaigns.
Enable	Enable data-driven decisions for product, pricing, and promotions.

Cumulative Distribution

```
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
```

```
-----CUMULATIVE DISTRIBUTION-----
/* =====
PROBLEM
Build a cumulative distribution of customers across
revenue buckets (Low < $50, Medium $50-$150, High > $150).
Return bucket counts + running (cumulative) %.
===== */
-- One compact query: per-customer totals + bucket + counts + cumulative %
SELECT
  CHOOSE(BucketOrder, 'Low', 'Medium', 'High') AS RevenueBucket, -- label
  COUNT(*) AS UserCount,
  SUM(COUNT(*)) OVER (ORDER BY BucketOrder) AS CumulativeUsers,
  CAST(
    100.0 * SUM(COUNT(*)) OVER (ORDER BY BucketOrder)
    / NULLIF(SUM(COUNT(*)) OVER (), 0)
    AS DECIMAL(6,2)
  ) AS CumulativePercent
FROM (
  -- Per-customer revenue, then map to a numeric bucket (1..3)
  SELECT
    CASE
      WHEN SUM(oi.Quantity * oi.UnitPrice) < 50 THEN 1
      WHEN SUM(oi.Quantity * oi.UnitPrice) <= 150 THEN 2 -- inclusive
      ELSE 3
    END AS BucketOrder
  FROM ba.Customers c
  JOIN ba.Orders o ON o.CustomerID = c.CustomerID
  JOIN ba.OrderItems oi ON oi.OrderID = o.OrderID
  GROUP BY c.CustomerID
) t
GROUP BY BucketOrder
ORDER BY BucketOrder;

/* =====
EXPLANATION
- Inner query: SUM(Quantity*UnitPrice) per customer, then
CASE -> 1=Low, 2=Medium, 3=High.
- Outer query: COUNT(*) per bucket.
- Window SUM over BucketOrder gives cumulative users & %,
=====
```

60 %

1 0

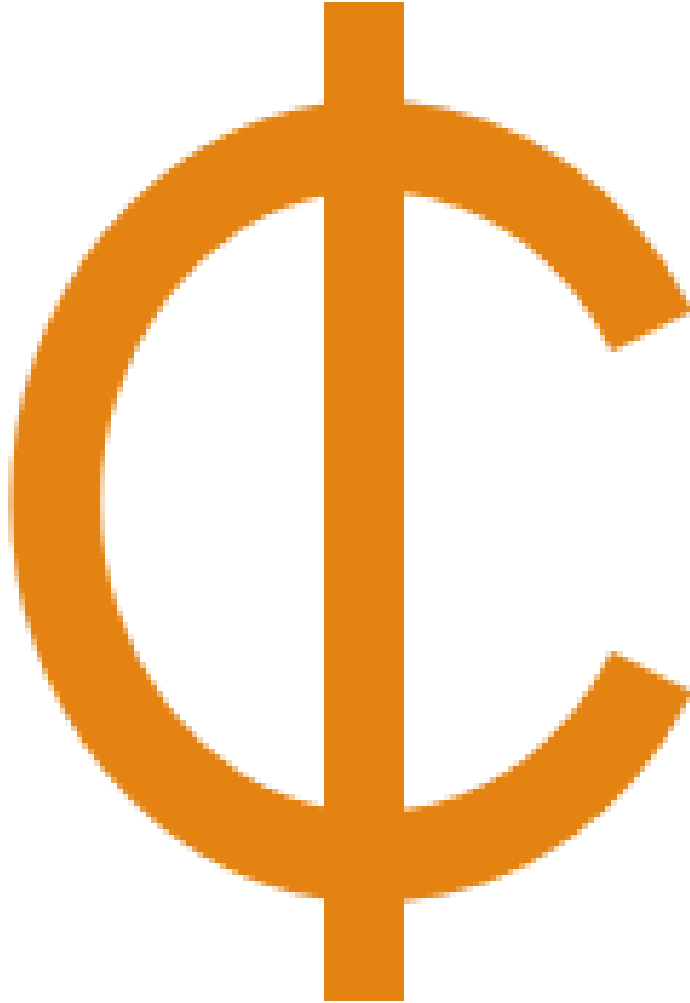
Results Messages

	RevenueBucket	UserCount	CumulativeUsers	CumulativePercent
1	Low	9	9	5.03
2	Medium	24	33	18.44
3	High	146	179	100.00

Discussion: Insights

- ❑ Which revenue bucket has the largest share of users?
- ❑ Does the user count align with the revenue contribution?
- ❑ Which segment shows the highest customer value?
- ❑ What business strategies can be applied for each group?
- ❑ How can marketing, pricing, or loyalty plans differ by bucket?
- ❑ Share insights and reflect on what the data reveals about user behavior.





Connecting to Previous Topics

- ❑ Previously, we explored ARPU (Average Revenue per User) and CLV (Customer Lifetime Value).
- ❑ Both metrics quantify customer worth over time.
- ❑ The histogram now adds a visual layer to understand spending distribution.
- ❑ Together, they provide a complete customer value picture — who buys, how often, and how much.
- ❑ This connection helps identify key segments to target for growth and retention.

Common SQL Mistakes

01

Missing
GROUP BY
with aggregate
functions.

02

Overlapping
CASE ranges
causing double
counting.

03

Using wrong
data types for
numeric fields.

04

Ignoring **NULL**
or zero values
in calculations.

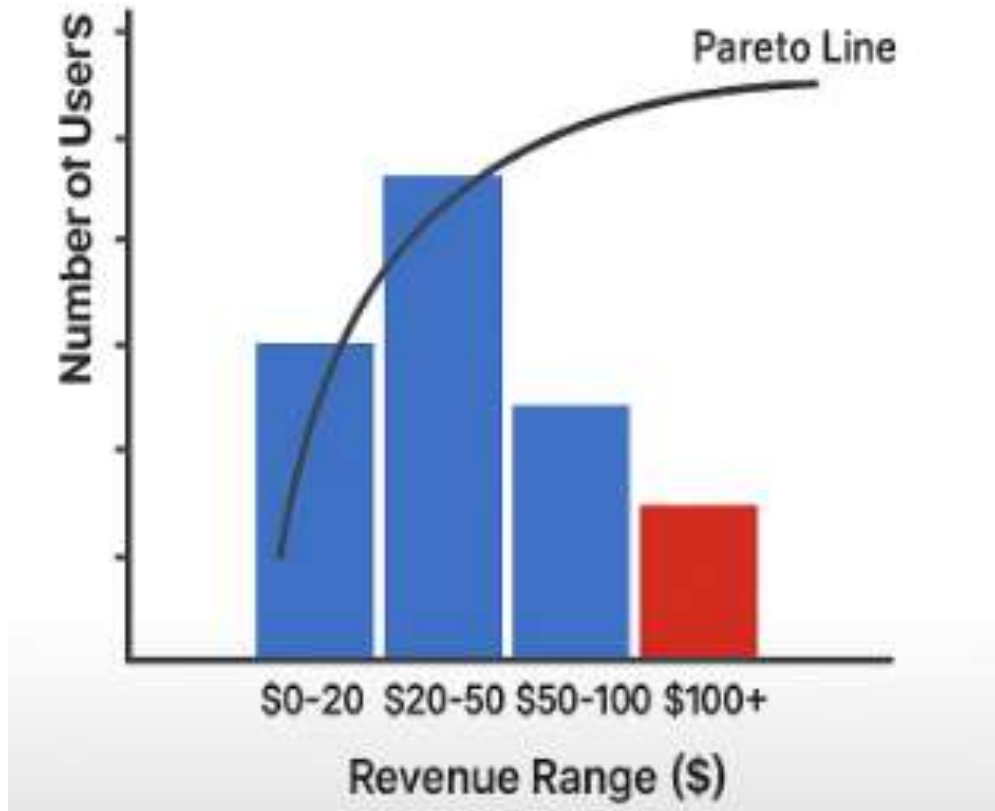
05

Missing aliases
for subqueries
or columns.

Debugging Tips for SQL Bucketing

- 🔍 Use `SELECT DISTINCT` to confirm unique bucket names.
- 📊 Apply `ORDER BY TotalRevenue` to verify sorting accuracy.
- ✓ Check that every user is assigned to a bucket (no NULLs).
- ⚙️ Test with a small dataset before running full query.
- 📈 Compare totals with the original table to ensure no data loss.
- 🔗 Review CASE logic boundaries to avoid overlaps.

Visual Tip: Histogram Chart



Plot bucket (e.g., revenue range) on the X-axis and user count on the Y-axis.

Use distinct colors to highlight key groups (e.g., high spenders in red).

Add a Pareto line or label to show the top 20% users contributing 80% of revenue.

Include axis labels and titles for readability.

Keep design clean — avoid 3D or unnecessary effects.

Helps students see skewness and spending concentration clearly.

Beyond Revenue Buckets

Bucketing isn't just for revenue — it can apply to any metric.

Orders by Frequency: Identify occasional vs. loyal customers.

Customers by Recency: See who purchased recently vs. long ago.

Products by Profit Margin: Find top-performing vs. low-margin items.

Use the same SQL CASE logic to create these segments.

Helps build RFM (Recency, Frequency, Monetary) models in analytics.

Business Decision Matrix

Revenue	Frequency	Customer Type	Recommended Action
High	Frequent	Loyal / VIP	Offer loyalty rewards, early access, exclusive deals
High	Infrequent	Premium but inactive	Send re-engagement offers or reminders
Low	Frequent	Engaged but low spenders	Upsell or bundle products to increase spend
Low	Infrequent	Dormant users	Send win-back campaigns or discounts

```

1  /*
2  PROBLEM (easy):
3  Count customers in revenue buckets (incl. Zero/Null) and
4  sort the result as Zero/Dormant → Low → Medium → High.
5  */
6
7  USE BooksDemo;
8  GO
9
10 -- One clean query: totals → bucket labels + sort key + counts
11 SELECT
12     x.RevenueBucket,
13     COUNT(*) AS Customers
14 FROM (
15     SELECT
16         -- map each customer to a label and a numeric sort key
17         CASE
18             WHEN ISNULL(t.TotalRevenue, 0) = 0 THEN 'Zero/Dormant'
19             WHEN t.TotalRevenue < 50      THEN 'Low'
20             WHEN t.TotalRevenue <= 150    THEN 'Medium'    -- inclusive
21             ELSE 'High'
22         END AS RevenueBucket,
23         CASE
24             WHEN ISNULL(t.TotalRevenue, 0) = 0 THEN 0
25             WHEN t.TotalRevenue < 50      THEN 1
26             WHEN t.TotalRevenue <= 150    THEN 2
27             ELSE 3
28         END AS BucketOrder
29     FROM ba.Customers c
30     LEFT JOIN (

```

```

31         SELECT o.CustomerID,
32                SUM(oi.Quantity * oi.UnitPrice) AS TotalRevenue
33         FROM ba.Orders o
34         LEFT JOIN ba.OrderItems oi ON oi.OrderID = o.OrderID
35         GROUP BY o.CustomerID
36     ) t ON t.CustomerID = c.CustomerID
37 ) x
38 GROUP BY x.RevenueBucket, x.BucketOrder
39 ORDER BY x.BucketOrder;
40
41 /*
42 EXPLANATION
43
44 - Inner SELECT: compute TotalRevenue per customer, then assign
45   a bucket label and a numeric BucketOrder.
46 - Outer SELECT: COUNT(*) per bucket; ORDER BY BucketOrder avoids
47   using non-grouped columns in ORDER BY (fixes your error).
48 - Buckets are non-overlapping: 0 → Zero/Dormant, <50 → Low,
49   50-150 → Medium, >150 → High.
50 */
51

```

60 %



1



0



Results

Messages

	RevenueBucket	Customers
1	Zero/Dormant	21
2	Low	9
3	Medium	24
4	High	146

Handling Zero/Null Revenue Users

Visualizing Buckets in the Right Order

- ❑ Always display buckets in logical order: Low → Medium → High.
- ❑ Avoid alphabetical sorting (which may mix up levels).
- ❑ Use a numeric sort key or CASE statement to control order.

Example SQL:

```
ORDER BY CASE
```

```
    WHEN RevenueBucket = 'Low' THEN 1
```

```
    WHEN RevenueBucket = 'Medium' THEN 2
```

```
    WHEN RevenueBucket = 'High' THEN 3
```

```
END;
```

- ❑ Ensures charts read naturally, like a histogram, not random bars.
- ❑ Makes data easier to interpret and present in dashboards.

SUMMARY

- ❑ Learned how to visualize user revenue distribution using histograms.
- ❑ Understood the concept of bucketing – grouping continuous values into categories.
- ❑ Created Low, Medium, and High revenue segments using SQL CASE statements.
- ❑ Explored how ordering buckets correctly improves readability and insight.
- ❑ Connected bucketing to business decision-making (e.g., identifying VIP or dormant users).

Q&A

